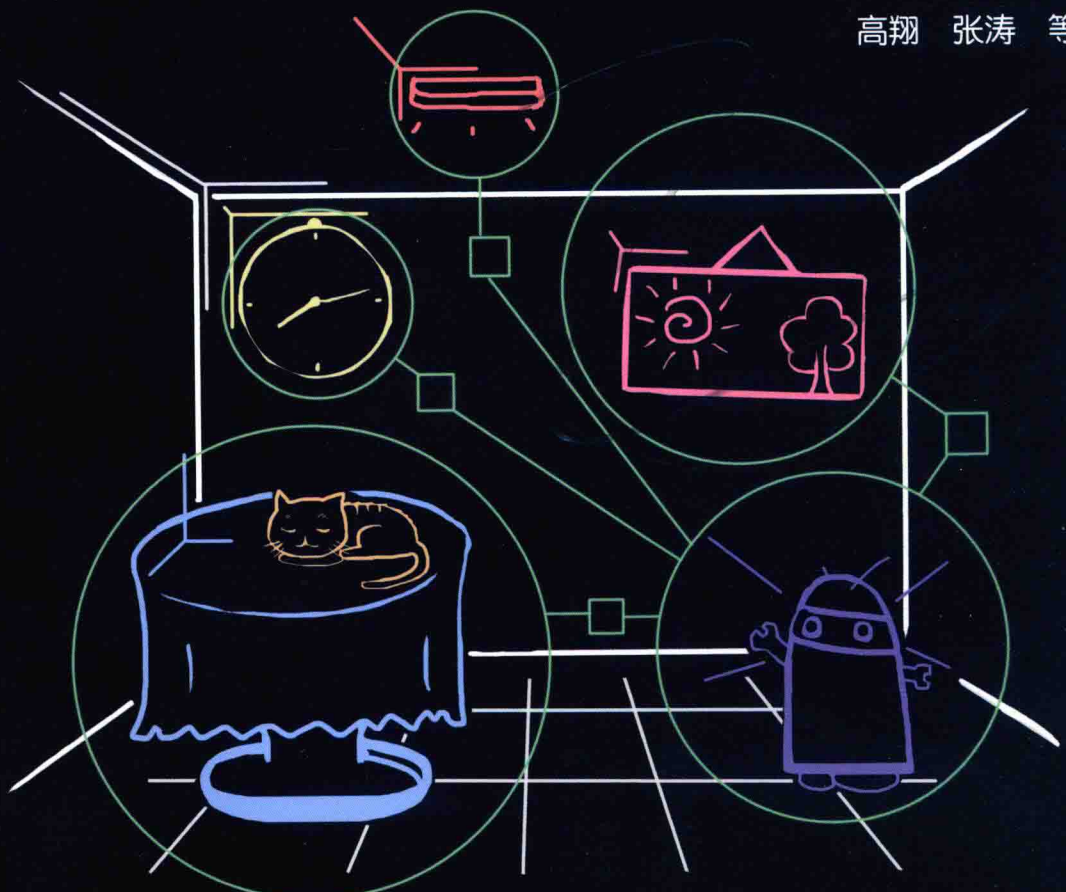
Broadview®
www.broadview.com.cn

视觉SLAM十四讲

从理论到实践

第2版

高翔　张涛　等著



中国工信出版集团
電子工業出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
http://www.phei.com.cn

Broadview®
www.broadview.com.cn
博文视点·IT出版旗舰品牌
技术凝聚实力·专业创新出版

隆重向读者力荐《视觉SLAM十四讲：从理论到实践（第2版）》。一方面，本书是业界少有的涵盖从基础理论到代码实例，系统性讲解SLAM的书；另一方面，本书的作者和地平线颇有渊源，高翔曾经是我们的算法实习生，颜沁睿是我们的工程师，都是在SLAM领域非常杰出的青年专家，走在技术实践前沿。在移动互联网大潮之后，自动驾驶、无人机、服务机器人等人工智能硬件会成为下一个产业爆发点，其中关键的技术之一就是动态定位和环境建模的SLAM技术。本书是国内非常有价值的有关SLAM技术的书籍，适合有志于从事机器人技术的研究生和工程师阅读，一定会让读者很有收获。

—— 地平线公司创始人，中国人工智能学会副秘书长，

我在新加坡和加拿大给学生讲视觉SLAM时常常觉得缺乏一本适合初学者的教材。高翔博士的《视觉SLAM十四讲：从理论到实践（第2版）》从基础的四元数、李代数讲起，涵盖了卡尔曼滤波、Bundle Adjustment、位姿图等高级优化工具。书中更有最近十多年成功系统的概述，从2003年的MonoSLAM直到2016年的ORB-SLAM。通篇既有清晰的理论叙述，又辅以大量示例程序，是一本非常好的视觉SLAM教材。

—— 加拿大西蒙弗雷泽大学终身教授，谭平

视觉SLAM随着近年增强现实、无人驾驶等应用的兴起而重新获得重大关注。视觉SLAM属于计算机视觉和机器人研究的交叉领域，因此涉及的基础知识广而分散。国内专门的研究机构相对较少，学生入门的门槛较高。幸运的是，本书不仅有深入浅出的讲解，同时注重理论和实战的结合，大大降低了国内学生和相关从业者的进入门槛。因此，本书非常值得初学者学习实践。

—— 杭州易现先进科技 AR算法总监，刘海伟

SLAM技术是移动机器人实现真正自主的核心技术之一。高翔博士的《视觉SLAM十四讲：从理论到实践（第2版）》详细地介绍了经典视觉SLAM算法涉及的数学基础、理论框架和编程知识，是一本难得的视觉SLAM专业基础教程。

—— 东北大学机器人科学与工程学院博士生导师，方正

博文视点Broadview
新浪微博 weibo.com
@博文视点Broadview

责任编辑：郑柳洁
封面设计：吴海燕

反馈意见或投稿
邮箱：zhenglj@phei.com.cn
微信号：Alinamercy

上架建议：人工智能

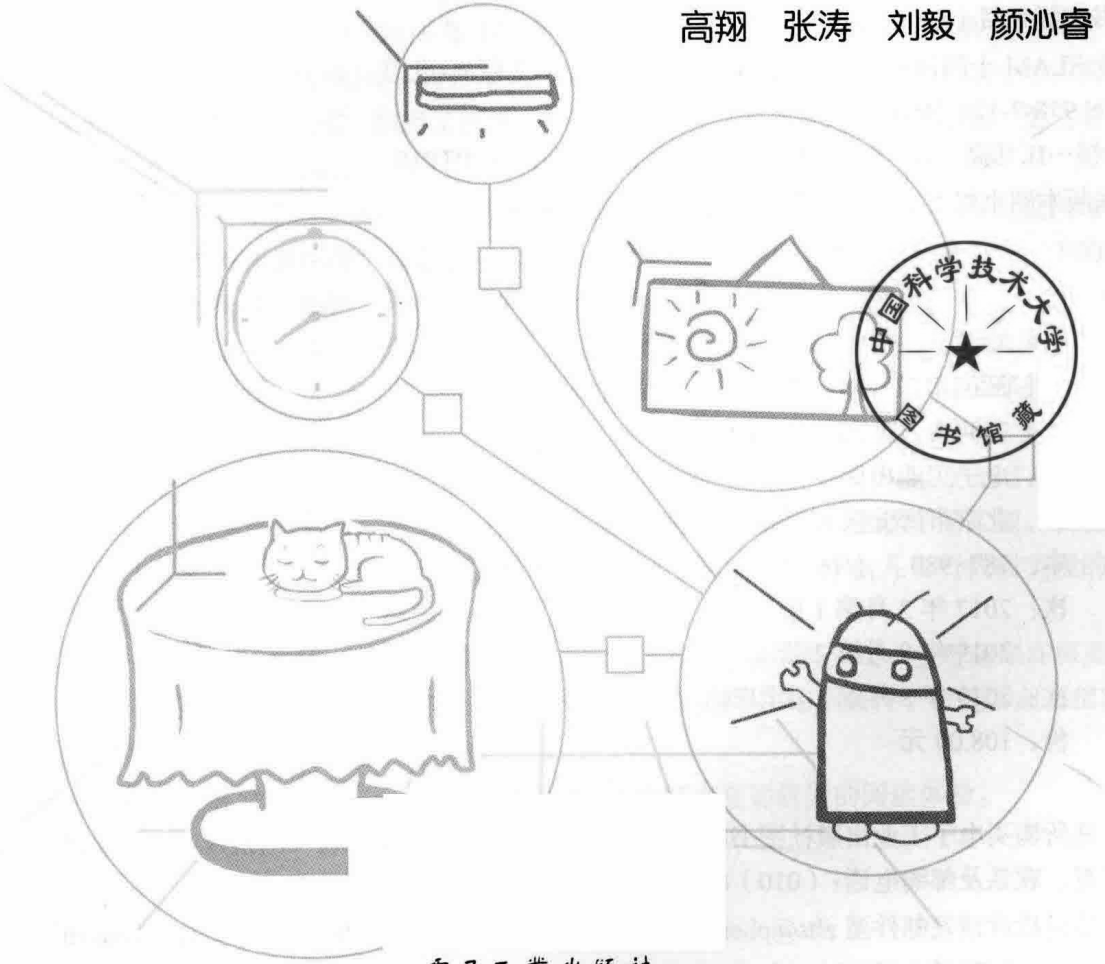
ISBN 978-7-121-36942-1
9 787121 369421 >
定价：108.00元

视觉SLAM十四讲

从理论到实践

第2版

高翔　张涛　刘毅　颜沁睿　著



中国科学技术大学
图书馆藏

電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

内容简介

本书系统介绍了视觉 SLAM（同时定位与地图构建）所需的基本知识与核心算法，既包括数学理论基础，如三维空间的刚体运动、非线性优化，又包括计算机视觉的算法实现，如多视图几何、回环检测等。此外，本书还提供了大量的实例代码供读者学习研究，从而更深入地掌握这些内容。

本书可以作为对 SLAM 感兴趣的研究人员的入门自学材料，也可以作为高等院校相关专业的本科生或研究生教材。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。
版权所有，侵权必究。

图书在版编目（CIP）数据

视觉 SLAM 十四讲：从理论到实践 / 高翔等著. — 2 版. —北京：电子工业出版社，2019.8
ISBN 978-7-121-36942-1
I. ①视…II. ①高…III. ①人工智能—视觉跟踪—研究 IV. ①TP18
中国版本图书馆 CIP 数据核字（2019）第 125604 号

责任编辑：郑柳洁
印　　刷：三河市君旺印务有限公司
装　　订：三河市君旺印务有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路 173 信箱　　邮编：100036
开　　本：787×980　1/16　　印张：25.5　　字数：576.8 千字　　彩插：4
版　　次：2017 年 3 月第 1 版
　　2019 年 8 月第 2 版
印　　次：2019 年 8 月第 1 次印刷
定　　价：108.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888，88258888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

本书咨询联系方式：（010）51260888-819，faq@phei.com.cn。

作者简介

·高翔·

清华大学自动化系博士，慕尼黑工业大学博士后。研究兴趣主要为计算机视觉、定位与建图、机器学习等，主要著、译作包括《视觉 SLAM 十四讲：从理论到实践》《机器人学中的状态估计》，在 *RAS*、*Auto Robotics*、IROS 等期刊和会议上发表论文，现从事自动驾驶车辆研发工作。

·张涛·

清华大学自动化系教授、系主任。1995年9月至1999年9月在清华大学自动化系检测技术与自动化装置专业学习，获博士学位。1999年10月至2002年9月在日本国立佐贺大学大学院工学系研究科系统控制专业学习，获博士学位。研究方向包括机器人、控制理论、人工智能等。

·刘毅·

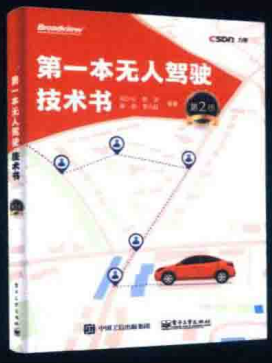
华中科技大学人工智能与自动化学院博士。长期致力于计算机视觉、三维重建、视觉SLAM、传感器融合的研究和应用。现于广州视源电子科技有限公司（CVTE）的中央研究院担任高级研究员。

·颜沁睿·

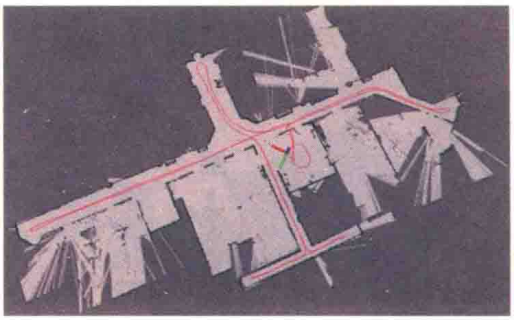
比利时荷语鲁汶大学（KU Leuven）人工智能专业硕士。致力于推动计算机视觉技术在智能驾驶领域的产品化演进，对视觉SLAM算法进行系统异构化加速。曾带领团队在FPGA平台上实现能够实时半稠密建图的单目SLAM系统，被英特尔公司选为官方示范应用案例。现担任地平线公司地图定位系统架构师。

好书分享

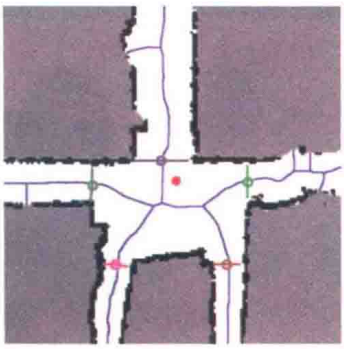
《机器人感知：因子图在SLAM中的应用》
ISBN 978-7-121-33811-3
[美] Frank Dellaert　Michael Kaess　著
刘富强　董靖　译
定价：59.00元



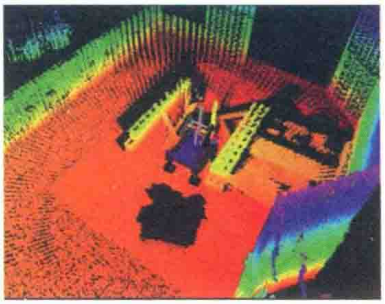
《第一本无人驾驶技术书（第2版）》
ISBN 978-7-121-36493-8
刘少山　唐洁　吴双　李力耘　等著



2D栅格地图



2D拓扑地



3D点云地图

3D网格地图

图2-10　形形色色的地图

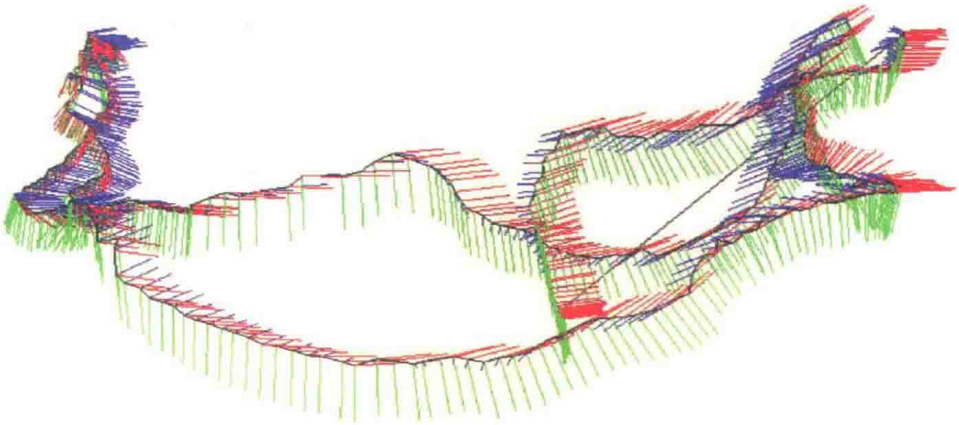


图3-3　位姿可视化的结果

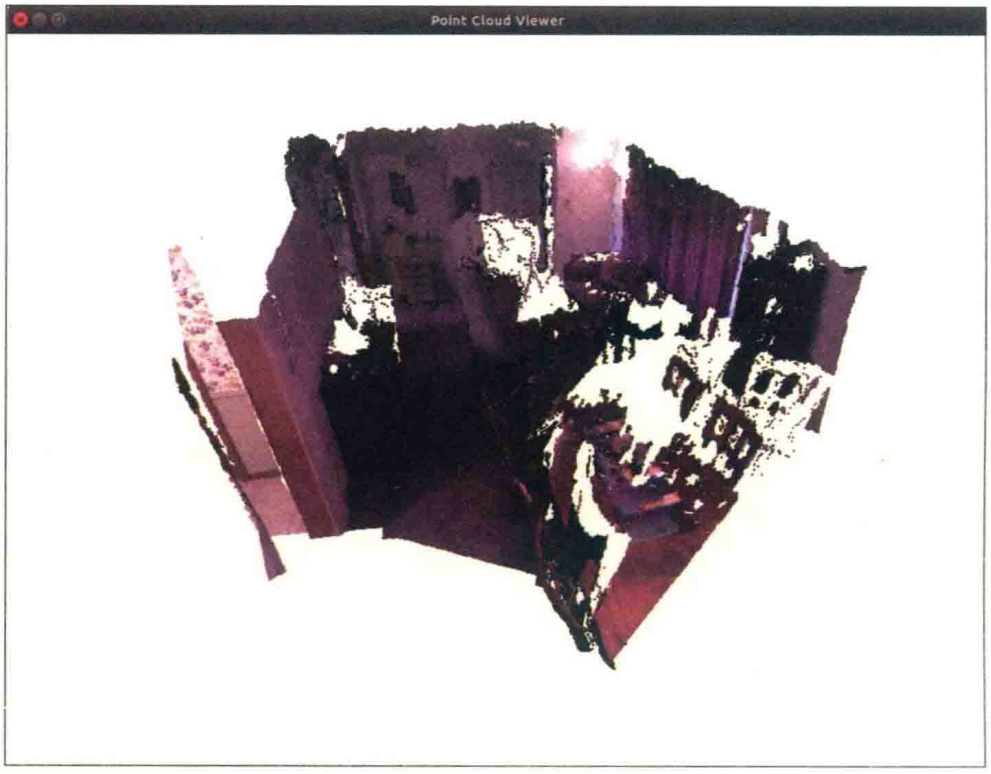


图5-10　拼合的点云地图

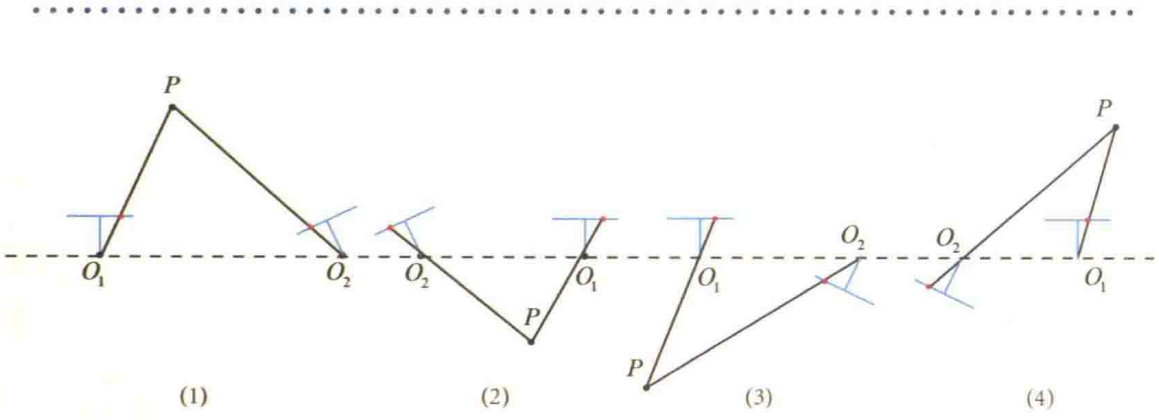


图7-10　分解本质矩阵得到的4个解。在保持投影点（红色点）不变的情况下，两个相机及空间点一共有4种可能的情况

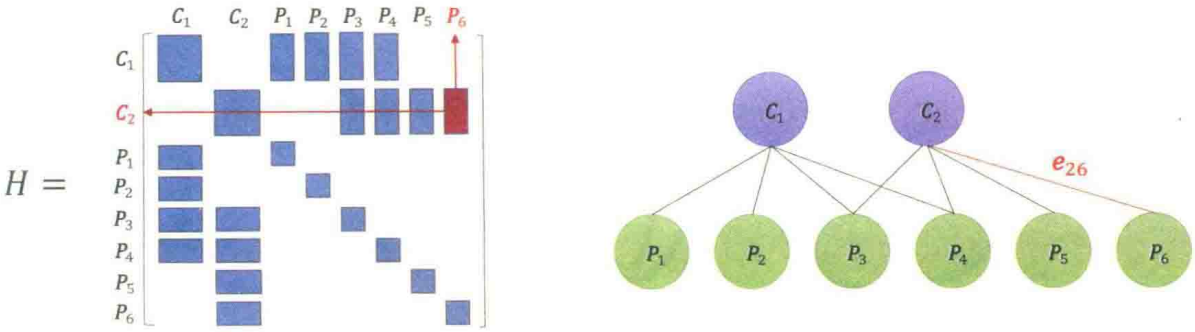
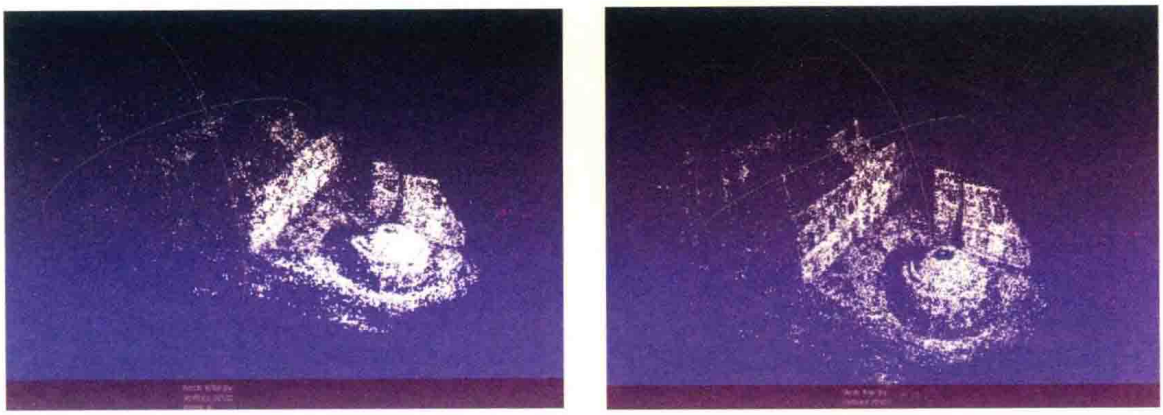


图9-7　H矩阵中非零矩阵块和图中边的对应关系。如左图H矩阵中右侧的红色矩阵块，表示在右图中其对应的变量C_2和P_6之间存在一条边e_{26}



（a）　　（b）

图9-13　优化前后的可视化点云。（a）为优化前的初始值；（b）为优化后的优化值

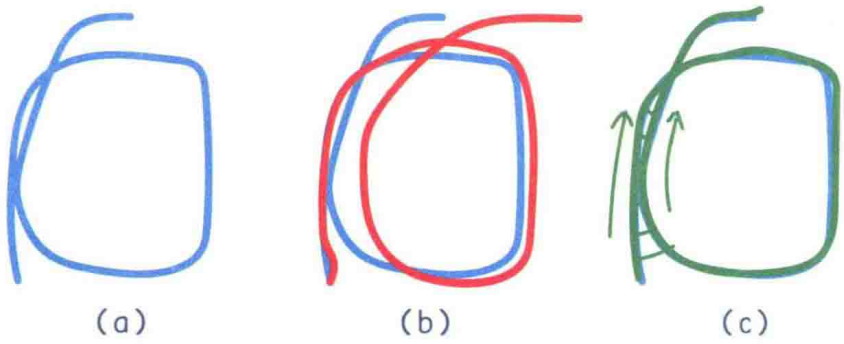


图11-1　漂移示意图。（a）真实轨迹；（b）由于前端只给出相邻帧间的估计，优化后的位姿图出现漂移；（c）添加回环检测后的位姿图可以消除累积误差

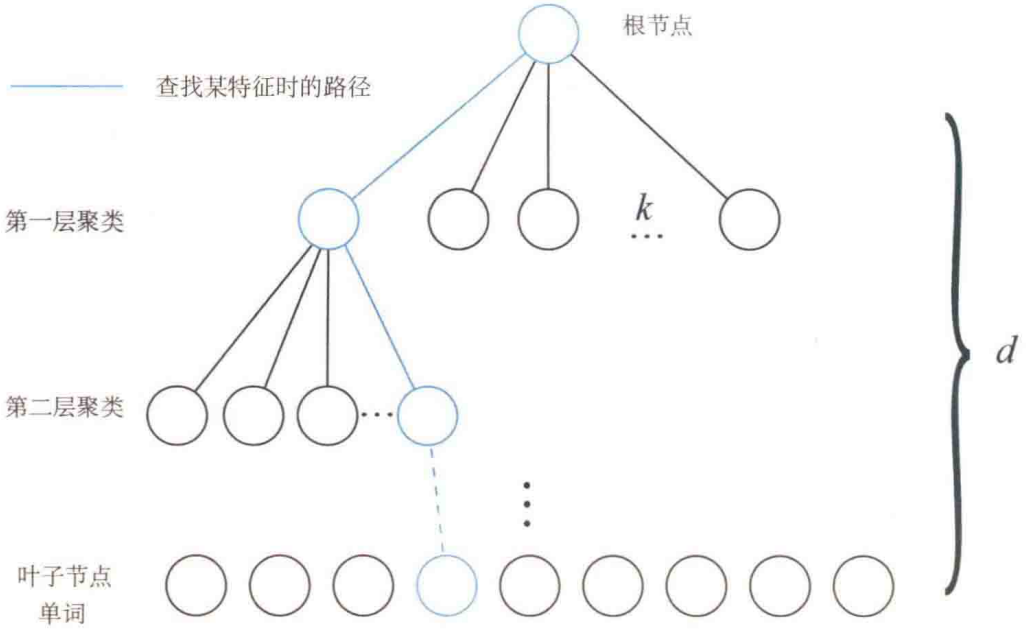


图11-4　*K*叉树字典示意图。训练字典时，逐层使用K-means 聚类。根据已知特征查找单词时，可逐层比对，找到对应的单词

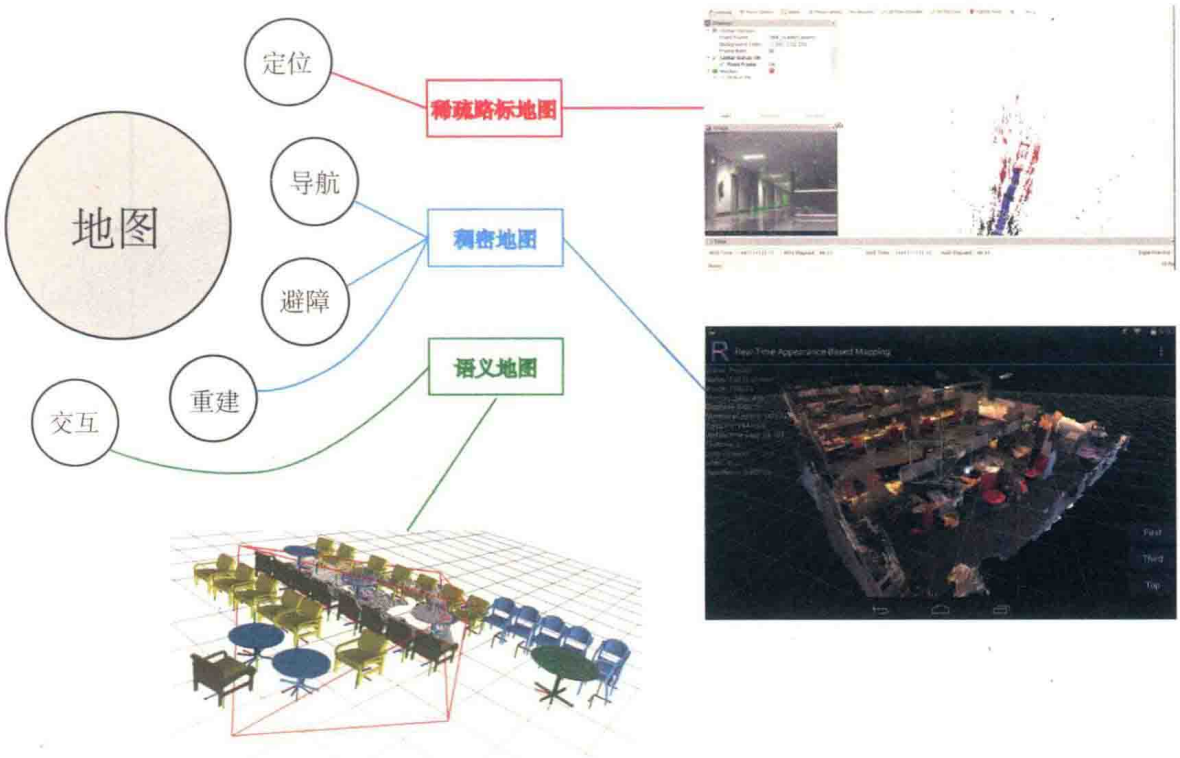


图12-1　各种地图的示意图。例子分别来自参考文献[88, 119, 120]

图13-1　从简单的事物出发，逐渐搭建复杂但优秀的作品

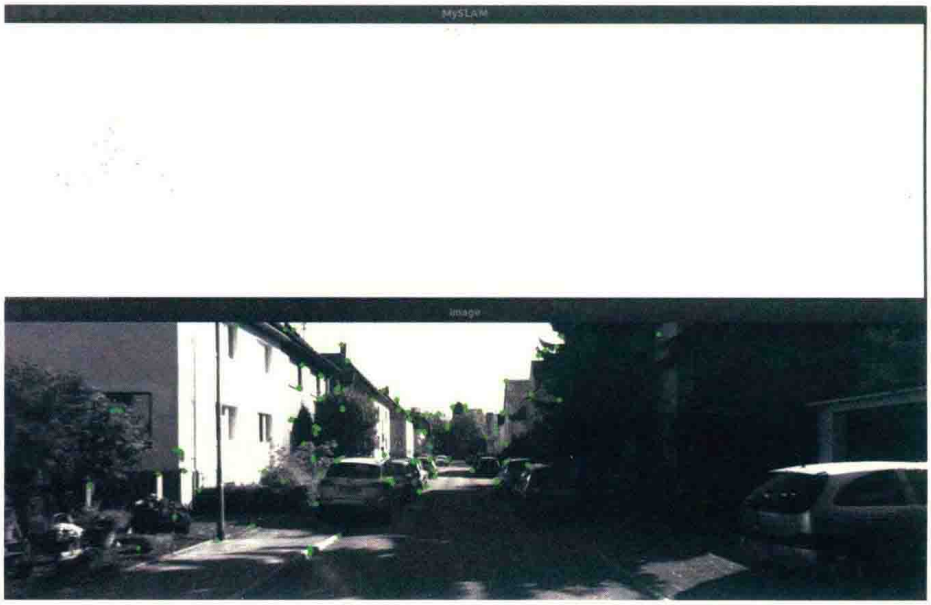


图13-4　视觉里程计运行截图

（a）　　（b）

图14-1　MonoSLAM 的运行时截图。（a）追踪特征点在图像中的表示；（b）特征点在三维空间中的表示

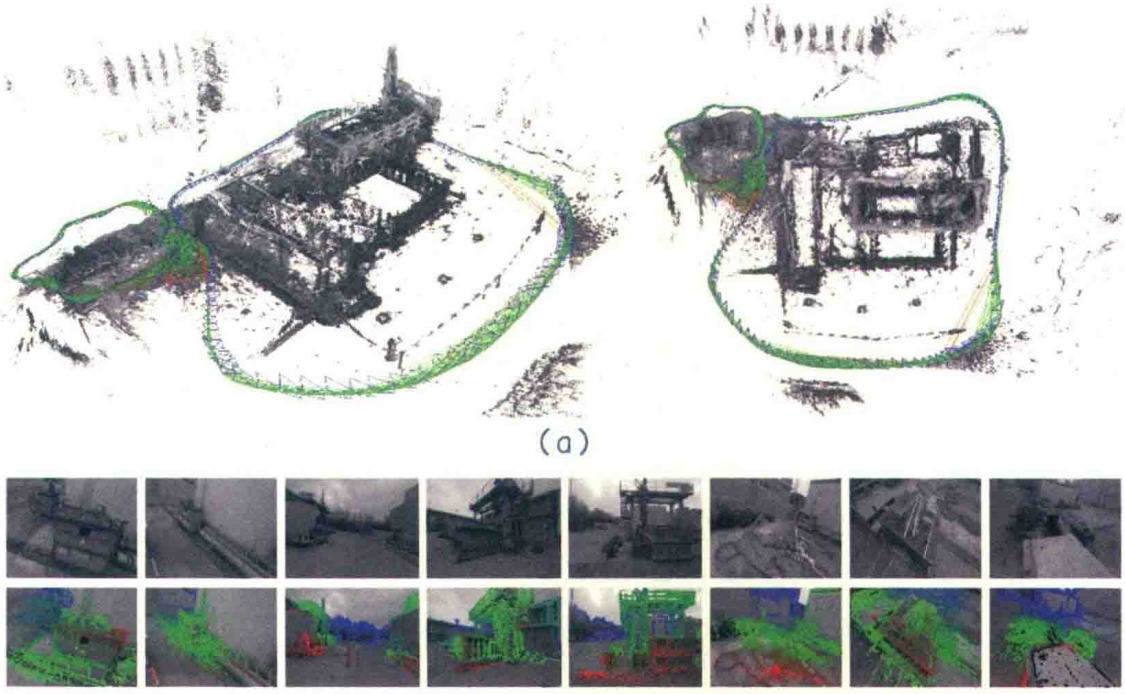


图14-4　LSD-SLAM 的运行情况。（a）为估计的轨迹与地图；（b）为图像中被建模的部分，即具有较好的像素梯度的部分

图14-5　SVO 跟踪关键点的图片

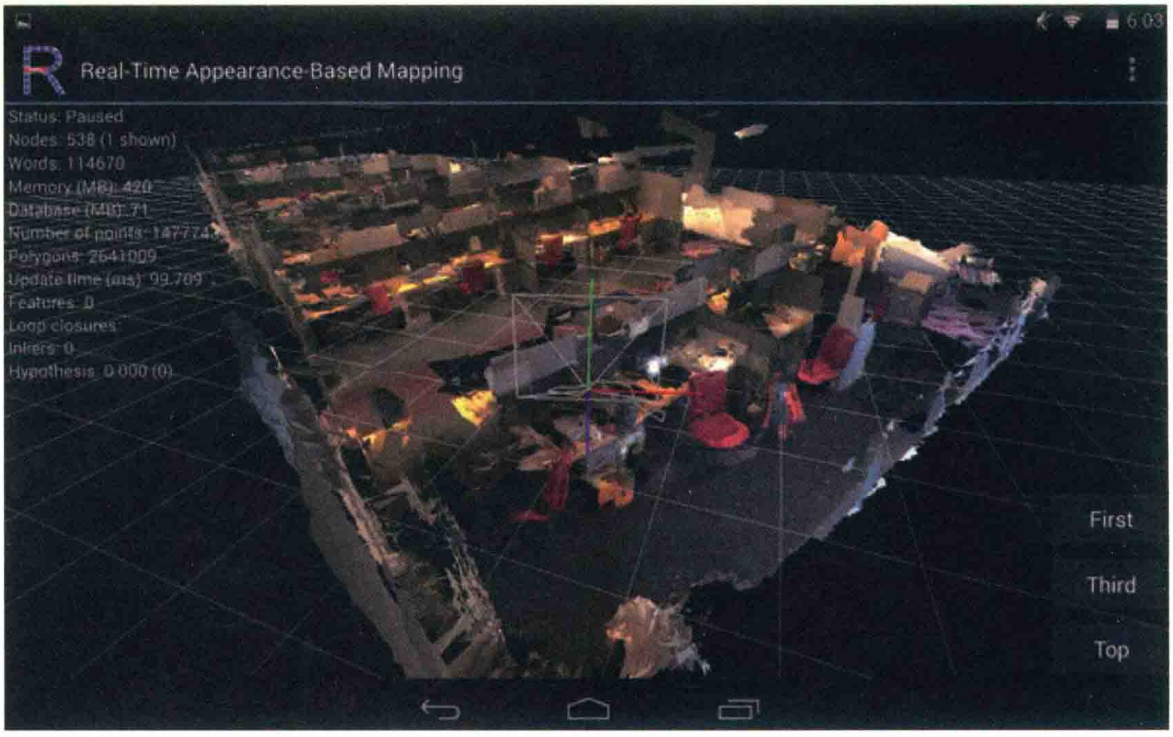
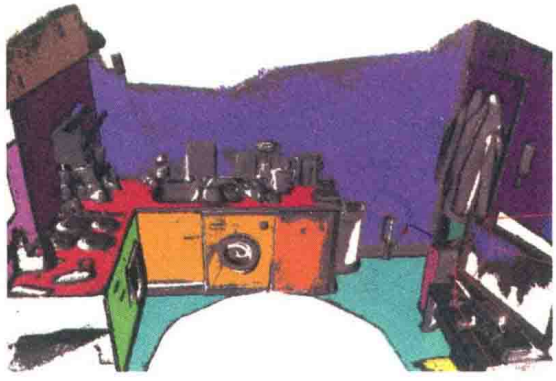


图14-6　RTAB-MAP 在Google Project Tango 上的运行样例

（a）



（b）

图14-8　语义SLAM 的一些结果，（a）和（b）分别来自文献[152, 154]

第二版序

《视觉 SLAM 十四讲：从理论到实践》出版已经两年多。两年来，这本书经历了 13 次重印，在 GitHub 上拥有 2500 个星星，也在业界引起了广泛的关注和讨论。大多数读者评价是正面的，当然，书中也有些地方不够令人满意。例如，这本书面向初学者，有些应该深入的地方讲得不够深入；书中的数学符号不够统一，有些地方容易令读者产生误解；工程实践章节内容不够丰富，介绍较浅，等等。实际上，我在 2016 年中期开始创作第 1 版，所有文字、图片和代码都是从零开始准备的，再加上当时在读博士，也是第一次写这么厚的书，错漏在所难免。2018 年，我在慕尼黑工大给学生讲 SLAM 课程，期间又积累了一些材料，所以本书从内容上更丰富、更合理。在第 1 版的基础上做了如下改动：

1. **更多的实例**。增加了一些实验代码来介绍算法的原理。在第 1 版中，多数实践代码调用了各种库中的内置函数，现在我认为更深入地介绍底层计算会更好，所以本书中的许多代码，除了调用库函数，还提供了底层的实现。
2. **更深入的内容**。主要是从第 7 讲至第 12 讲的部分，同时删除了一些泛泛而谈的边角料（比如 GTSAM 相关内容[①]）。对第 1 版大部分数学公式进行了审查，重写了那些容易引起误解的内容。
3. **更完善的工程项目**。将第 1 版的第 9 讲移至第 13 讲。于是，我们可以在介绍了所有必要知识之后，向大家展现一个完整的 SLAM 系统是如何工作的。相比于第 1 版，我在本书的项目中将追求以精简的代码实现完整的功能，你会得到一个由几百行代码实现的、有完整前后端的 SLAM 系统。
4. **更通俗、简洁的表达**。我觉得这是一本好书的标准，特别是当介绍一些看起来高深莫测的数学知识时。我重新制作了部分插图，使它们即使在黑白印刷条件下也能看起来很清楚。

当然，每讲前的简笔画我是不会改的！

总之，我尽量做到深入浅出，也希望本书能够给你带来更加舒适的阅读体验。

①因子图优化现在已有完整的书籍《机器人感知：因子图在 SLAM 中的应用》，用一小节很难介绍清楚。

目录

第 1 讲　预备知识 **1**

1.1　本书讲什么 1

1.2　如何使用本书 3

1.2.1　组织方式 3

1.2.2　代码 5

1.2.3　面向的读者 6

1.3　风格约定 6

1.4　致谢和声明 8

第 1 部分　数学基础 **10**

第 2 讲　初识 SLAM **11**

2.1　引子：小萝卜的例子 13

2.2　经典视觉 SLAM 框架 19

2.2.1　视觉里程计 20

2.2.2　后端优化 21

2.2.3　回环检测 22

2.2.4　建图 23

2.3　SLAM 问题的数学表述 24

2.4　实践：编程基础 27

2.4.1　安装 Linux 操作系统 27

2.4.2　Hello SLAM 29

2.4.3　使用 cmake 30

2.4.4　使用库 32

2.4.5　使用 IDE 35

第 3 讲 三维空间刚体运动 40
3.1 旋转矩阵 42
3.1.1 点、向量和坐标系 42
3.1.2 坐标系间的欧氏变换 43
3.1.3 变换矩阵与齐次坐标 46
3.2 实践：Eigen 47
3.3 旋转向量和欧拉角 53
3.3.1 旋转向量 53
3.3.2 欧拉角 54
3.4 四元数 56
3.4.1 四元数的定义 56
3.4.2 四元数的运算 57
3.4.3 用四元数表示旋转 59
3.4.4 四元数到其他旋转表示的转换 59
3.5 * 相似、仿射、射影变换 61
3.6 实践：Eigen 几何模块 62
3.6.1 Eigen 几何模块的数据演示 62
3.6.2 实际的坐标变换例子 64
3.7 可视化演示 66
3.7.1 显示运动轨迹 66
3.7.2 显示相机的位姿 69
第 4 讲 李群与李代数 71
4.1 李群与李代数基础 73
4.1.1 群 73
4.1.2 李代数的引出 74
4.1.3 李代数的定义 76
4.1.4 李代数 $\mathfrak{so}(3)$ 76
4.1.5 李代数 $\mathfrak{se}(3)$ 77
4.2 指数与对数映射 78
4.2.1 SO(3) 上的指数映射 78
4.2.2 SE(3) 上的指数映射 80

4.3 李代数求导与扰动模型 81
4.3.1 BCH 公式与近似形式 81
4.3.2 SO(3) 上的李代数求导 83
4.3.3 李代数求导 84
4.3.4 扰动模型（左乘） 85
4.3.5 SE(3) 上的李代数求导 85
4.4 实践：Sophus 86
4.4.1 Sophus 的基本使用方法 86
4.4.2 例子：评估轨迹的误差 89
4.5 * 相似变换群与李代数 92
4.6 小结 93
第 5 讲 相机与图像 95
5.1 相机模型 97
5.1.1 针孔相机模型 97
5.1.2 畸变模型 100
5.1.3 双目相机模型 103
5.1.4 RGB-D 相机模型 104
5.2 图像 106
5.3 实践：计算机中的图像 107
5.3.1 OpenCV 的基本使用方法 107
5.3.2 图像去畸变 112
5.4 实践：3D 视觉 113
5.4.1 双目视觉 113
5.4.2 RGB-D 视觉 115
第 6 讲 非线性优化 119
6.1 状态估计问题 121
6.1.1 批量状态估计与最大后验估计 121
6.1.2 最小二乘的引出 123
6.1.3 例子：批量状态估计 125
6.2 非线性最小二乘 126
6.2.1 一阶和二阶梯度法 127
6.2.2 高斯牛顿法 128

6.2.3 列文伯格—马夸尔特方法 130
6.3 实践：曲线拟合问题 132
6.3.1 手写高斯牛顿法 132
6.3.2 使用 Ceres 进行曲线拟合 136
6.3.3 使用 g2o 进行曲线拟合 141
6.4 小结 148

第 2 部分 实践应用 150

第 7 讲 视觉里程计 1 151
7.1 特征点法 153
7.1.1 特征点 153
7.1.2 ORB 特征 155
7.1.3 特征匹配 158
7.2 实践：特征提取和匹配 159
7.2.1 OpenCV 的 ORB 特征 159
7.2.2 手写 ORB 特征 162
7.2.3 计算相机运动 165
7.3 2D−2D：对极几何 165
7.3.1 对极约束 165
7.3.2 本质矩阵 168
7.3.3 单应矩阵 170
7.4 实践：对极约束求解相机运动 172
7.5 三角测量 177
7.6 实践：三角测量 178
7.6.1 三角测量代码 178
7.6.2 讨论 179
7.7 3D−2D：PnP 180
7.7.1 直接线性变换 180
7.7.2 P3P 182
7.7.3 最小化重投影误差求解 PnP 184
7.8 实践：求解 PnP 188
7.8.1 使用 EPnP 求解位姿 188

7.8.2 手写位姿估计 189
7.8.3 使用 g2o 进行 BA 优化 191
7.9 3D-3D：ICP 196
7.9.1 SVD 方法 196
7.9.2 非线性优化方法 198
7.10 实践：求解 ICP 199
7.10.1 实践：SVD 方法 199
7.10.2 实践：非线性优化方法 201
7.11 小结 203
第 8 讲 视觉里程计 2 205
8.1 直接法的引出 207
8.2 2D 光流 208
8.3 实践：LK 光流 210
8.3.1 使用 LK 光流 210
8.3.2 用高斯牛顿法实现光流 211
8.3.3 光流实践小结 218
8.4 直接法 218
8.4.1 直接法的推导 218
8.4.2 直接法的讨论 221
8.5 实践：直接法 221
8.5.1 单层直接法 221
8.5.2 多层直接法 226
8.5.3 结果讨论 227
8.5.4 直接法优缺点总结 230
第 9 讲 后端 1 232
9.1 概述 234
9.1.1 状态估计的概率解释 234
9.1.2 线性系统和 KF 236
9.1.3 非线性系统和 EKF 239
9.1.4 EKF 的讨论 241
9.2 BA 与图优化 242
9.2.1 投影模型和 BA 代价函数 242

9.2.2 BA 的求解 243
9.2.3 稀疏性和边缘化 245
9.2.4 鲁棒核函数 251
9.3 实践：Ceres BA 253
9.3.1 BAL 数据集 253
9.3.2 Ceres BA 的书写 253
9.4 实践：g2o 求解 BA 257
9.5 小结 263
第 10 讲 后端 2 265
10.1 滑动窗口滤波和优化 266
10.1.1 实际环境下的 BA 结构 266
10.1.2 滑动窗口法 267
10.2 位姿图 270
10.2.1 位姿图的意义 270
10.2.2 位姿图的优化 270
10.3 实践：位姿图优化 272
10.3.1 g2o 原生位姿图 272
10.3.2 李代数上的位姿图优化 277
10.3.3 小结 282
第 11 讲 回环检测 283
11.1 概述 285
11.1.1 回环检测的意义 285
11.1.2 回环检测的方法 286
11.1.3 准确率和召回率 287
11.2 词袋模型 289
11.3 字典 291
11.3.1 字典的结构 291
11.3.2 实践：创建字典 292
11.4 相似度计算 295
11.4.1 理论部分 295
11.4.2 实践：相似度的计算 296

11.5 实验分析与评述 300
11.5.1 增加字典规模 300
11.5.2 相似性评分的处理 302
11.5.3 关键帧的处理 302
11.5.4 检测之后的验证 303
11.5.5 与机器学习的关系 303
第 12 讲 建图 305
12.1 概述 306
12.2 单目稠密重建 308
12.2.1 立体视觉 308
12.2.2 极线搜索与块匹配 309
12.2.3 高斯分布的深度滤波器 311
12.3 实践：单目稠密重建 314
12.3.1 实验分析与讨论 323
12.3.2 像素梯度的问题 324
12.3.3 逆深度 325
12.3.4 图像间的变换 326
12.3.5 并行化：效率的问题 327
12.3.6 其他的改进 327
12.4 RGB-D 稠密建图 328
12.4.1 实践：点云地图 328
12.4.2 从点云重建网格 333
12.4.3 八叉树地图 336
12.4.4 实践：八叉树地图 338
12.5 * TSDF 地图和 Fusion 系列 340
12.6 小结 343
第 13 讲 实践：设计 SLAM 系统 345
13.1 为什么要单独列工程章节 346
13.2 工程框架 347
13.3 实现 349
13.3.1 实现基本数据结构 349
13.3.2 前端 354

13.3.3 后端 357
13.4 实验效果 361
第 14 讲 SLAM：现在与未来 363
14.1 当前的开源方案 364
14.1.1 MonoSLAM 364
14.1.2 PTAM 365
14.1.3 ORB-SLAM 366
14.1.4 LSD-SLAM 369
14.1.5 SVO 370
14.1.6 RTAB-MAP 371
14.1.7 其他 372
14.2 未来的 SLAM 话题 372
14.2.1 视觉 + 惯性导航 SLAM 373
14.2.2 语义 SLAM 374
14.2.3 SLAM 的未来 375
附录 A 高斯分布的性质 377
附录 B 矩阵求导 380
附录 C ROS 入门 382
参考文献 385

读者服务

轻松注册成为博文视点社区用户（www.broadview.com.cn），您即可享受以下服务。

- **提交勘误**：您对书中内容的修改意见可在 提交勘误 处提交，若被采纳，将获赠博文视点社区积分（在您购买电子书时，积分可用来抵扣相应金额）。
- **与作者交流**：在页面下方 读者评论 处留下您的疑问或观点，与作者和其他读者一同学习交流。

页面入口：`http://www.broadview.com.cn/36942`

第 1 讲 预备知识

1.1 本书讲什么

这是一本介绍视觉 SLAM 的书。

那么，SLAM 是什么？

SLAM 是 **S**imultaneous **L**ocalization **a**nd **M**apping 的缩写，中文译作“**同时定位与地图构建**”[1]。它是指搭载特定**传感器**的主体，在**没有环境先验信息**的情况下，于**运动过程中**建立**环境**的模型，同时估计自己的**运动**[2]。如果这里的传感器主要为相机，那就称为“视觉 SLAM”。

本书的主题就是视觉 SLAM。这里我们刻意把许多定义放到一句话中，希望帮助读者建立一个较明确的概念。首先，SLAM 的目的是解决“定位”与“地图构建”这两个问题。也就是说，一边要估计传感器自身的位置，一边要建立周围环境的模型。那么怎么解决呢？这需要用到传感器的信息。传感器以一定形式观察外部世界，但不同传感器观察的方式不同。之所以要花一本书的篇幅讨论这个问题，是因为它很难——特别是我们希望**实时地**、在**没有先验知识**的情况下进行 SLAM。当用相机作为传感器时，我们要做的就是根据一张张连续运动的图像（它们形成了一段视频），从中推断相机的运动，以及周围环境的情况。

这似乎是个很直观的问题。我们自己走进陌生的环境时不就是这么做的吗？

在计算机视觉（Computer Vision）创立之初，人们就想象着有朝一日计算机将和人一样，通过眼睛去观察世界，理解周遭的物体，探索未知的领域——这是一个美妙而又浪漫的梦想，吸引了无数的科研人员日夜为之奋斗[3]。我们曾经以为这件事情并不困难，然而进展却远不如预想的那么顺利。我们眼中的花草树木、虫鱼鸟兽，在计算机中却是那样的不同：它们只是一个个由数字排列而成的矩阵。让计算机理解图像的内容，就像让我们自己理解这些数字一样困难。我们

既不了解自己如何理解图像，也不知道计算机该如何理解、探索这个世界。于是我们困惑了很久，直到几十年后的今天，才发现了一点点成功的迹象：通过人工智能（Artificial Intelligence）中的机器学习（Machine Learning）技术，一方面，计算机渐渐能够辨别出物体、人脸、声音、文字——尽管它所用的方式（统计建模）与我们是如此不同。另一方面，在 SLAM 发展了将近 30 年之后，我们的相机才渐渐开始能够认识到自身的位置，发觉自己在运动——虽然方式还是和人类有巨大的差异。不过，至少研究者们已经成功地搭建出种种实时 SLAM 系统，有的能够快速跟踪自身位置，有的甚至能够进行实时的三维重建。

这件事情确实很困难，但我们已经有了很大的进展。更令人兴奋的是，近年来随着科技的发展，涌现出了一大批与 SLAM 相关的应用点。在许多地方，我们都希望知道自身的位置：室内的扫地机和移动机器人需要定位，野外的自动驾驶汽车需要定位，空中的无人机需要定位，虚拟现实和增强现实的设备也需要定位。SLAM 是那样重要，没有它，扫地机就无法在房间自主地移动，只能盲目地游荡；家用机器人就无法按照指令准确到达某个房间；虚拟现实也将永远固定在座椅之上[①]——所有这些新奇的事物都无法出现在现实生活中，那将多么令人遗憾。

今天的研究者和应用开发人员逐渐意识到了 SLAM 技术的重要性。在国际上，SLAM 已经有近三十年的研究历史，也一直是机器人和计算机视觉的研究热点。21 世纪以来，以视觉传感器为中心的**视觉 SLAM 技术**，在理论和实践上都经历了明显的转变与突破，正逐步从实验室研究迈向市场应用。同时，我们又遗憾地发现，至少在国内，与 SLAM 相关的论文、书籍仍然非常匮乏，让许多对 SLAM 技术感兴趣的初学者无从一窥门径。虽然 SLAM 的理论框架基本趋于稳定，但其编程实现仍然较为复杂，有着较高的技术门槛。刚步入 SLAM 领域的研究者，不得不花很长的时间，学习大量的知识，走许多弯路才得以接近 SLAM 技术的核心。

本书全面系统地介绍了以视觉传感器为主体的视觉 SLAM 技术，我们希望它能（部分地）填补这方面资料的空白。我们会详细地介绍 SLAM 的理论背景、系统架构，以及各个模块的主流做法。同时，**极其重视实践**：本书介绍的**所有**重要算法，都将给出可以运行的实际代码，以求加深读者的理解。在第 2 版中，我们会讨论大多数算法的内在原理，而非简单地从函数库中进行调用。之所以这么做，主要是考虑到 SLAM 是一项和实践紧密相关的技术。再漂亮的数学理论，如果不能转化为可以运行的代码，就仍是可望而不可及的空中楼阁，没有实际意义。我们相信，实践出真知，实践出真爱。只有实际地演算过各种算法，你才能真正认识 SLAM，真正地喜欢上科研。

SLAM 自 1986 年提出以来[4]，关于它的文献数以千计，想要对 SLAM 发展史上的所有算法及变种做一个完整的说明，是十分困难而且没有必要的。本书会介绍 SLAM 所牵涉的背景知识，例如射影几何、计算机视觉、状态估计理论、李群与李代数等，并在这些背景知识之上，给出 SLAM 这棵大树的主干，而略去一部分形状奇特、纹理复杂的枝叶。我们认为这种做法是有效的。如果读者能够掌握主干的精髓，那么自然会有能力去探索那些边缘的、细节的、错综复杂的前沿知识。所以，我们的目的是，让 SLAM 的初学者通过阅读本书快速地成长为能够探索这个领域边

①目前，虚拟现实头盔普遍只追踪原地旋转运动，而无法追踪更大范围内的平移运动。

缘的研究者。即便你已经是 SLAM 领域的研究人员，本书的一些内容也可能让你觉得陌生，甚至产生新的见解。

目前，与 SLAM 相关的书籍主要有《概率机器人》（*Probabilistic Robotics*）[5]、《计算机视觉中的多视图几何》（*Multiple View Geometry in Computer Vision*）[3]、《机器人学中的状态估计》（*State Estimation for Robotics: A Matrix-Lie-Group Approach*）[6]①等。它们内容丰富、论述全面、推导严谨，是 SLAM 研究者中脍炙人口的经典教材。然而就目前来看，还存在两个重要的问题。其一，这些图书的目的在于介绍基础理论，SLAM 只是其应用之一。因此，它们并不能算是专门讲解 SLAM 的书籍。其二，它们的内容偏重于数学理论，基本不涉及编程实现，导致读者经常出现“书能看懂却不会编程”的情况。而我们认为，只有读者亲自实现了算法，调试了各个参数，才能谈得上真正理解了问题本身。

本书内容会涉及 SLAM 的历史、理论、算法、现状，并把完整的 SLAM 系统分成几个模块：视觉里程计、后端优化、建图，以及回环检测。我们将陪着读者一点点实现这些模块中的核心部分，探讨它们在什么情况下有效，什么情况下会出问题，并指导大家在自己的机器上运行这些代码。你会接触到一些**必要的**数学理论和许多编程知识，会用到 Eigen、OpenCV、PCL、g2o、Ceres 等库②，掌握它们在 Linux 操作系统中的使用方法。

在写作风格上，我们不想把本书写成枯燥的理论书籍。技术类图书应该是严谨可靠的，但严谨不意味着刻板。一本优秀的技术书应该是生动有趣且易于理解的。如果你觉得“这个作者怎么这么不正经”，敬请原谅，因为笔者不是一个非常严肃的人③。无论如何，有一件事是可以肯定的：只要你对这门新技术感兴趣，在学习本书的过程中肯定会有所收获！你会掌握与 SLAM 相关的理论知识，你的编程能力也将有明显的进步。在很多时候，你会有一种“我们在陪你一起做科研”的感觉，这正是笔者希望的。但愿你能在此过程中发现研究的乐趣，喜欢这种“通过一番努力，看到事情顺利运行”的成就感。

好了，话不多说，祝你旅行愉快！

1.2　如何使用本书

1.2.1　组织方式

本书名为“视觉 SLAM 十四讲”。顾名思义，我们会像在学校里讲课那样，以“讲”作为本书的基本单元。每一讲都对应一个固定的主题，其中会穿插“理论部分”和“实践部分”两种内容。通常是理论部分在前，实践部分在后。在理论部分中，我们将介绍**理解算法所必需**的数学知

①已有中文版，笔者也参与了翻译。

②如果你完全没有听说过它们，那么应该感到兴奋，这说明你会从本书中收获很多知识。

③你会经常在脚注中发现一些神奇的东西。

识，并且大多数时候以叙述的方式，而不是像数学教科书那样用“定义—定理—推论”的方式，因为我们觉得这样的方式阅读起来更容易一些，尽管有时候显得不那么严谨。实践部分主要是编程实现，讨论程序里各部分的含义及实验结果。看到标题中带有“实践”两个字的章节，你就应该（兴致勃勃地）打开电脑，和我们一起愉快地编写代码。

值得一提的是，我们只会把与解决问题相关的数学知识放在书里，并尽量保持浅显。因为笔者是工科生，所以要勇敢地承认，某些做法只要经验上够用，没必要非得在数学上追求完备。只要我们知道这些算法在绝大多数实际场景下能够工作，并且数学家们（通过冗长而且复杂的证明和讨论）说明在什么情况下可能不工作，那么笔者就表示满意，而不刻意追究那些看似完美的证明（当然，它们固有自己不可否认的价值）。由于 SLAM 牵涉到了太多数学背景，为了防止使本书变成数学教科书，我们把一些细节上的推导和证明留作习题和补充阅读材料，方便感兴趣的读者进一步阅读参考文献，更深入地掌握相关细节。

在每一讲的最后，我们设计了一些习题。其中，带 * 号的习题是具有一定难度的。我们强烈建议读者把习题都练习一遍，这对你掌握这些知识很有帮助[①]。

全书内容主要分为两个部分。

第一部分为**数学基础**篇，我们会以浅显易懂的方式，铺垫与视觉 SLAM 相关的数学知识，包括：

- 第 1 讲是预备知识，介绍本书的基本信息，习题部分主要包括一些自测题。
- 第 2 讲为 SLAM 系统概述，介绍一个 SLAM 系统由哪些模块组成，各模块的具体工作是什么。实践部分介绍编程环境的搭建过程及 IDE 的使用。
- 第 3 讲介绍三维空间刚体运动，你将接触到旋转矩阵、欧拉角、四元数的相关知识，并且在 Eigen 中使用它们。
- 第 4 讲介绍李群与李代数。即便你现在不懂李代数为何物，也没有关系。你将学到李代数的定义和使用方式，然后通过 Sophus 操作它们。
- 第 5 讲介绍针孔相机模型及图像在计算机中的表达。你将用 OpenCV 调取相机的内外参数。
- 第 6 讲介绍非线性优化，包括状态估计理论基础、最小二乘问题、梯度下降方法。你会完成一个使用 Ceres 和 g2o 进行曲线拟合的实验。

这些就是我们要用到的所有数学知识。当然，其中还隐含了你以前学过的高等数学和线性代数。笔者保证它们看起来都不会很难，至少没有听上去那么难。当然，若你想进一步深入挖掘，我们会提供一些参考资料供你阅读，那些材料可能会比正文里讲的知识难一些。

第二部分为**实践应用**篇。我们会使用第一部分介绍的理论，讲述视觉 SLAM 中各个模块的工作原理。

---

①它们也可能成为今后相关行业的面试题，或许还能帮你在找工作时给面试官留个好印象。

- 第 7 讲为特征点法的视觉里程计。该讲内容比较多，包括特征提取与匹配、对极几何约束的计算、PnP 和 ICP 等。在实践中，你将用这些方法估计两个图像之间的运动。
- 第 8 讲为直接法的视觉里程计。你将学习光流和直接法的原理，然后实现一个简单的直接法运动估计。
- 第 9 讲为后端优化，主要为对 Bundle Adjustment（BA）的深入讨论，包括基本的 BA，以及如何利用稀疏性加速求解过程。你将用 Ceres 和 g2o 分别书写一个 BA 程序。
- 第 10 讲主要介绍后端优化中的位姿图。位姿图是表达关键帧之间约束的一种更紧凑的形式。我们会介绍 SE(3) 和 Sim(3) 的位姿图，同时你将使用 g2o 对一个位姿球进行优化。
- 第 11 讲为回环检测，主要介绍以词袋方法为主的回环检测。你将使用 DBoW3 书写字典训练程序和回环检测程序。
- 第 12 讲为地图构建。我们会讨论如何使用单目进行稠密深度图的估计（以及这是多么不可靠），然后讨论 RGB-D 的稠密地图构建过程。你会书写极线搜索与块匹配的程序，然后在 RGB-D 中遇到点云地图和八叉树地图的构建问题。
- 第 13 讲是工程实践，你将搭建一个双目视觉里程计框架，综合运用先前学过的知识，实现它的基本功能。在这个过程中，你会碰到一些问题，例如优化的必要性、关键帧的选择等。我们会在 Kitti 数据集上测试它的性能，讨论一些改进的手段。
- 第 14 讲主要介绍当前的开源 SLAM 方案及未来的发展方向。相信在阅读了前面的知识之后，你会更容易理解它们的原理，实现自己的新想法。

最后，如果你完全看不懂上面在说什么，那么恭喜你！这本书很适合你！加油！

1.2.2　代码

本书所有源代码均托管在 GitHub 上：

`https://github.com/gaoxiang12/slambook2`

注意后面有一个 2，表示这是本书的代码。笔者强烈建议读者下载代码以供随时查看。代码是按章节划分的，比如，第 7 讲的内容就会放在 ch7 文件夹中。此外，对于书中用到的一些小型库，会以压缩包的形式放在 3rdparty 文件夹下。在第 2 版中，我们用 git submodule 工具来保证读者使用的软件版本与书中的完全一致，所以读者不必担心软件版本问题。对于像 OpenCV 那样的大中型库，我们会在它们第一次出现时介绍其安装方法。如果你对代码有任何疑问，请单击 GitHub 上的 Issues 按钮，提交问题。如果确实是代码出现问题，我们会及时进行修改；即使是你的理解有偏差，笔者也会尽量回复。如果你不习惯使用 GitHub，那么单击右侧包含 download 字样的按钮，将代码下载至本地即可。

1.2.3 面向的读者

本书面向对 SLAM 感兴趣的学生和研究人员。阅读本书需要一定的基础，我们假设你具备以下知识：

- **高等数学、线性代数、概率论**。这些是大部分读者应该在大学本科阶段接触过的基本数学知识[①]。你应当明白矩阵和向量是什么，或者做微分和积分是什么意思。对于 SLAM 中用到的专业知识，我们会单独介绍。
- **C++ 语言基础**。因为我们采用 C++ 作为编码语言，所以建议读者至少熟悉这门语言的语法。比如，你应该知道类是什么，如何使用 C++ 标准库，模板类如何使用，等等。我们会避免过多地使用技巧，但有些地方确实无法避免。此外，我们还使用了一些 C++ 11 标准的内容，不过，我们会在用到的地方加以解释。
- **Linux 基础**。我们的开发环境是 Linux 而非 Windows，并且只提供 Linux 下的源程序，**不会**再提供 Windows 下的开发方法介绍。**我们认为，掌握 Linux 是一个 SLAM 研究人员所必需的，请初学者暂时不要问为什么，把本书的知识学好之后相信你会和我们有同样的想法**。各种程序库在 Linux 下的配置都非常便捷，你也会在此过程中体会到 Linux 的便利。如果读者此前从未使用过 Linux，那么最好找一本 Linux 的教材稍加学习（掌握基本知识即可，一般就是相关图书的前面几章内容）。我们不要求读者具备多么高超的 Linux 操作技能，但希望读者至少知道“打开终端，进入代码目录”是如何操作的。本讲的习题里有一些 Linux 知识自测题，如果你清楚自测题的答案，那么阅读本书代码不会有任何问题。

对 SLAM 感兴趣但不具备上述知识的读者，可能在阅读本书时会感到困难。如果你不了解 C++ 的基本知识，可以读一点 *C++ Primer Plus* 之类的图书入门；如果你缺少相关的数学知识，也可以先阅读一些相关数学教材补充知识，不过我们认为，对大多数大学本科水平的朋友，读懂本书所需的数学背景肯定是具备了。代码方面，你最好花点时间亲自输入一遍，再调节里面的参数，看看效果会发生怎样的改变。这会对学习很有帮助。

本书可作为 SLAM 相关课程的教材，也可作为课外自学材料使用。

1.3 风格约定

本书既有数学理论介绍，也有编程实现，因此，为方便阅读，对不同内容采用了不同排版方式加以区分。

①实际上，每个人都至少需要学三遍线性代数：本科一遍，研究生一遍，工作时期一遍。

1. 数学公式单独列出，重要的公式还在右侧标了序号，例如：

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}. \tag{1.1}$$

数学字体采用国标风格。标量使用斜体字（如 a, α），向量和矩阵使用粗斜体（如 $\boldsymbol{a}, \boldsymbol{A}, \boldsymbol{\Sigma}$，希腊字母除外）。空心粗体代表特殊集合，如实数集 $\mathbb{R}$、整数集 $\mathbb{Z}$。李代数部分使用哥特体，如 $\mathfrak{so}(3), \mathfrak{se}(3)$。

2. 程序代码以方框框出，使用不同的字体和小一些的字号，左侧带有行号。如果程序较长，方框会延续到下一页。总之，看起来像这样：

示例代码

```
#include <iostream>
using namespace std;

int main ( int argc, char** argv )
{
    cout<<"Hello"<<endl;
    return 0;
}
```

3. 当代码数量较多或有的部分与之前列出的重复，不适合完全列在书中时，我们会**仅给出重要片段**，并以“片段”二字注明。因此，再说一遍，我们强烈建议读者到 GitHub 上下载所有源代码，完成练习，以更好地掌握本书知识。
4. 由于排版原因，书中展示的代码可能与 GitHub 中的代码有稍许不同，请以 GitHub 上的代码为准。
5. 我们用到的每个库，在第一次出现时会有比较详细的安装和使用说明，在后续的使用中不再赘述。所以，建议读者按章节顺序阅读本书内容。
6. 每一讲的开头会列出本讲的内容提要，而末尾会有小结和练习题。引用的参考文献在全书末尾列出。
7. 以星号开头的章节是选读部分，读者可以根据兴趣阅读。跳过它们不会对理解后续章节产生影响。
8. 文中重要的内容以**黑体**标出，相信你已经习惯了。
9. 我们设计的实验大多数是演示性质的。看懂了它们不代表你已经熟悉整个库的使用。所以我们建议你在课外花一点时间，对本书经常用的几个库进行深入学习。
10. 本书的习题和选读内容可能需要你自己搜索额外材料，所以你需要学会使用搜索引擎。

1.4 致谢和声明

在本书漫长的写作过程中，笔者得到了许多人的帮助，包括但不限于：

- 毕业于中科院的贺一家博士为第 5 讲的相机模型部分提供了材料。
- 颜沁睿提供了第 7 讲的公式推导材料。
- 毕业于华中科大的刘毅博士为本书第 6 讲和第 10 讲提供了材料。
- 众多的老师、同学为本书提供了修改意见：肖锡臻、谢晓佳、张明明、耿欣、李帅杰、刘富强、袁梦、孙志明、陈昊升、王京、朱晏辰、丁文东、范帝楷、衡昱帆、高扬、李少朋、吴博、闫雪娇、张腾、郑帆、卢美奇、杨楠，等等。在此向他们表示感谢。

此外，感谢笔者的导师张涛教授一直以来对笔者的支持和帮助。感谢电子工业出版社郑柳洁编辑的支持。没有他们的帮助，本书不可能以现在的面貌来到读者面前。本书的成书与出版是所有人共同努力的结晶，尽管笔者没法把他们都列在作者列表中，但是它的出版离不开他们的工作。[①]

本书写作过程中参考了大量文献和论文。其中大部分数学理论知识是前人研究的成果，并非笔者的原创。一小部分实验设计来自各开源代码的演示程序，不过大部分是笔者自己编写的。此外，也有一些图片摘自公开发表的期刊或会议论文，文中均已注明。未做说明的图像，或为原创，或来自网络，恕不一一列举。如有问题，请与我们联系，我们会在第一时间加以修正。

本书涉及的知识点众多，错漏在所难免。如有疑问，欢迎通过电子邮件与笔者联系。

笔者的邮箱是：gao.xiang.thu@gmail.com。

感谢笔者的爱人刘丽莲女士长期的理解和支持。本书是献给她的。

习题（基本知识自测题）

1. 有线性方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$，若已知 $\boldsymbol{A}, \boldsymbol{b}$，需要求解 $\boldsymbol{x}$，该如何求解？这对 $\boldsymbol{A}$ 和 $\boldsymbol{b}$ 有哪些要求？提示：从 $\boldsymbol{A}$ 的维度和秩角度来分析。
2. 高斯分布是什么？它的一维形式是什么样子？它的高维形式是什么样子？
3. 你知道 C++ 中的**类**吗？你知道 STL 吗？你使用过它们吗？
4. 你以前怎样书写 C++ 程序？（你完全可以说只在 Visual C++ 6.0 下写过 C++ 工程，只要你有写 C++ 和 C 语言的经验就行。）
5. 你知道 C++11 标准吗？你听说过或用过其中哪些新特性？有没有其他的标准？
6. 你知道 Linux 吗？你有没有至少使用过一种（不算安卓）操作系统，比如 Ubuntu？
7. Linux 的目录结构是什么样的？你知道哪些基本命令，比如 ls, cat 等？

① “笔者”指高翔，前面说“笔者”不正经不包括上述其他作者。他们都是敬业乐群的好同志。

8. 如何在 Ubuntu 系统中安装软件（不打开软件中心的情况下）？这些软件被安装在什么地方？如果只知道模糊的软件名称（比如想要装一个名称中含有 Eigen 的库），应该如何安装它？

9.* 花一个小时学习 Vim，因为你迟早会用它。你可以在终端中输入 vimtutor 阅读一遍所有内容。我们不需要你非常熟练地操作它，只要能够在学习本书的过程中使用它输入代码即可。**不要在它的插件上浪费时间，不要想着把 Vim 用成 IDE，我们只用它做文本编辑的工作。**

第1部分

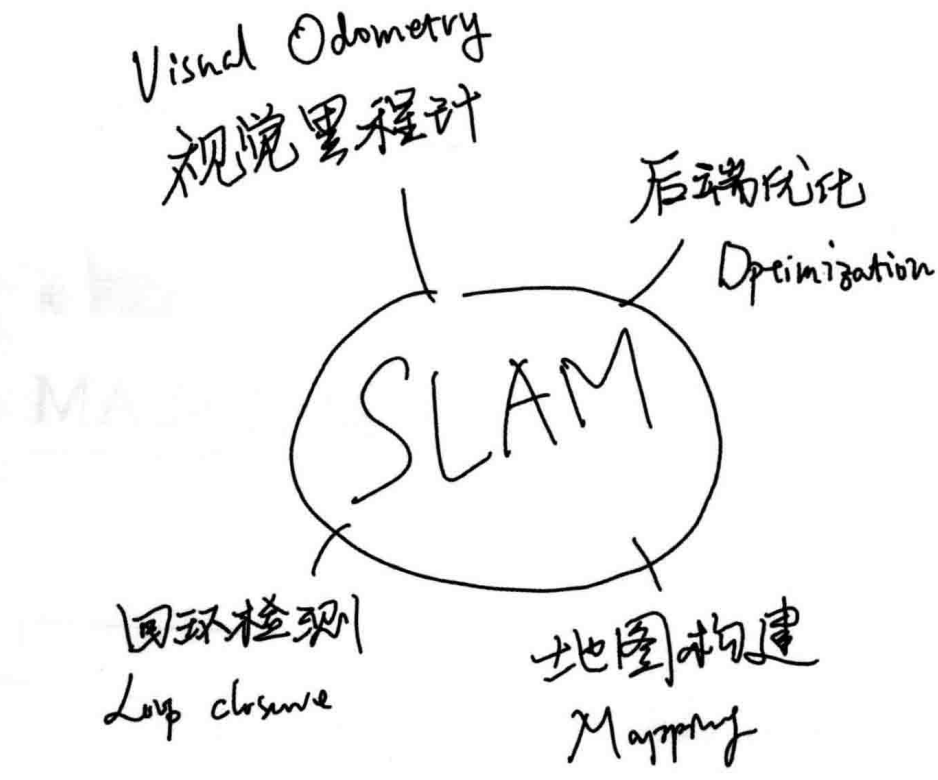
数学基础

第2讲 初识SLAM

主要目标

1. 理解一个视觉 SLAM 框架由哪几个模块组成，各模块的任务是什么。
2. 搭建编程环境，为开发和实验做准备。
3. 理解如何在 Linux 下编译并运行一个程序，如果程序出了问题，又该如何调试它。
4. 掌握 cmake 的基本使用方法。

本讲概括地介绍了一个视觉 SLAM 系统的结构，作为后续内容的大纲。实践部分介绍环境搭建、程序基本知识，最后完成一个“Hello SLAM”程序。


Visual Odometry
视觉里程计
后端优化
Optimization
SLAM
回环检测
Loop closure
地图构建
Mapping

ape-get ...
installing ...

2.1　引子：小萝卜的例子

假设我们组装了一台叫作“小萝卜”的机器人，大概的样子如图 2-1 所示。

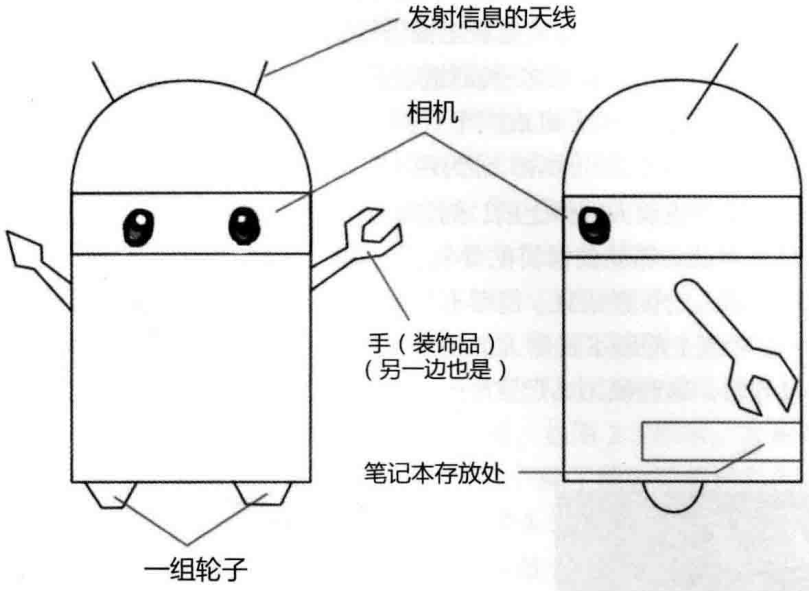


图 2-1　小萝卜设计图。左边：正视图；右边：侧视图。设备有相机、轮子、笔记本，手是装饰品

先不要管它这个简笔画的风格。另外，虽然有点像“安卓”，但它并不是靠安卓系统来计算的。我们把一台笔记本塞进了它的后备箱内（方便我们随时拿出来调试程序）。它能做点什么呢？

我们希望小萝卜具有**自主运动能力**，这是非常基本的功能。虽然世界上也有放在桌面像摆件一样的机器人，能够和人说话或播放音乐（听说还卖得不错），但是一台平板电脑完全可以胜任这些事情。作为机器人，我们希望小萝卜能够在房间里自由移动。不管我们在哪里招呼一声，它都会滴溜溜地走过来。

你会发现自主运动能力是许多高级功能的前提。不管是扫地还是搬东西，首先要让它动起来。要移动就得有轮子和电机，所以我们在小萝卜的下方安装了轮子（足式机器人步态很复杂）。有了轮子，机器人就能够四处行动了，但不加规划和控制的话，小萝卜不知道行动的目标，就只能四处乱走，更糟糕的情况下会撞上墙造成损毁。而要规划和控制，首先需要**感知**周边的环境。为此，我们在它的脑袋上安装了一个相机。安装相机的主要动机，是考虑到这样一个机器人**和人类非常相似**——从画面上一眼就能看出。有眼睛、大脑和四肢的人类，能够在任意环境里轻松自在地行走、探索，我们（天真地）觉得机器人也能够完成这件事。为了使小萝卜能够探索一个房间，它至少需要知道两件事：

1. 我在什么地方？——定位。
2. 周围环境是什么样？——建图。

“定位”和“建图”，可以看成感知的“内外之分”。作为一个“内外兼修”的小萝卜，一方面要明白自身的**状态**（即位置），另一方面也要了解外在的**环境**（即地图）。当然，解决这两个问题的方法非常多。例如，我们可以在房间地板上铺设导引线，在墙壁上贴识别二维码，在桌子上放置无线电定位设备（这其实是现在很多仓储物流机器人的做法）。如果在室外，还可以在小萝卜脑袋上安装 GPS 信号接收器（像手机或汽车一样）。有了这些东西，定位问题是否就解决了呢？我们不妨把这些传感器（如图 2-2 所示）分为两类。

一类传感器是**携带于机器人本体上**的，例如机器人的轮式编码器、相机、激光传感器，等等。另一类是**安装于环境中**的，例如前面讲的导轨、二维码标志，等等。安装于环境中的传感设备，通常能够直接测量机器人的位置信息，简单有效地解决定位问题。然而，由于它们要求环境必须由人工布置，在一定程度上限制了机器人的使用范围。例如，室内环境往往没有 GPS 信号，绝大多数园区无法铺设导轨，这时该怎么定位呢？

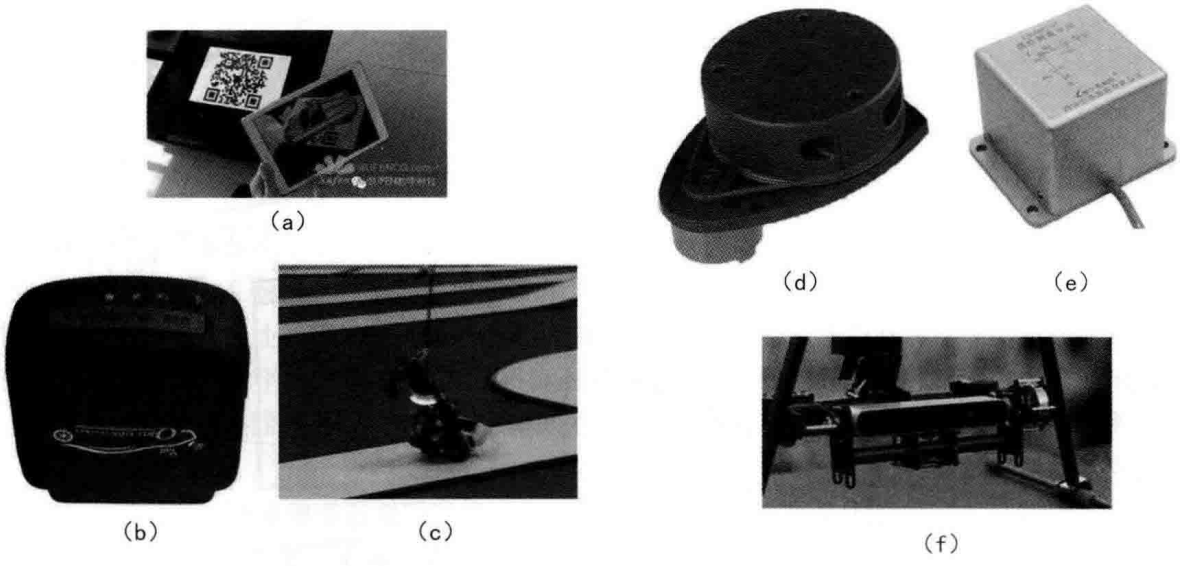


图 2-2 一些传感器。（a）利用二维码进行定位的增强现实软件；（b）GPS 定位装置；（c）铺设导轨的小车；（d）激光雷达；（e）IMU 单元；（f）双目相机

我们看到，这类传感器**约束**了外部环境。只有在这些约束满足时，基于它们的定位方案才能工作。反之，当约束无法满足时，我们就无法进行定位。所以，虽然这类传感器简单可靠，但它们无法提供一个普遍的、通用的解决方案。相对地，那些携带于机器人本体上的传感器，比如激光传感器、相机、轮式编码器、惯性测量单元（Inertial Measurement Unit，IMU）等，它们测量的通常都是一些间接的物理量而不是直接的位置数据。例如，轮式编码器会测量轮子转动的角度，

IMU 测量运动的角速度和加速度，相机和激光传感器则读取外部环境的某种观测数据。我们只能通过一些间接的手段，从这些数据推算自己的位置。虽然听上去这是一种迂回战术，但更明显的好处是，它们没有对环境提出任何要求①，从而使得这种定位方案可适用于未知环境。

回顾前面讨论过的 SLAM 定义，我们在 SLAM 中非常强调未知环境。所以理论上，我们不限制小萝卜的使用环境②，这意味着我们没法假设像 GPS 或导轨这样的外部传感器都能顺利工作。因此，使用携带式的传感器来完成 SLAM 是我们重点关心的问题。特别地，当谈论视觉 SLAM 时，我们主要是指如何用**相机**解决定位和建图问题。同样，如果传感器主要是激光，那就称为激光 SLAM。激光 SLAM 相对成熟，比如 2005 年出版的《概率机器人》[5] 中就介绍了许多关于激光 SLAM 的知识，在 ROS 里也能找到许多关于激光定位、激光建图的现成软件。

视觉 SLAM 是本书的主题，所以我们尤其关心小萝卜的眼睛能够做些什么事。SLAM 中使用的相机与我们平时见到的单反摄像头并不是同一个东西。它更加简单，通常不携带昂贵的镜头，而是以一定速率拍摄周围的环境，形成一个连续的视频流。普通的摄像头能以每秒钟拍摄 30 张图片的速度采集图像，高速相机则更快一些。按照工作方式的不同，相机可以分为单目（Monocular）相机、双目（Stereo）相机和深度（RGB-D）相机三大类，如图 2-3 所示。直观看来，单目相机只有一个摄像头，双目有两个，而 RGB-D 的原理较复杂，除了能够采集到彩色图片，还能读出每个像素与相机之间的距离。深度相机通常携带多个摄像头，工作原理和普通相机不尽相同，在第 5 讲会详细介绍其工作原理，此处读者只需有一个直观概念即可。此外，SLAM 中还有全景相机[7]、Event 相机[8] 等特殊或新兴的种类。虽然偶尔能看到它们在 SLAM 中的应用，不过到目前为止还没有成为主流。从样子上看，小萝卜使用的似乎是水平的双目相机。

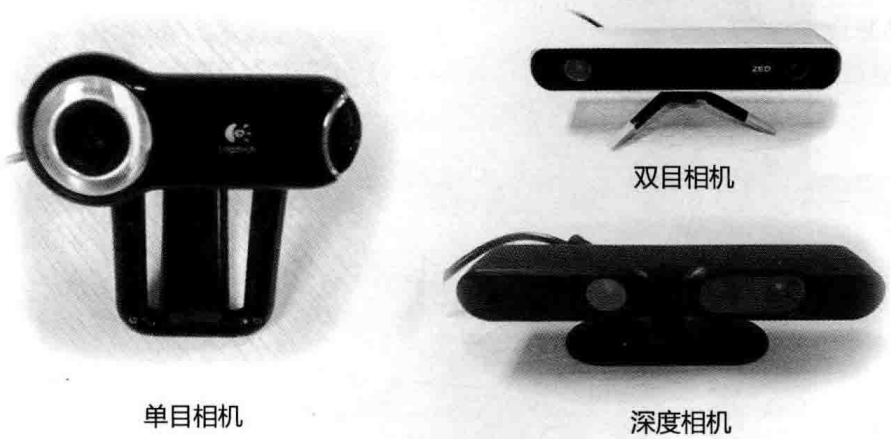


图 2-3　形形色色的相机：单目、双目和深度相机

①话虽如此，但实际上我们至少要求传感器在该环境下能够工作，所以不要举一些“轮式编码器在宇宙空间中没法工作，相机在黑屋子里看不见东西”的极端例子。

②不过现实中我们都会有一个大概的范围，例如室内或室外。

我们来分别看一看各种相机用来做 SLAM 时有什么特点。

单目相机

只使用一个摄像头进行 SLAM 的做法称为单目 SLAM（Monocular SLAM）。这种传感器结构特别简单，成本特别低，所以单目 SLAM 非常受研究者关注。你肯定见过单目相机的数据：照片。是的，作为一张照片，它有什么特点呢？

照片本质上是拍摄某个场景（Scene）在相机的成像平面上留下的一个**投影**。**它以二维的形式记录了三维的世界**。显然，这个过程丢掉了场景的一个维度，也就是所谓的深度（或距离）。在单目相机中，我们无法通过单张图片计算场景中物体与相机之间的**距离**（远近）。之后我们会看到，这个距离将是 SLAM 中非常关键的信息。我们见过大量的图像，形成了一种天生的直觉，对大部分场景都有一个直观的**距离感（空间感）**，它可以帮助我们判断图像中物体的远近关系。例如，我们能够辨认出图像中的物体，并且知道其大致的大小；近处的物体会挡住远处的物体，而太阳、月亮等天体一般在很远的地方；物体受光照后会留下影子，等等。这些信息都可以帮助我们判断物体的远近，但也存在一些情况会使这种距离感失效，这时我们就无法判断物体的远近及其真实大小了。图 2-4 就是这样一个例子，我们无法仅通过这张图像判断后面那些小人是真实的人，还是小型模型。除非我们转换视角，观察场景的三维结构。换言之，在单张图像里，你无法确定一个物体的真实大小。它可能是一个**很大但很远**的物体，也可能是一个**很近但很小**的物体。由于近大远小的透视关系，它们可能在图像中变成同样大小的样子。

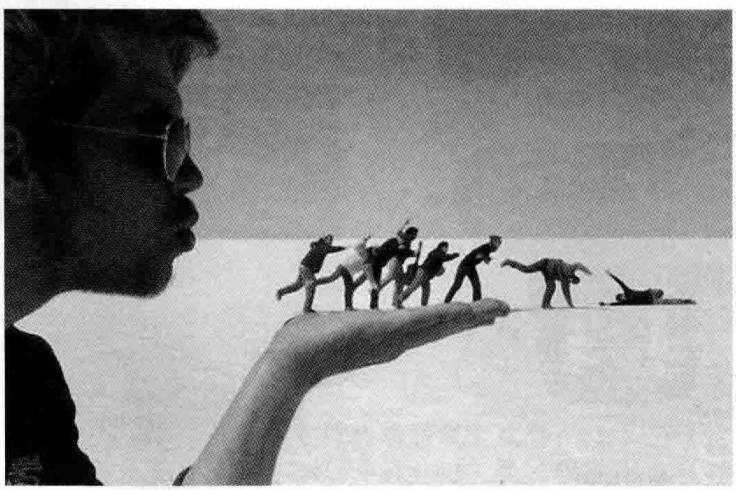


图 2-4 单目视觉中的尴尬：不知道深度时，手掌上的人是真人还是模型呢

由于单目相机拍摄的图像只是三维空间的二维投影，所以，如果真想恢复三维结构，必须改变相机的视角。在单目 SLAM 中也是同样的原理。我们必须移动相机，才能估计它的**运动**（Motion），同时估计场景中物体的远近和大小，不妨称之为**结构**（Structure）。那么，怎么估计这些运动和结构呢？想象你坐在一辆运动的列车中。一方面，如果列车往右边移动，那么我们看到的东西就会

往左边移动——这就给我们推测运动带来了信息。另一方面，我们还知道：**近处的物体移动快，远处的物体移动慢，极远处（无穷远处）的物体（如太阳、月亮）看上去是不动的**。于是，当相机移动时，这些物体在图像上的运动就形成了**视差**（Disparity）。通过视差，我们就能定量地判断哪些物体离得远，哪些物体离得近。

然而，即使我们知道了物体远近，它们仍然只是一个相对的值。比如我们在看电影时，虽然能够知道电影场景中哪些物体比另一些大，但无法确定电影里那些物体的“真实尺度”：那些大楼是真实的高楼大厦，还是放在桌上的模型？而摧毁大厦的是真实怪兽，还是穿着特摄服装的演员？如果把相机的运动和场景大小同时放大两倍，单目相机所看到的像是一样的。同样地，把这个大小乘以任意倍数，我们都将看到一样的景象。这说明，单目 SLAM 估计的轨迹和地图将与真实的轨迹和地图相差一个因子，也就是所谓的**尺度**（Scale）[①]。由于单目 SLAM 无法仅凭图像确定这个真实尺度，所以又称为**尺度不确定性**（Scale Ambiguity）。

平移之后才能计算深度，以及无法确定真实尺度，这两件事情给单目 SLAM 的应用造成了很大的麻烦。其根本原因是通过单张图像无法确定深度。所以，为了得到这个深度，人们开始使用双目相机和深度相机。

双目相机和深度相机

使用双目相机和深度相机的目的是通过某种手段测量物体与相机之间的距离，克服单目相机无法知道距离的缺点。一旦知道了距离，场景的三维结构就可以通过单个图像恢复，同时消除尺度不确定性。尽管都是为了测量距离，但双目相机与深度相机测量深度的原理是不一样的。双目相机由两个单目相机组成，但这两个相机之间的距离〔称为**基线**（Baseline）〕是已知的。我们通过这个基线来估计每个像素的空间位置——这和人眼非常相似。我们人类可以通过左右眼图像的差异判断物体的远近，在计算机上也是同样的道理（如图 2-5 所示）。如果对双目相机进行拓展，则可以搭建多目相机，不过本质上并没有什么不同。

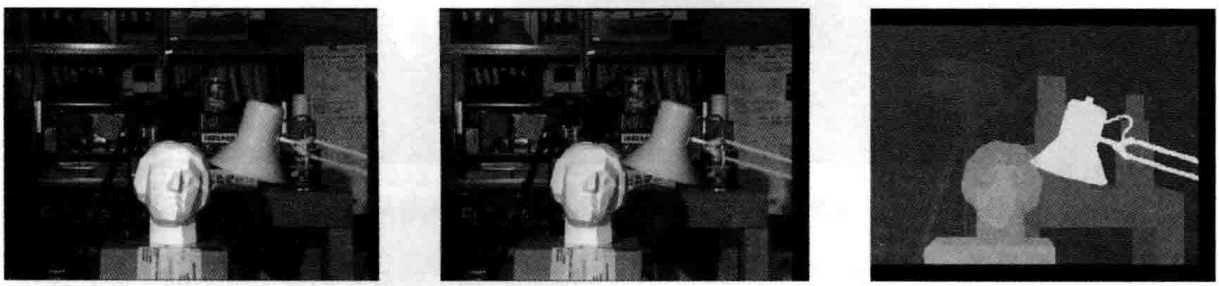


图 2-5　双目相机的数据：左眼图像，右眼图像。通过左右眼的差异，能够判断场景中物体与相机之间的距离

①数学上的原因将会在视觉里程计一讲中解释。

计算机上的双目相机需要大量的计算才能（不太可靠地）估计每一个像素点的深度，相比于人类真是非常笨拙[①]。双目相机测量到的深度范围与基线相关。基线距离越大，能够测量到的物体就越远，所以无人车上搭载的双目相机通常会是个很大的家伙。双目相机的距离估计是比较左右眼的图像获得的，并不依赖其他传感设备，所以它既可以应用在室内，又可应用于室外。双目或多目相机的缺点是配置与标定均较为复杂，其深度量程和精度受双目的基线与分辨率所限，而且视差的计算非常消耗计算资源，需要使用 GPU 和 FPGA 设备加速，才能实时输出整张图像的距离信息。因此在现有的条件下，计算量是双目的主要问题之一。

深度相机（又称 RGB-D 相机，在本书中主要使用 RGB-D 这个名称）是 2010 年前后兴起的一种相机，它最大的特点是可以通过红外结构光或 Time-of-Flight（ToF）原理，像激光传感器那样，通过主动向物体发射光并接收返回的光，测出物体与相机之间的距离。它并不像双目相机那样通过软件计算来解决，而是通过物理的测量手段，所以相比于双目相机可节省大量的计算资源（如图 2-6 所示）。目前常用的 RGB-D 相机包括 Kinect/Kinect V2、Xtion Pro Live、RealSense 等，在一些手机上人们也用它来识别人脸。不过，现在多数 RGB-D 相机还存在测量范围窄、噪声大、视野小、易受日光干扰、无法测量透射材质等诸多问题，在 SLAM 方面，主要用于室内，室外则较难应用。

图 2-6 RGB-D 数据：深度相机可以直接测量物体的图像和距离，从而恢复三维结构

我们讨论了几种常见的相机，相信通过以上的说明，你已经对它们有了直观的了解。现在，想象相机在场景中运动的过程，我们将得到一系列连续变化的图像[②]。视觉 SLAM 的目标，是通过这样的一些图像，进行定位和地图构建。这件事情并没有想象的那么简单。它不是某种算法，

①笔者女儿三个月大时已经能够辨认并抓取放在她面前的玩具了，笔者觉得她比许多大学实验室里的机器人都要智能。
②你可以用手机录个小视频试试。

只要我们输入数据，就可以往外不断地输出定位和地图信息。SLAM 需要一个完善的算法框架，而经过研究者们长期的努力工作，我们已经发展出了一套比较成熟的框架。

2.2　经典视觉 SLAM 框架

下面来看经典的视觉 SLAM 框架（如图 2-7 所示），我们来了解视觉 SLAM 究竟由哪几个模块组成。

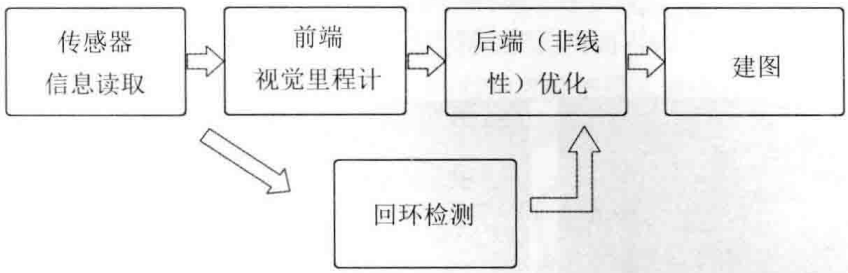


图 2-7　经典的视觉 SLAM 框架

整个视觉 SLAM 流程包括以下步骤。

1. **传感器信息读取**。在视觉 SLAM 中主要为相机图像信息的读取和预处理。如果是在机器人中，还可能有码盘、惯性传感器等信息的读取和同步。
2. **前端视觉里程计**（Visual Odometry，VO）。视觉里程计的任务是估算相邻图像间相机的运动，以及局部地图的样子。VO 又称为前端（Front End）。
3. **后端（非线性）优化**（Optimization）。后端接受不同时刻视觉里程计测量的相机位姿，以及回环检测的信息，对它们进行优化，得到全局一致的轨迹和地图。由于接在 VO 之后，又称为后端（Back End）。
4. **回环检测**（Loop Closure Detection）。回环检测判断机器人是否到达过先前的位置。如果检测到回环，它会把信息提供给后端进行处理。
5. **建图**（Mapping）。它根据估计的轨迹，建立与任务要求对应的地图。

经典的视觉 SLAM 框架是过去十几年的研究成果。这个框架本身及其包含的算法已经基本定型，并且已经在许多视觉程序库和机器人程序库中提供。依靠这些算法，我们能够构建一个视觉 SLAM 系统，使之在正常的工作环境里实时定位与建图。因此，我们说，**如果把工作环境限定在静态、刚体、光照变化不明显、没有人为干扰的场景**，那么这种场景下的 SLAM 技术已经相当成熟[9]。

读者可能还没有理解上面几个模块的概念，下面就来详细介绍各个模块具体的任务。但是，准确理解其工作原理需要一些数学知识，我们将放到本书的第二部分进行介绍。目前读者只需对各模块有一个直观的、定性的理解即可。

2.2.1 视觉里程计

视觉里程计关心**相邻图像**之间的相机运动，最简单的情况当然是两张图像之间的运动关系。例如，当看到图 2-8 时，我们会自然地反应出右图应该是左图向左旋转一定角度的结果（在视频情况下感觉会更自然）。我们不妨思考：自己是怎么知道“向左旋转”这件事情的呢？人类早已习惯于用眼睛探索世界，估计自己的位置，但又往往难以用理性的语言描述我们的直觉[①]。看到图 2-8 时，我们会自然地认为，这个场景中离我们近的是吧台，远处是墙壁和黑板。当相机向左转动时，吧台离我们近的部分出现在视野中，而右侧远处的柜子则移出了视野。通过这些信息，我们判断相机应该是向左旋转了。

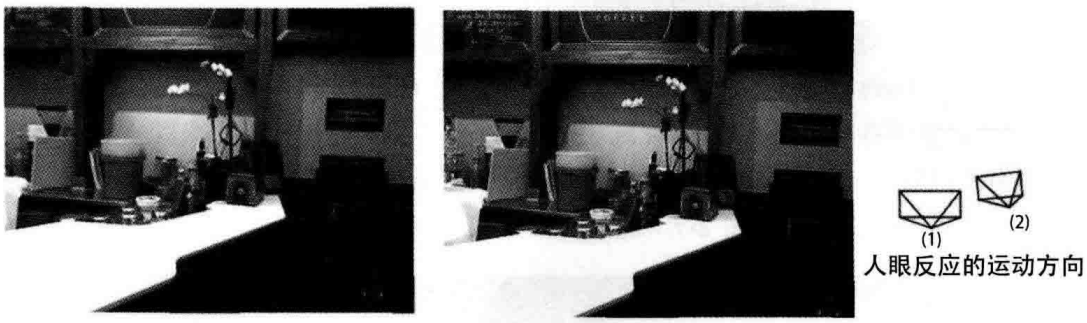


图 2-8 相机拍摄到的图片与人眼反应的运动方向

如果进一步问：能否确定旋转了多少度，平移了多少厘米？我们就很难给出一个确切的答案了。因为我们的直觉对这些具体的数字并不敏感。但是，在计算机中，又必须精确地测量这段运动信息。所以我们要问：**计算机是如何通过图像确定相机的运动的呢？**

前面也提过，在计算机视觉领域，人类在直觉上看来十分自然的事情，在计算机视觉中却非常困难。图像在计算机里只是一个数值矩阵。这个矩阵里表达着什么东西，计算机毫无概念（这也正是现在机器学习要解决的问题）。而在视觉 SLAM 中，我们只能看到一个个像素，知道它们是某些空间点在相机的成像平面上投影的结果。所以，为了定量地估计相机运动，必须先**了解相机与空间点的几何关系**。

要讲清这个几何关系及视觉里程计的实现方法，需要铺垫一些背景知识。读者现在只需知道，视觉里程计能够通过相邻帧间的图像估计相机运动，并恢复场景的空间结构。称它为“里程计”是因为它和实际的里程计一样，只计算相邻时刻的运动，而和过去的信息没有关联。在这一点上，视觉里程计就像一种只有短时记忆的物种（不过可以不限于两帧，数量可以更多一些，例如 5～10 帧）。

---

①在很多涉及计算机视觉和机器学习的任务中，情况都是这样。拜近几年深度学习崛起所赐，我们连机器是怎么计算的都已经看不懂了。

现在，假定我们已有了一个视觉里程计，估计了两张图像间的相机运动。那么，一方面，只要把相邻时刻的运动“串”起来，就构成了机器人的运动轨迹，从而解决了定位问题。另一方面，我们根据每个时刻的相机位置，计算出各像素对应的空间点的位置，就得到了地图。这么说来，有了视觉里程计，是不是就解决了 SLAM 问题呢？

视觉里程计确实是 SLAM 的关键，我们也会花大量的篇幅来介绍它。然而，仅通过视觉里程计来估计轨迹，将不可避免地出现**累积漂移**（Accumulating Drift）。这是由于视觉里程计在最简单的情况下只估计两个图像间的运动造成的。我们知道，每次估计都带有一定的误差，而由于里程计的工作方式，先前时刻的误差将会传递到下一时刻，导致经过一段时间之后，估计的轨迹将不再准确（如图 2-9 所示）。例如，机器人先向左转 90°，再向右转 90°。由于误差，我们把第一个 90° 估计成了 89°。那我们就会尴尬地发现，向右转之后机器人的估计位置并没有回到原点。更糟糕的是，即使之后的估计完全准确，与真实值相比，都会带上这 −1° 的误差。

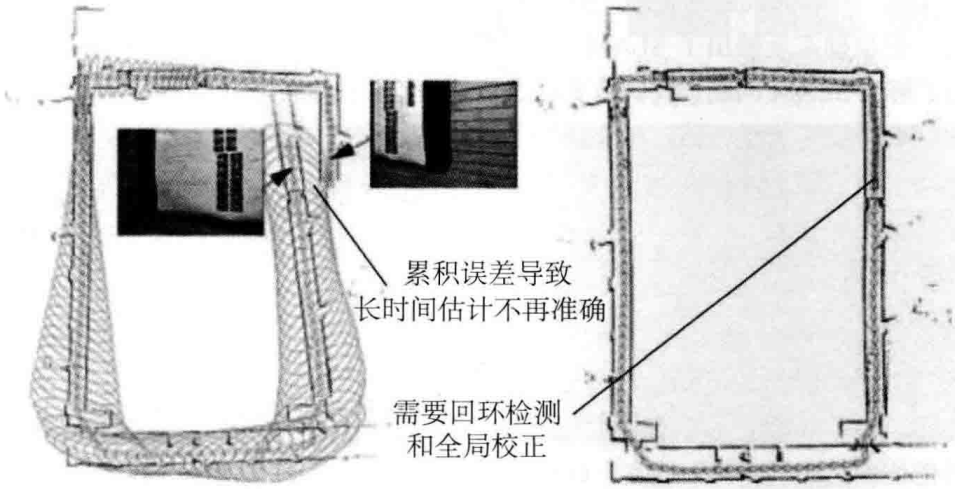


图 2-9　累积误差与回环检测的校正结果[10]

这也就是所谓的**漂移**（Drift）。它将导致我们无法建立一致的地图。你会发现原本直的走廊变成了斜的，而原本 90° 的直角不再是 90°——这实在是一件令人难以忍受的事情！为了解决漂移问题，我们还需要两种技术：**后端优化**[①]和**回环检测**。回环检测负责把“机器人回到原始位置”的事情检测出来，而后端优化则根据该信息，校正整个轨迹的形状。

2.2.2　后端优化

笼统地说，后端优化主要指处理 SLAM 过程中的**噪声**问题。虽然我们很希望所有的数据都是准确的，但是在现实中，再精确的传感器也带有一定的噪声。便宜的传感器测量误差较大，昂

①更多时候称为后端。由于主要使用的是优化方法，故称为后端优化。

贵的可能会小一些，有的传感器还会受磁场、温度的影响。所以，除了解决“如何从图像估计出相机运动”，我们还要关心这个估计带有多大的噪声，这些噪声是如何从上一时刻传递到下一时刻的，而我们又对当前的估计有多大的自信。后端优化要考虑的问题，就是如何从这些带有噪声的数据中估计整个系统的状态，以及这个状态估计的不确定性有多大——这称为最大后验概率估计（Maximum-a-Posteriori，MAP）。这里的状态既包括机器人自身的轨迹，也包含地图。

相对地，视觉里程计部分有时被称为“前端”。在 SLAM 框架中，前端给后端提供待优化的数据，以及这些数据的初始值。而后端负责整体的优化过程，它往往面对的只有数据，不必关心这些数据到底来自什么传感器。**在视觉 SLAM 中，前端和计算机视觉研究领域更为相关，比如图像的特征提取与匹配等，后端则主要是滤波与非线性优化算法。**

从历史意义上来说，现在我们称为后端优化的部分，在很长一段时间内直接被称为“SLAM 研究”。早期的 SLAM 问题是一个**状态估计**问题——正是后端优化要解决的。在最早提出 SLAM 的一系列论文中，当时的人们称它为“空间状态不确定性的估计”（Spatial Uncertainty）[4, 11]。虽然有些晦涩，但也确实反映出了 SLAM 问题的本质：**对运动主体自身和周围环境空间不确定性的估计**。为了解决 SLAM 问题，我们需要状态估计理论，把定位和建图的不确定性表达出来，然后采用滤波器或非线性优化，估计状态的均值和不确定性（方差）。状态估计与非线性优化的具体内容将在第 6 讲、第 9 讲和第 10 讲介绍。让我们暂时跳过它的原理说明，继续往下介绍。

2.2.3 回环检测

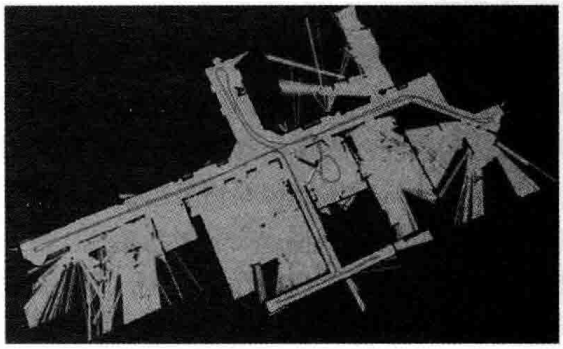
回环检测，又称闭环检测，主要解决位置估计**随时间漂移**的问题。怎么解决呢？假设实际情况下机器人经过一段时间的运动后回到了原点，但是由于漂移，它的位置估计值却没有回到原点。怎么办呢？如果有某种手段，让机器人知道“回到了原点”这件事，或者把“原点”识别出来，我们再把位置估计值“拉”过去，就可以消除漂移了。这就是所谓的回环检测。

回环检测与“定位”和“建图”二者都有密切的关系。事实上，我们认为，地图存在的主要意义是让机器人知晓自己到过的地方。为了实现回环检测，我们需要让机器人具有**识别到过的场景**的能力。它的实现手段有很多。例如像前面说的那样，我们可以在机器人下方设置一个标志物（如一张二维码图片）。它只要看到了这个标志，就知道自己回到了原点。该标志物实质上是一种环境中的传感器，对应用环境做了限制（万一不能贴二维码怎么办？）。我们更希望机器人能使用携带的传感器——也就是图像本身，来完成这一任务。例如，可以判断**图像间的相似性**来完成回环检测。这一点和人是相似的。当我们看到两张相似的图片时，容易辨认它们来自同一个地方。如果回环检测成功，则可以显著地减小累积误差。所以，视觉回环检测实质上是一种计算图像数据相似性的算法。由于图像的信息非常丰富，使得正确检测回环的难度降低了不少。

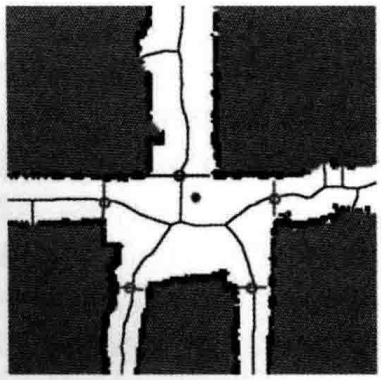
在检测到回环之后，我们会把“A 与 B 是同一个点”这样的信息告诉后端优化算法。然后，后端根据这些新的信息，把轨迹和地图调整到符合回环检测结果的样子。这样，如果我们有充分而且正确的回环检测，则可以消除累积误差，得到全局一致的轨迹和地图。

2.2.4　建图

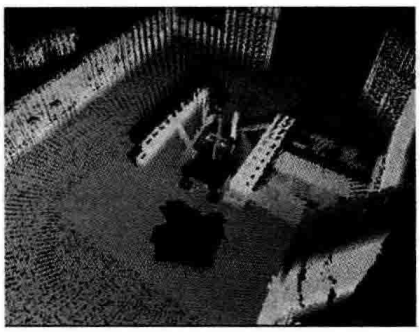
建图是指构建地图的过程。地图（如图 2-10 所示）是对环境的描述，但这个描述并不是固定的，需要视 SLAM 的应用而定。



2D 栅格地图



2D 拓扑地图



3D 点云地图

3D 网格地图

图 2-10　形形色色的地图[12]（见彩插）

对家用扫地机器人这种主要在低矮平面里运动的机器人来说，只需要一个二维的地图，标记哪里可以通过，哪里存在障碍物，就够它在一定范围内导航了。而对相机而言，它有 6 自由度的运动，我们至少需要一张三维的地图。有时，我们想要一个漂亮的重建结果，不仅是一组空间点，还需要带纹理的三角面片。有时，我们又不关心地图的样子，只需要知道“A 点到 B 点可通过，而 B 点到 C 点不行”这样的事情。甚至，有时不需要地图，或者地图可以由其他人提供，例如，行驶的车辆往往可以得到已绘制好的当地地图。

对于地图，我们有太多的想法和需求。因此，相比于前面提到的视觉里程计、后端优化和回环检测，建图并没有一个固定的形式和算法。一组空间点的集合可以称为地图，一个漂亮的 3D 模型也是地图，一个标记着城市、村庄、铁路、河道的图片还是地图。地图的形式随 SLAM 的应用场合而定。大体上讲，可以分为**度量地图**与**拓扑地图**两种。

度量地图（Metric Map）

度量地图强调精确地表示地图中物体的位置关系，通常用稀疏（Sparse）与稠密（Dense）对其分类。稀疏地图进行了一定程度的抽象，并不需要表达所有的物体。例如，我们选择一部分具有代表意义的东西，称之为路标（Landmark），那么一张稀疏地图就是由路标组成的地图，而不是路标的部分就可以忽略。相对地，稠密地图着重于建模所有看到的东西。定位时用稀疏路标地图就足够了。而用于导航时，则往往需要稠密地图（否则撞上两个路标之间的墙怎么办？）。稠密地图通常按照某种分辨率，由许多个小块组成，在二维度量地图中体现为许多个小格子（Grid），而在三维度量地图中则体现为许多小方块（Voxel）。通常，一个小块含有占据、空闲、未知三种状态，以表达该格内是否有物体。当查询某个空间位置时，地图能够给出该位置是否可以通过的信息。这样的地图可以用于各种导航算法，如 A*、D*[①]等，为机器人研究者所重视。但是我们也看到，一方面，这种地图需要存储每一个格点的状态，会耗费大量的存储空间，而且多数情况下地图的许多细节部分是无用的。另一方面，大规模度量地图有时会出现一致性问题。很小的一点转向误差，可能会导致两间屋子的墙出现重叠，使地图失效。

拓扑地图（Topological Map）

相比于度量地图的精确性，拓扑地图更强调地图元素之间的关系。拓扑地图是一个图（Graph），由节点和边组成，只考虑节点间的连通性，例如只关注 A、B 点是连通的，而不考虑如何从 A 点到达 B 点。它放松了地图对精确位置的需要，去掉了地图的细节，是一种更为紧凑的表达方式。然而，拓扑地图不擅长表达具有复杂结构的地图。如何对地图进行分割，形成节点与边，又如何使用拓扑地图进行导航与路径规划，仍是有待研究的问题。

2.3 SLAM 问题的数学表述

通过前面的介绍，读者应该对 SLAM 中各个模块的组成和主要功能有了直观的了解，但仅靠直观印象并不能帮助我们写出可以运行的程序。我们要把它上升到理性层次，也就是用数学语言来描述 SLAM 的过程。我们会用到一些变量和公式，但请读者放心，笔者会尽量让它保持足够地清楚。

假设小萝卜正携带着某种传感器在未知环境里运动，怎么用数学语言描述这件事呢？首先，由于相机通常是在某些时刻采集数据的，所以我们也只关心这些时刻的位置和地图。这就把一段连续时间的运动变成了离散时刻 $t = 1, \cdots, K$ 当中发生的事情。在这些时刻，用 $\boldsymbol{x}$ 表示小萝卜自身的位置。于是各时刻的位置就记为 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_K$，它们构成了小萝卜的轨迹。地图方面，我们假设地图是由许多个**路标**组成的，而每个时刻，传感器会测量到一部分路标点，得到它们的观测数

①https://en.wikipedia.org/wiki/A*_search_algorithm

据。不妨设路标点一共有 N 个，用 $\boldsymbol{y}_1, \cdots, \boldsymbol{y}_N$ 表示它们。

在这样的设定中，“小萝卜携带着传感器在环境中运动”，由如下两件事情描述：

1. 什么是**运动**？我们要考察从 $k-1$ 时刻到 k 时刻，小萝卜的位置 $\boldsymbol{x}$ 是如何变化的。
2. 什么是**观测**？假设小萝卜在 k 时刻于 $\boldsymbol{x}_k$ 处探测到了某一个路标 $\boldsymbol{y}_j$，我们要考察如何用数学语言来描述这件事情。

先来看运动。通常，机器人会携带一个测量自身运动的传感器，例如码盘或惯性传感器。这个传感器可以测量有关运动的读数，但不一定直接就是位置之差，还可能是加速度、角速度等信息。有时我们也给小萝卜发送指令，例如“前进 1 米”“左转 90°”，或者“油门踩到底”“刹车”等。无论是何种情况，我们都能使用一个通用的、抽象的数学模型来说明此事：

$$\boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k\right). \tag{2.1}$$

这里，$\boldsymbol{u}_k$ 是运动传感器的读数或者输入[①]，$\boldsymbol{w}_k$ 为该过程中加入的噪声。注意，我们用一个一般函数 f 来描述这个过程，而不指明 f 具体的作用方式。这使得整个函数可以指代任意的运动传感器/输入，成为一个通用的方程，而不必限定于某个特殊的传感器上。我们把它称为**运动方程**。

噪声的存在使得这个模型变成了随机模型。换句话说，即使我们下达“前进 1 米”的命令，也不代表小萝卜真的前进了 1 米。如果所有指令都是准确的，也就没必要**估计**了。事实上，小萝卜可能某次只前进了 0.9 米，另一次前进了 1.1 米，再一次可能由于轮胎打滑，干脆没有前进。于是，每次运动过程中的噪声是随机的。如果我们不理会这个噪声，那么只根据指令来确定的位置可能与实际位置相差十万八千里。

与运动方程相对应，还有一个**观测方程**。观测方程描述的是，当小萝卜在 $\boldsymbol{x}_k$ 位置上看到某个路标点 $\boldsymbol{y}_j$ 时，产生了一个观测数据 $\boldsymbol{z}_{k,j}$。同样，用一个抽象的函数 h 来描述这个关系：

$$\boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right). \tag{2.2}$$

这里，$\boldsymbol{v}_{k,j}$ 是这次观测里的噪声。由于观测所用的传感器形式更多，这里的观测数据 $\boldsymbol{z}$ 及观测方程 h 也有许多不同的形式。

读者或许会说，我们用的函数 f, h，似乎并没有具体地说明运动和观测是怎么回事？同时，这里的 $\boldsymbol{x}$,$\boldsymbol{y}$,$\boldsymbol{z}$ 又是什么呢？事实上，根据小萝卜的真实运动和传感器的种类，存在着若干种**参数化**（Parameterization）方式。什么叫参数化呢？举例来说，假设小萝卜在平面中运动，那么，它的位姿[②]由两个位置和一个转角来描述，即 $\boldsymbol{x}_k = [x_1, x_2, \theta]_k^{\mathrm{T}}$，其中 x_1, x_2 是两个轴上的位置而 θ 为转角。同时，输入的指令是两个时间间隔位置和转角的变化量 $\boldsymbol{u}_k = [\Delta x_1, \Delta x_2, \Delta\theta]_k^{\mathrm{T}}$，于是，此

①有些研究者认为运动传感器的输入应该放到观测方程，但是在实践中这两种做法基本是一样的。

②在本书中，我们以“位姿”这个词表示“位置”加上“姿态”。

时运动方程就可以具体化为

$$\begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}_k = \begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}_{k-1} + \begin{bmatrix} \Delta x_1 \\ \Delta x_2 \\ \Delta\theta \end{bmatrix}_k + \boldsymbol{w}_k. \tag{2.3}$$

这是简单的线性关系。不过，并不是所有的输入指令都是位移和角度的变化量，例如“油门”或者“控制杆”的输入就是速度或加速度量，所以也存在着其他形式更加复杂的运动方程，那时我们可能需要进行动力学分析。

关于观测方程，以小萝卜携带着的一个二维激光传感器为例。我们知道激光传感器观测一个 2D 路标点时，能够测到两个量：路标点与小萝卜本体之间的距离 r 和夹角 ϕ。记路标点为 $\boldsymbol{y}_j = [y_1, y_2]_j^{\mathrm{T}}$，位姿为 $\boldsymbol{x}_k = [x_1, x_2]_k^{\mathrm{T}}$，观测数据为 $\boldsymbol{z}_{k,j} = [r_{k,j}, \phi_{k,j}]^{\mathrm{T}}$，那么观测方程就写为

$$\begin{bmatrix} r_{k,j} \\ \phi_{k,j} \end{bmatrix} = \begin{bmatrix} \sqrt{(y_{1,j} - x_{1,k})^2 + (y_{2,j} - x_{2,k})^2} \\ \arctan\left(\dfrac{y_{2,j} - x_{2,k}}{y_{1,j} - x_{1,k}}\right) \end{bmatrix} + \boldsymbol{v}. \tag{2.4}$$

考虑视觉 SLAM 时，传感器是相机，那么观测方程就是“对路标点拍摄后，得到图像中的像素”的过程。这个过程牵涉相机模型的描述，将在第 5 讲中详细介绍，这里暂且略过。

可见，针对不同的传感器，这两个方程有不同的参数化形式。如果我们保持通用性，把它们取成通用的抽象形式，那么 SLAM 过程可总结为两个基本方程：

$$\begin{cases} \boldsymbol{x}_k = f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k), & k = 1, \cdots, K \\ \boldsymbol{z}_{k,j} = h(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}), & (k,j) \in \mathcal{O} \end{cases}. \tag{2.5}$$

其中 $\mathcal{O}$ 是一个集合，记录着在哪个时刻观察到了哪个路标（通常不是每个路标在每个时刻都能看到的——我们在单个时刻很可能只看到一小部分）。这两个方程描述了最基本的 SLAM 问题：当知道运动测量的读数 $\boldsymbol{u}$，以及传感器的读数 $\boldsymbol{z}$ 时，如何求解定位问题（估计 $\boldsymbol{x}$）和建图问题（估计 $\boldsymbol{y}$）？这时，我们就把 SLAM 问题建模成了一个**状态估计问题**：如何通过带有噪声的测量数据，估计内部的、隐藏着的状态变量？

状态估计问题的求解，与两个方程的具体形式，以及噪声服从哪种分布有关。按照运动和观测方程是否为线性，噪声是否服从高斯分布进行分类，分为**线性/非线性**和**高斯/非高斯**系统。其中线性高斯系统（Linear Gaussian，LG 系统）是最简单的，它的无偏的最优估计可以由卡尔曼滤波器（Kalman Filter，KF）给出。而在复杂的非线性非高斯系统（Non-Linear Non-Gaussian，NLNG 系统）中，我们会使用以扩展卡尔曼滤波器（Extended Kalman Filter，EKF）和非线性优化两大

类方法去求解。直至 21 世纪早期，以 EKF 为主的滤波器方法在 SLAM 中占据了主导地位。我们会在工作点处把系统线性化，并以预测—更新两大步骤进行求解（见第 10 讲）。最早的实时视觉 SLAM 系统就是基于 EKF[2] 开发的。随后，为了克服 EKF 的缺点（例如线性化误差和噪声高斯分布假设），人们开始使用粒子滤波器（Particle Filter）等其他滤波器，乃至使用非线性优化的方法。时至今日，主流视觉 SLAM 使用以图优化（Graph Optimization）为代表的优化技术进行状态估计[13]。我们认为优化技术已经明显优于滤波器技术，只要计算资源允许，通常都偏向于使用优化方法（见第 10 讲和第 11 讲）。

相信读者已经对 SLAM 的数学模型有了大致的了解，然而我们仍需澄清一些问题。首先，要说明机器人**位置 x 是什么**。我们还未明确解释**位置**的意义。也许读者能够理解，在平面中运动的小萝卜可以用两个坐标加一个转角的形式将位置参数化。笔者的漫画风格有些二次元，小萝卜在更多时候是一个三维空间里的机器人。其次，我们知道三维空间的运动由 3 个轴构成，所以小萝卜的运动要由 3 个轴上的平移，以及绕着 3 个轴的旋转来描述，一共有 6 个自由度。那是否意味着随便用一个 $\mathbb{R}^6$ 中的向量就能描述它了呢？我们将发现事情并没有那么简单。对 6 自由度的**位姿**[①]，如何表达它，如何优化它，都需要一定篇幅来介绍，这将是第 3 讲和第 4 讲的主要内容。随后，我们要说明在视觉 SLAM 中，**观测方程**如何参数化。换句话说，空间中的路标点是如何投影到一张照片上的。这需要解释相机的成像模型，我们将在第 5 讲介绍。最后，当知道了这些信息，**怎么求解上述方程**？这需要非线性优化的知识，是第 6 讲的内容。

这些内容组成了本书数学知识的部分。在对它们进行铺垫之后，我们就能仔细讨论视觉里程计、后端优化等更详细的知识了。可以看到，本讲介绍的内容构成了本书的提要。如果读者还没有很好地理解上面的概念，不妨回过头再阅读一遍。下面就要开始介绍程序啦！

2.4　实践：编程基础

2.4.1　安装 Linux 操作系统

终于到了令人兴奋的实践环节啦！你是否准备好了呢？为了完成本书的实践环节，我们需要准备一台电脑。你可以使用笔记本或台式机，最好是你个人的电脑，因为我们需要在上面安装操作系统进行实验。

我们的程序以 Linux 上的 C++ 程序为主。在实验过程中，我们会使用大量程序库。大部分程序库只对 Linux 提供了较好的支持，而在 Windows 上的配置则相对（相当）麻烦。因此，我们不得不假定你已经具备 Linux 的基本知识（参见第 1 讲的练习题），包括使用基本的命令，了解软件如何安装。当然，你不必了解如何在 Linux 下开发 C++ 程序，这正是下面要详细谈的。

我们先来搭建本书所需的实验环境。作为一本面向初学者的书，我们使用 Ubuntu 系统作为

①我们以后称它为位姿（Pose），以与位置进行区别。我们说的位姿，包含了旋转（Rotation）和平移（Translation）。

开发环境。在 Linux 的各大发行版中，Ubuntu 及其衍生版本一直享有对新手用户友好的美誉。Ubuntu 是一个开源操作系统，它的系统和软件可以在官方网站（`http://cn.ubuntu.com`）免费下载，并且提供了详细的安装方式说明。同时，清华、中科大等国内高校也提供了 Ubuntu 软件源，使软件的安装十分便捷。

本书的第 1 版使用 Ubuntu 14.04 作为默认开发环境。在本书中，我们将默认版本更新至较新的 **Ubuntu 18.04**（如图 2-11 所示），以便研究者使用。如果你想换换风格，那么 Ubuntu Kylin、Debian、Deepin 和 Linux Mint 也是不错的选择。笔者保证书中所有代码在 Ubuntu 18.04 下经过了良好的测试，但如果你选择其他发行版，笔者无法确定是否会遇到问题。你可能需要花费一些时间解决问题（不过也可以把它们当作锻炼自己的机会）。大体来说，Ubuntu 对各种库的支持均较为完善，软件也非常丰富。尽管我们不限制你具体使用哪种 Linux 发行版，但在讲解中，**我们会以 Ubuntu 18.04 为例**，且主要使用 Ubuntu 下的命令（例如 apt-get），所以在其他版本的 Ubuntu 下面不会有明显的区别。一般情况下，程序在 Linux 间移植不会非常烦琐。如果你想在 Windows 或 OS X 下使用本书中的程序，则需要有一定的移植经验。

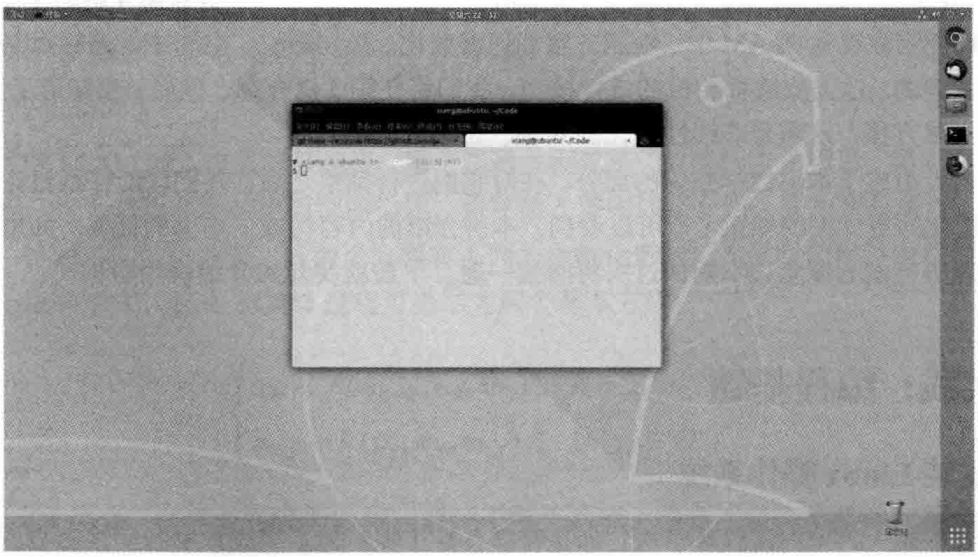


图 2-11　一个运行在虚拟机中的 Ubuntu 18.04

现在，请大家在自己的电脑上安装好 Ubuntu 18.04。关于 Ubuntu 的安装，可以在网上搜到大量教程，只要照做即可，此处略过。最简单的方式是使用虚拟机（如图 2-11 所示），但需要占用大量内存（我们的经验是 4GB 以上）和 CPU 才能保持流畅；你也可以安装双系统，这样运行速度会快一些，但需要一个空白的 U 盘作为启动盘。另外，虚拟机软件对外部硬件的支持往往不够好，如果希望使用实际的传感器（双目、Kinect 等），则建议使用双系统。

关于安装的小提示：

- 安装操作系统时请断开网络连接，并且不要选择“安装中下载更新”，这样可以提高安装

速度。可以在系统安装完毕后再更新。如果你有 SSD 硬盘，这个过程大概用时 15 分钟。

- 安装完成后，请务必把软件源设置到离你较近的服务器上，以获得更快的下载速度。例如，笔者使用清华的软件源通常能以每秒 10MB 的速度安装软件[①]。

现在，假设你已经成功安装好 Ubuntu，无论是使用虚拟机还是双系统的方式。如果你还不熟悉 Ubuntu，则可以试试它的各种软件，体验它的界面和交互方式[②]。不过笔者必须**提醒**你，特别是新手朋友：不要在 Ubuntu 的用户界面上花费太多时间！Linux 有许多可能浪费时间的地方，你可能会找到某些小众的软件、一些游戏，甚至会为找一张壁纸花费不少时间。但是请记住，你是用 Linux 来工作的。特别是在本书中，你是用 Linux 来学习 SLAM 的，所以要尽量把时间花在学习 SLAM 上。

我们选择一个目录，放置本书中 SLAM 程序的代码。例如，可以将代码放到家目录（/home）的“slambook2”下。以后，我们将把这个目录称为“**代码根目录**”。同时，可以另外选择一个目录，把本书的 Git 代码复制下来，方便做实验时随时对照。本书的代码是按章节划分的。例如，本讲的代码将在 slambook2/ch2 下，第 3 讲的代码则在 slambook2/ch3 下。所以，现在请读者进入 slambook2/ch2 下（你应该会新建文件夹并进入该文件夹了吧）。

2.4.2　Hello SLAM

我们从最基本的程序开始。与许多计算机类书籍一样，我们来书写一个 Hello SLAM 程序。不过在做这件事之前，我们先来聊聊程序是什么。

在 Linux 中，程序是一个具有执行权限的文件。它可以是一个脚本，也可以是一个二进制文件，不过我们不限定它的后缀名（不像 Windows 那样需要指定成.exe 文件）。我们常用的 cd、ls 等命令，就是位于/bin 目录下的可执行文件。而对于其他地方的可执行程序，只要它有可执行权限，当我们在终端中输入程序名时，它就会运行。在使用 C++ 编程时，我们先书写一个文本文件：

slambook2/ch2/helloSLAM.cpp

```
#include <iostream>
using namespace std;

int main(int argc, char **argv) {
    cout << "Hello SLAM!" << endl;
    return 0;
}
```

然后用一个叫作**编译器**的程序，把这个文本文件编译成可执行程序。显然，这是一个非常简单的程序。你应该能毫不费力地看懂它，所以这里不多加解释——如果实际情况不是这样，请你

①感谢 TUNA 同学们的维护！

②大多数人第一次看到 Ubuntu 系统都觉得很漂亮。

先补习 C++ 的基本知识。这个程序只是把一个字符串输出到屏幕上而已。你可以用文本编辑器 gedit（或 Vim，如果你在上一讲学习了 Vim）输入这些代码，并保存在上面列出的路径下。现在，我们用编译器 g++（g++ 是一个 C++ 编译器）把它编译成一个可执行文件。输入：

终端输入：

```
g++ helloSLAM.cpp
```

如果顺利，那么这条命令应该没有任何输出。如果计算机上出现“command not found”的错误信息，则说明你可能还没有安装 g++，请用如下命令进行安装：

终端输入：

```
sudo apt-get install g++
```

如果出现别的错误，请再检查一遍刚才的程序是否输入正确。

刚才这条编译命令把 helloSLAM.cpp 文本文件编译成了一个可执行程序。我们检查当前目录，会发现多了一个 a.out 文件，而且它具有执行权限（终端里颜色不同）。我们输入./a.out 即可运行此程序[①]：

终端输入：

```
% ./a.out
Hello SLAM!
```

如我们所想，这个程序输出“Hello SLAM!”，告诉我们它在正确运行。

请回顾我们之前做的事情。在这个例子中，我们用编辑器输入了 helloSLAM.cpp 的源代码，然后调用 g++ 编译器对它进行编译，得到了可执行文件。g++ 默认把源文件编译成 a.out 这个名字的程序（虽然有些古怪，但是可以接受的）。如果我们愿意，也可以指定这个输出的文件名（留作习题）。这是一个极其简单的例子，我们**使用了大量的默认参数，几乎省略了所有中间步骤**，为的是给读者一个简洁的印象（虽然你可能没有体会到）。下面我们要用 cmake 来编译这个程序。

2.4.3 使用 cmake

理论上，任意一个 C++ 程序都可以用 g++ 来编译。但当程序规模越来越大时，一个工程可能有许多个文件夹和源文件，这时输入的编译命令将越来越长。通常，一个小型 C++ 项目可能含有十几个类，各类间还存在着复杂的依赖关系。其中一部分要编译成可执行文件，另一部分编译成库文件。如果仅靠 g++ 命令，则需要输入大量的编译指令，整个编译过程会变得异常烦琐。因此，对于 C++ 项目，使用一些工程管理工具会更加高效。在历史上，工程师们曾使用 makefile

①程序前头的% 为提示符，不要连这个也打进去。

进行自动编译，但下面要谈的 cmake 比它更加方便。并且 cmake 在工程上广泛使用，我们会看到后面提到的大多数库都使用 cmake 管理源代码。

在一个 cmake 工程中，我们会用 cmake 命令生成一个 makefile 文件，然后，用 make 命令根据这个 makefile 文件的内容编译整个工程。读者可能还不知道 makefile 是什么，不过没关系，我们会通过例子来学习。仍然以上面的 helloSLAM.cpp 为例，这次我们不是直接使用 g++，而是用 cmake 来制作一个工程，然后编译它。在 slambook2/ch2/中新建一个 CMakeLists.txt 文件，内容如下：

slambook2/ch2/CMakeLists.txt

```
# 声明要求的cmake最低版本
cmake_minimum_required( VERSION 2.8 )

# 声明一个cmake工程
project( HelloSLAM )

# 添加一个可执行程序
# 语法：add_executable( 程序名　源代码文件 )
add_executable( helloSLAM helloSLAM.cpp )
```

CMakeLists.txt 文件用于告诉 cmake 要对这个目录下的文件做什么事情。CMakeLists.txt 文件的内容需要遵守 cmake 的语法。在这个示例中，我们演示了最基本的工程：指定一个工程名和一个可执行程序。根据注释，读者应该理解每句话做了些什么。

现在，在当前目录下（slambook2/ch2/），调用 cmake 对该工程进行 cmake 编译[①]：

终端输入：

```
cmake .
```

cmake 会输出一些编译信息，然后在当前目录下生成一些中间文件，其中最重要的就是 MakeFile[②]。由于 MakeFile 是自动生成的，我们不必修改它。现在，用 make 命令对工程进行编译：

终端输入：

```
% make
Scanning dependencies of target helloSLAM
[100%] Building CXX object CMakeFiles/helloSLAM.dir/helloSLAM.cpp.o
Linking CXX executable helloSLAM
[100%] Built target helloSLAM
```

①指令最后有一个句点，请不要忘记，这表示在当前目录下进行 cmake。

②MakeFile 是一个自动化编译的脚本，读者现在可以将它理解成一系列自动生成的编译指令，无须理会其内容。

在编译过程中会输出一个编译进度。如果顺利通过，我们就可以得到在 CMakeLists.txt 中声明的那个可执行程序 helloSLAM。执行它：

终端输入：

```
% ./helloSLAM
Hello SLAM!
```

因为我们并没有修改源代码，所以得到的结果和之前是一样的。请读者想想这种做法和之前直接使用 g++ 编译的区别。这次我们使用了先执行 cmake 再执行 make 的做法，执行 cmake 的过程处理了工程文件之间的关系，而执行 make 过程实际调用了 g++ 来编译程序。虽然这个过程中多了调用 cmake 和 make 的步骤，但我们对项目的编译管理工作，**从输入一串 g++ 命令，变成了维护若干个比较直观的 CMakeLists.txt 文件**，这将明显降低维护整个工程的难度。例如，如果想新增一个可执行文件，只需在 CMakeLists.txt 中添加一行“add_executable”命令即可，而后续的步骤是不变的。cmake 会帮我们解决代码的依赖关系，无须输入一大串 g++ 命令。

现在这个过程中唯一让我们不满的是，cmake 生成的中间文件还留在我们的代码文件中。当想要发布代码时，我们并不希望把这些中间文件一同发布出去。这时我们还需要把它们一个个地删除，十分不便。一种更好的做法是让这些中间文件都放在一个中间目录中，在编译成功后，把这个中间目录删除即可。所以，更常见的编译 cmake 工程的做法如下：

终端输入：

```
mkdir build
cd build
cmake ..
make
```

我们新建了一个中间文件夹“build”，然后进入 build 文件夹，通过 cmake .. 命令对上一层文件夹，也就是代码所在的文件夹进行编译。这样，cmake 产生的中间文件就会生成在 build 文件夹中，与源代码分开。当发布源代码时，只要把 build 文件夹删掉即可。请读者自行按照这种方式对 ch2 中的代码进行编译，然后调用生成的可执行程序（请记得把上一步产生的中间文件删除）。

2.4.4 使用库

在一个 C++ 工程中，并不是所有代码都会编译成可执行文件。只有带有 main 函数的文件才会生成可执行程序。而另一些代码，我们只想把它们打包成一个东西，供其他程序调用。这个东西叫作**库**（Library）。

一个库往往是许多算法、程序的集合，我们会在之后的练习中接触到许多库。例如，OpenCV 库提供了许多计算机视觉相关的算法，而 Eigen 库提供了矩阵代数的计算。因此，我们要学习

如何用 cmake 生成库，并且使用库中的函数。现在我们演示如何自己编写一个库。书写如下 libHelloSLAM.cpp 文件：

slambook2/ch2/libHelloSLAM.cpp

```
//这是一个库文件
#include <iostream>
using namespace std;

void printHello() {
    cout << "Hello SLAM" << endl;
}
```

这个库提供了一个 printHello 函数，调用此函数将输出一条信息。但是它没有 main 函数，这意味着这个库中没有可执行文件。我们在 CMakeLists.txt 里加上如下内容：

slambook2/ch2/CMakeLists.txt

```
add_library( hello libHelloSLAM.cpp )
```

这条命令告诉 cmake，我们想把这个文件编译成一个叫作“hello”的库。然后，和上面一样，使用 cmake 编译整个工程：

终端输入：

```
cd build
cmake ..
make
```

这时，在 build 文件夹中就会生成一个 libhello.a 文件，这就是我们得到的库。

在 Linux 中，库文件分成**静态库**和**共享库**两种[①]。静态库以.a 作为后缀名，共享库以.so 结尾。所有库都是一些函数打包后的集合，差别在于**静态库每次被调用都会生成一个副本，而共享库则只有一个副本**，更省空间。如果想生成共享库而不是静态库，只需使用以下语句即可。

slambook2/ch2/CMakeLists.txt

```
add_library( hello_shared SHARED libHelloSLAM.cpp )
```

此时得到的文件就是 libhello_shared.so。

库文件是一个压缩包，里面有编译好的二进制函数。如果仅有.a 或.so 库文件，那么我们并不知道里面的函数到底是什么，调用的形式又是什么样的。为了让别人（或者自己）使用这个库，

①你多半猜错了，它并不叫作动态库。

我们需要提供一个**头文件**，说明这些库里都有些什么。因此，对于库的使用者，**只要拿到了头文件和库文件，就可以调用这个库**。下面编写 libhello 的头文件。

slambook2/ch2/libHelloSLAM.h

```
#ifndef LIBHELLOSLAM_H_
#define LIBHELLOSLAM_H_
// 上面的宏定义是为了防止重复引用这个头文件而引起的重定义错误

// 打印一句Hello的函数
void printHello();

#endif
```

这样，根据这个文件和我们刚才编译得到的库文件，就可以使用 printHello 函数了。最后，我们写一个可执行程序来调用这个简单的函数：

slambook2/ch2/useHello.cpp

```
#include "libHelloSLAM.h"

// 使用libHelloSLAM.h中的printHello()函数
int main(int argc, char **argv) {
    printHello();
    return 0;
}
```

然后，在 CMakeLists.txt 中添加一个可执行程序的生成命令，**链接**到刚才使用的库上：

slambook2/ch2/CMakeLists.txt

```
add_executable( useHello useHello.cpp )
target_link_libraries( useHello hello_shared )
```

通过这两行语句，useHello 程序就能顺利使用 hello_shared 库中的代码了。这个小例子演示了如何生成并调用一个库。请注意，对于他人提供的库，我们也可用同样的方式对它们进行调用，整合到自己的程序中。

除了已经演示的功能，cmake 还有许多语法和选项，这里不一一列举。实际上，cmake 很像一个正常的编程语言，有变量、条件控制语句，所以你也可以像学习编程一样学习 cmake。习题中包含了一些 cmake 的阅读材料，感兴趣的读者可自行阅读。现在，简单回顾我们之前做了哪些事：

1. 程序代码由头文件和源文件组成。
2. 带有 main 函数的源文件编译成可执行程序，其他的编译成库文件。

3. 如果可执行程序想调用库文件中的函数，则它需要参考该库提供的头文件，以明白调用的格式。同时，要把可执行程序链接到库文件上。

这几个步骤应该是简单清楚的，但实际操作中你可能会遇到一些问题。例如，如果代码里引用了库的函数，但忘了把程序链接到库上，会发生什么呢？请试着把 CMakeLists.txt 中的链接部分去掉，看看会发生什么情况。你能看懂 cmake 报告的错误消息吗？

2.4.5　使用 IDE

最后，我们来谈谈如何使用集成开发环境（Integrated Development Environment，IDE）。前面的编程完全可以用一个简单的文本编辑器来完成。然而，你可能需要在各个文件之间跳转，查询某个函数的声明和实现。当文件很多时，这仍然很烦琐。IDE 为开发者提供了跳转、补全、断点调试等很多方便的功能，所以，我们建议读者选择一个 IDE 进行开发。

Linux 下的 IDE 有很多种。虽然与 Windows 下的 Visual Studio 还有一些差距，不过支持 C++ 开发的也有好几种，例如 Eclipse、Qt Creator、Code::Blocks、Clion、Visual Studio Code，等等。同样，我们不强制读者使用某种特定的 IDE，而仅给出我们的建议。我们使用的是 KDevelop 和 Clion（如图 2-12 和图 2-15 所示）[①]。KDevelop 是一款免费软件，位于 Ubuntu 系统的软件仓库中，意味着你可以用 apt-get 来安装它；而 Clion 则是收费软件，但持有学生邮箱可以免费使用一年。二者都是很好的 C++ 开发环境，优点列举如下：

1. 支持 cmake 工程。
2. 对 C++ 支持较好（包括 11 及之后的标准）。有高亮、跳转、补全等功能，能自动排版代码。
3. 能方便地看到各个文件和目录树。
4. 有一键编译、断点调试等功能。

基本上，它们都具备人们对 IDE 的正常功能要求，所以读者不妨尝试一下。下面我们稍微花一点篇幅介绍 KDevelop 和 Clion。

KDevelop 的使用

KDevelop 原生支持 cmake 工程。具体做法是，在终端建立 CMakeLists.txt 后，用 KDevelop 中的“工程 → 打开/导入工程”打开 CMakeLists.txt。软件会询问你几个问题，并默认建立一个 build 文件夹，帮你调用刚才的 cmake 和 make 命令。只要按下快捷键 F8，这些都可以自动完成。图 2-12 中界面的下面部分就显示了编译信息。

我们把适应 IDE 的任务交给读者自己来完成。如果你是从 Windows 转过来的，会觉得它的界面与 Visual C++ 或 Visual Studio 挺相似。请用 KDevelop 打开刚才的工程然后进行编译，看看它输出什么信息。相信你会觉得比打开终端更方便。

---

①Visual Studio Code 日趋完善，本身又免费，在开发者中大受欢迎，你不妨试试。

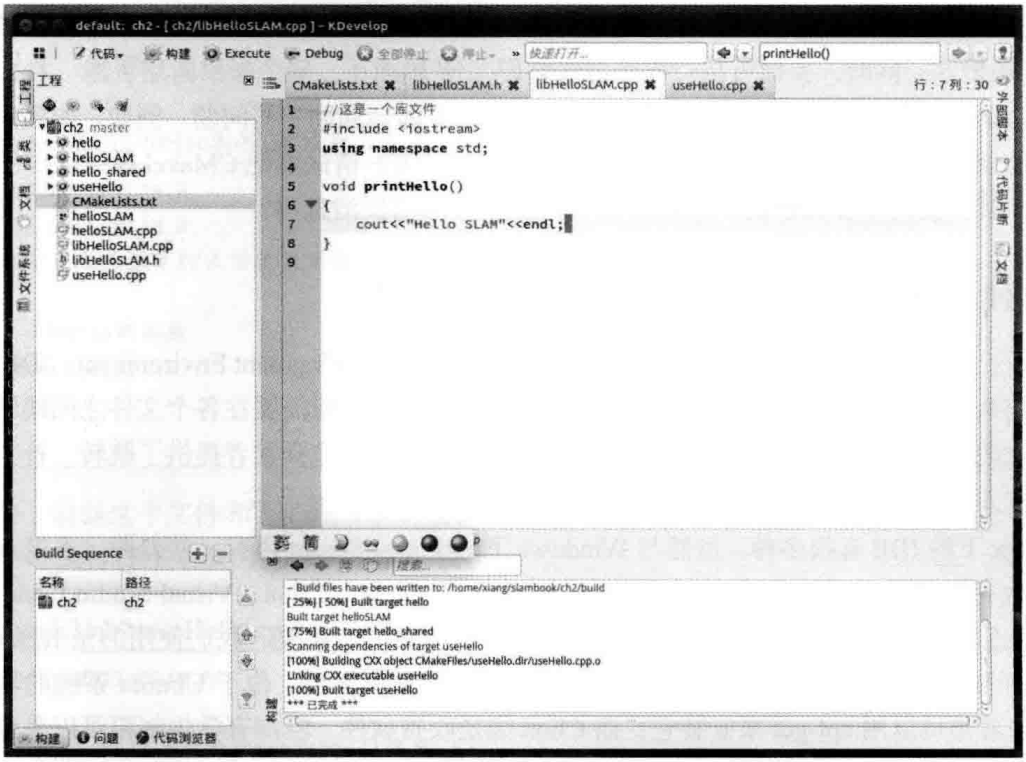


图 2-12 KDevelop 界面

本节重点讲如何在 IDE 中进行调试。在 Windows 下编程的同学多半会有在 Visual Studio 下断点调试的经历。不过在 Linux 中，默认的调试工具 gdb 只提供了文本界面，对新手来讲不太方便。有些 IDE 提供了断点调试功能（底层仍旧是 gdb），KDevelop 就是其中之一。要使用 KDevelop 的断点调试功能，你需要完成以下几件事：

1. 在 CMakeLists.txt 中把工程调为 Debug 编译模式，同时不要使用优化选项（默认不使用）。
2. 告诉 KDevelop 你想运行哪个程序。如果有参数，则要配置它的参数和工作目录。
3. 进入断点调试界面就可以单步运行，看到中间变量的值。

第一步，在 CMakeLists.txt 中加入下面的命令来设置编译模式：

slambook2/ch2/CMakeLists.txt

```
set( CMAKE_BUILD_TYPE "Debug" )
```

cmake 自带一些编译相关的内部变量，它们可以对编译过程进行更精细的控制。对于编译类型，通常有调试用的 Debug 模式与发布用的 Release 模式。在 Debug 模式中，程序运行较慢，但可以进行断点调试；而 Release 模式的运行速度较快，但没有调试信息。我们把程序设置成 Debug 模式，就能放置断点了。接下来，告诉 KDevelop 你想启动哪个程序。

第二步，打开“运行 → 配置启动器”，然后单击左侧的“Add New→ 应用程序”。在这一步中，我们的任务是告诉 KDevelop 想要启动哪一个程序。如图 2-13 所示，既可以选择一个 cmake 的工程目标（也就是我们用 add_executable 指令构建的可执行程序），也可以直接指向一个二进制文件。建议使用第二种方式，根据我们的经验，出现的问题更少。

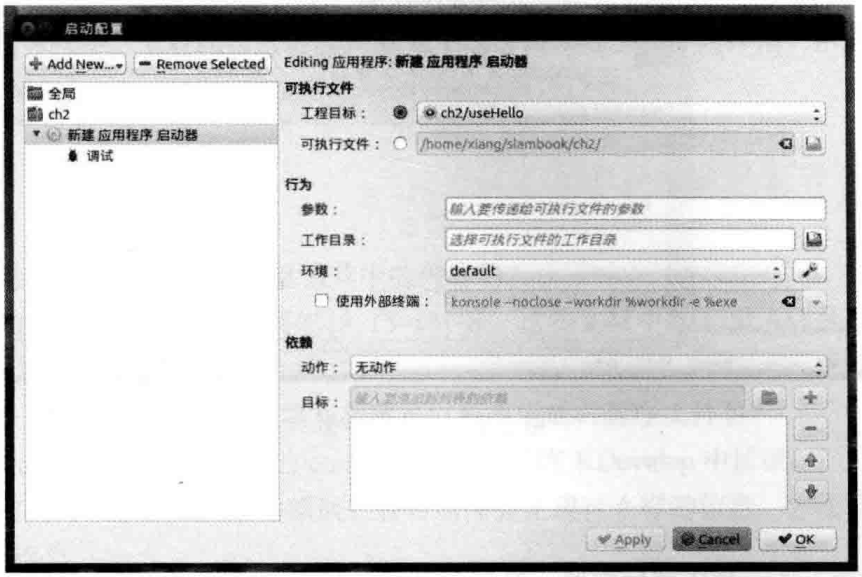


图 2-13　启动器设置界面

在第二栏里，可以设置程序的运行参数和工作目录。有时程序是有运行参数的，它们会作为 main 函数的参数被传入。如果没有则可以留空，对于工作目录也是如此。配置好这两项后，单击“OK”按钮保存配置结果。

刚才这几步我们配置了一个应用程序的启动项。对于每一个启动项，我们可以单击“Execute”按钮直接启动这个程序，也可以单击“Debug”按钮对它进行断点调试。读者可以试着单击“Execute”按钮，查看输出的结果。现在，为了调试这个程序，单击 printHello 那行的左侧，增加一个断点。然后单击“Debug”按钮，程序会停留在断点处等待，如图 2-14 所示。

调试时，KDevelop 会切换到调试模式，界面会发生一点变化。在断点处，可以用单步运行（F10 键）、单步跟进（F11 键）、单步跳出（F12 键）功能控制程序的运行。同时，可以展开左侧的界面，查看局部变量的值。或者单击“停止”按钮，结束调试。调试结束后，KDevelop 会回到正常的开发界面。

现在，你应该熟悉了整个断点调试的流程。今后，如果在程序运行阶段发生错误，导致程序崩溃，就可以用断点调试确定出错的位置，然后加以修正[①]。

①而不是直接给我们发邮件询问怎么处理遇到的问题。

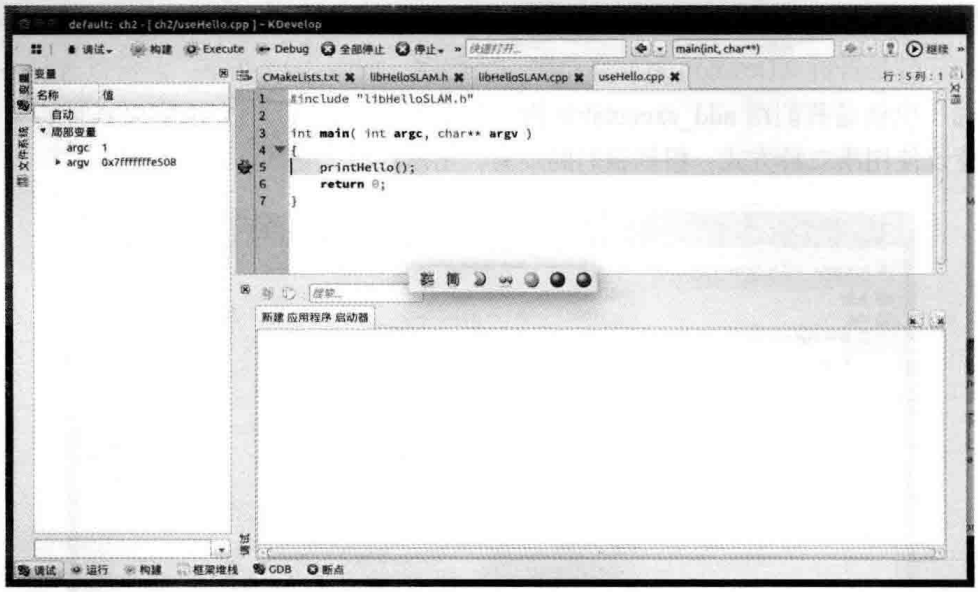


图 2-14 调试界面

Clion 的使用

Clion 与 KDevelop 相比更加完善，但是它需要正版账号，同时对主机的要求也更高[①]。在 Clion 中，你同样可以打开一个 CMakeLists.txt，或者指定目录。Clion 会替你完成 cmake-make 的过程。它的运行界面如图 2-15 所示。

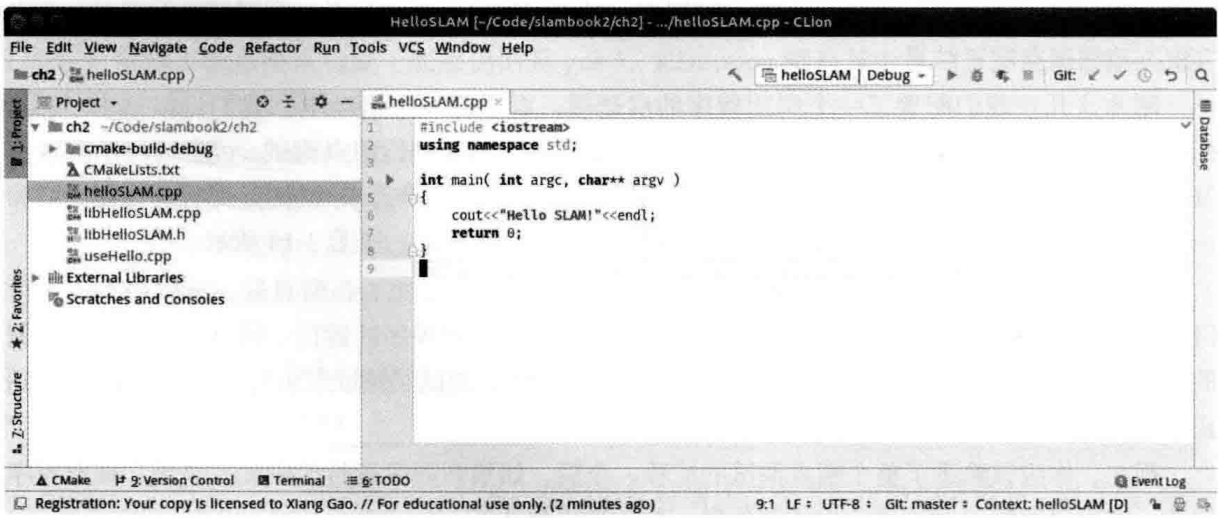


图 2-15 Clion 运行界面

①Clion 在 2018 年之后的版本异常卡顿，推荐用 2017 年前后的发布版。

同样地，在打开 Clion 之后，可以在界面右上角处选择你想运行或调试的程序，调整它们的启动参数和工作目录。单击该栏的小甲虫按钮可以启动断点调试模式。Clion 还有许多方便的功能，例如自动创建类、函数改名、自动调整编码风格等，请一定要试一试。

好了，如果你已经熟悉了 IDE 的使用，那么入门章节也就到此为止。你或许已经觉得笔者有些话唠了，所以在今后的实践部分中，我们不会再介绍怎么新建 build 文件夹，调用 cmake 和 make 命令来编译程序。笔者相信读者应该掌握了这些简单的步骤。同样地，由于本书用到的大多数第三方库都是 cmake 工程，你也会不断熟悉这个编译过程。

习题

1. 阅读文献 [1] 和 [14]，你能看懂其中的内容吗？
2.* 阅读 SLAM 的综述文献，例如 [9, 15–18] 等。这些文献中关于 SLAM 的看法与本书有何异同？
3. g++ 命令有哪些参数？怎么填写参数可以更改生成的程序文件名？
4. 使用 build 文件夹来编译你的 cmake 工程，然后在 KDevelop 中试试。
5. 刻意在代码中添加一些语法错误，看看编译会生成什么样的信息。你能看懂 g++ 的错误信息吗？
6. 如果忘了把库链接到可执行程序上，编译会报错吗？报什么样的错？
7.* 阅读《cmake 实践》，了解 cmake 的其他语法。
8.* 完善 Hello SLAM 小程序，把它做成一个小程序库，安装到本地硬盘中。然后，新建一个工程，使用 find_package 找这个库并调用。
9.* 阅读其他 cmake 教学材料，例如`https://github.com/TheErk/CMake-tutorial`。
10. 找到 KDevelop 的官方网站，看看它还有哪些特性。你都用上了吗？
11. 如果在第 1 讲学习了 Vim，那么请试试 KDevelop 的 Vim 编辑功能。

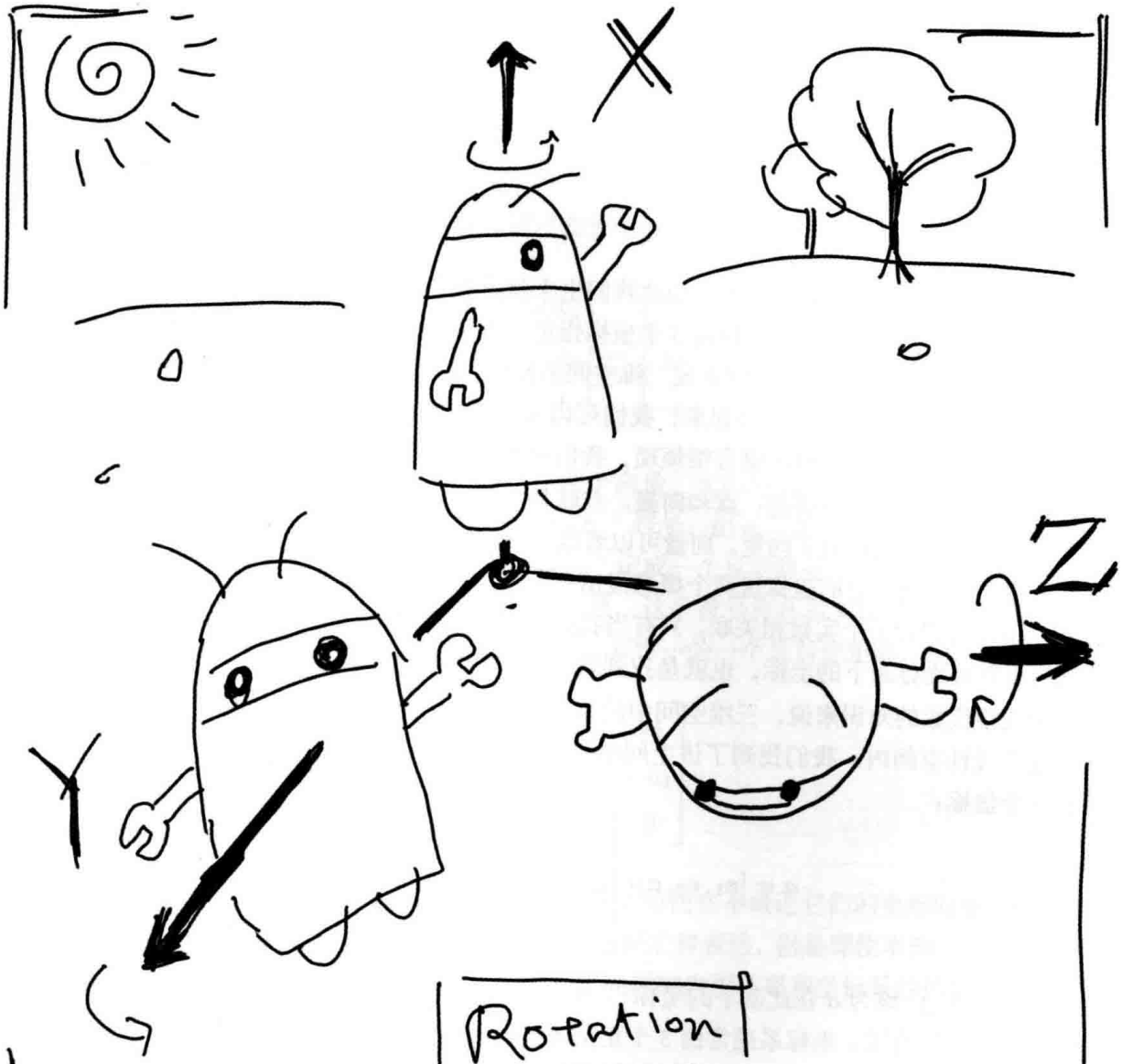
第3讲

三维空间刚体运动

主要目标

1. 理解三维空间的刚体运动描述方式：旋转矩阵、变换矩阵、四元数和欧拉角。
2. 掌握 Eigen 库的矩阵、几何模块的使用方法。

在第 2 讲中，我们讲解了视觉 SLAM 的框架与内容。本讲将介绍视觉 SLAM 的基本问题之一：**如何描述刚体在三维空间中的运动**？直观上看，我们当然知道这由一次旋转加一次平移组成。平移确实没有太大问题，但旋转的处理是件麻烦事。我们将介绍旋转矩阵、四元数、欧拉角的意义，以及它们是如何运算和转换的。在实践部分，我们将介绍线性代数库 Eigen。它提供了 C++ 中的矩阵运算，并且它的 Geometry 模块还提供了四元数等描述刚体运动的结构。Eigen 的优化非常完善，但是它的使用方法有一些特殊的地方，我们留到程序中再介绍。



Rotation

$$SO(3) = \{ R \mid R^T R = I, \det(R) = 1 \}$$

roll, pitch, yaw

$$q = q_0 + q_1 i + q_2 j + q_3 k$$

$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \in SE(3)$$

3.1 旋转矩阵

3.1.1 点、向量和坐标系

我们日常生活的空间是三维的，因此我们生来就习惯于三维空间的运动。三维空间由 3 个轴组成，所以一个空间点的位置可以由 3 个坐标指定。不过，我们现在要考虑的**刚体**，它不光有位置，还有自身的姿态。相机也可以看成三维空间的刚体，于是位置是指相机在空间中的哪个地方，而姿态则是指相机的朝向。结合起来，我们可以说，“相机正处于空间 $(0,0,0)$ 点处，朝向正前方”这样的话。但是这种自然语言很烦琐，我们更喜欢用数学语言来描述它。

我们从最基本的内容讲起：**点**和**向量**。点就是空间中的基本元素，没有长度，没有体积。把两个点连接起来，就构成了向量。向量可以看成从某点指向另一点的一个箭头。需要提醒读者的是，请不要把向量与它的**坐标**这两个概念混淆。一个向量是空间当中的一样东西，比如 $\boldsymbol{a}$。这里的 $\boldsymbol{a}$ 并不需要和若干个实数相关联。只有当我们指定这个三维空间中的某个**坐标系**时，才可以谈论该向量在此坐标系下的坐标，也就是找到若干个实数对应这个向量。

用线性代数的知识来说，三维空间中的某个点的坐标也可以用 $\mathbb{R}^3$ 来描述。怎么描述呢？假设在这个线性空间内，我们找到了该空间的一组**基**[①] $(\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3)$，那么，任意向量 $\boldsymbol{a}$ 在这组基下就有一个**坐标**：

$$\boldsymbol{a} = [\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = a_1\boldsymbol{e}_1 + a_2\boldsymbol{e}_2 + a_3\boldsymbol{e}_3. \tag{3.1}$$

这里 $(a_1, a_2, a_3)^{\mathrm{T}}$ 称为 $\boldsymbol{a}$ 在此基下的坐标[②]。坐标的具体取值，一是和向量本身有关，二是和坐标系（基）的选取有关。坐标系通常由 3 个正交的坐标轴组成（尽管也可以有非正交的，但实际中很少见）。例如，当给定 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 轴时，$\boldsymbol{z}$ 轴就可以通过右手（或左手）法则由 $\boldsymbol{x} \times \boldsymbol{y}$ 定义出来。根据定义方式的不同，坐标系又分为左手系和右手系。左手系的第 3 个轴与右手系方向相反。大部分 3D 程序库使用右手系（如 OpenGL、3D Max 等），也有部分库使用左手系（如 Unity、Direct3D 等）。

根据基本的线性代数知识，我们可以谈论向量与向量，以及向量与数之间的运算，例如数乘、加法、减法、内积、外积等。数乘和加减法都是相当基本的内容，也符合直观想象。例如，两个向量相加的结果就是把它们各自的坐标相加，减法亦然，等等。这里不再赘述。内外积对读者来说可能有些陌生，这里给出它们的运算方式。对于 $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^3$，通常意义下[③]的内积可以写成

---

①以防读者忘记，基就是张成这个空间的一组线性无关的向量，有些书里也叫**基底**。

②本书的向量为列向量，这和一般的数学书籍类似。

③内积也有形式化的法则，但本书只讨论通常定义的内积。

$$\boldsymbol{a}\cdot\boldsymbol{b}=\boldsymbol{a}^{\mathrm{T}}\boldsymbol{b}=\sum_{i=1}^{3}a_ib_i=|\boldsymbol{a}|\,|\boldsymbol{b}|\cos\langle\boldsymbol{a},\boldsymbol{b}\rangle\,. \tag{3.2}$$

其中 $\langle\boldsymbol{a},\boldsymbol{b}\rangle$ 指向量 $\boldsymbol{a},\boldsymbol{b}$ 的夹角。内积也可以描述向量间的投影关系。而外积则是这个样子：

$$\boldsymbol{a}\times\boldsymbol{b}=\begin{Vmatrix}\boldsymbol{e}_1 & \boldsymbol{e}_2 & \boldsymbol{e}_3\\ a_1 & a_2 & a_3\\ b_1 & b_2 & b_3\end{Vmatrix}=\begin{bmatrix}a_2b_3-a_3b_2\\ a_3b_1-a_1b_3\\ a_1b_2-a_2b_1\end{bmatrix}=\begin{bmatrix}0 & -a_3 & a_2\\ a_3 & 0 & -a_1\\ -a_2 & a_1 & 0\end{bmatrix}\boldsymbol{b}\overset{\text{def}}{=}\boldsymbol{a}^{\wedge}\boldsymbol{b}. \tag{3.3}$$

外积的结果是一个向量，它的方向垂直于这两个向量，大小为 $|\boldsymbol{a}|\,|\boldsymbol{b}|\sin\langle\boldsymbol{a},\boldsymbol{b}\rangle$，是两个向量张成的四边形的有向面积。对于外积运算，我们引入 $^{\wedge}$ 符号，把 $\boldsymbol{a}$ 写成一个矩阵。事实上是一个**反对称矩阵**（Skew-symmetric Matrix）[①]，你可以将 $^{\wedge}$ 记成一个反对称符号。这样就把外积 $\boldsymbol{a}\times\boldsymbol{b}$ 写成了矩阵与向量的乘法 $\boldsymbol{a}^{\wedge}\boldsymbol{b}$，把它变成了线性运算。这个符号将在后文经常用到，请记住它，并且此符号是一个一一映射，意味着任意向量都对应着唯一的一个反对称矩阵，反之亦然：

$$\boldsymbol{a}^{\wedge}=\begin{bmatrix}0 & -a_3 & a_2\\ a_3 & 0 & -a_1\\ -a_2 & a_1 & 0\end{bmatrix}. \tag{3.4}$$

同时，需要提醒读者的是，向量和加减法、内外积，即使在不谈论它们的坐标时也可以计算。例如，虽然内积在有坐标时，可以用两个向量的分量乘积之和表达，但是即使不知道它们的坐标，也可以通过长度和夹角来计算二者的内积。所以两个向量的内积结果和坐标系的选取是无关的。

3.1.2　坐标系间的欧氏变换

我们经常在实际场景中定义各种各样的坐标系。在机器人学中，你会给每一个连杆和关节定义它们的坐标系；在 3D 作图时，我们也会定义每一个长方体、圆柱体的坐标系。如果考虑运动的机器人，那么常见的做法是设定一个惯性坐标系（或者叫世界坐标系），可以认为它是固定不动的，例如图 3-1 中的 $x_{\mathrm{W}},y_{\mathrm{W}},z_{\mathrm{W}}$ 定义的坐标系。同时，相机或机器人是一个移动坐标系，例如 $x_{\mathrm{C}},y_{\mathrm{C}},z_{\mathrm{C}}$ 定义的坐标系。读者可能会问：相机视野中某个向量 $\boldsymbol{p}$，它在相机坐标系下的坐标为 $\boldsymbol{p}_{\mathrm{c}}$，而从世界坐标系下看，它的坐标为 $\boldsymbol{p}_{\mathrm{w}}$，那么，这两个坐标之间是如何转换的呢？这时，就需要先得到该点针对机器人坐标系的坐标值，再根据机器人位姿**变换**到世界坐标系中。我们需要一种数学手段来描述这个变换关系，稍后我们会看到，可以用一个矩阵 $\boldsymbol{T}$ 来描述它。

两个坐标系之间的运动由一个旋转加上一个平移组成，这种运动称为**刚体运动**。相机运动就

①反对称矩阵 $\boldsymbol{A}$ 满足 $\boldsymbol{A}^{\mathrm{T}}=-\boldsymbol{A}$。

是一个刚体运动。刚体运动过程中，同一个向量在各个坐标系下的长度和夹角都不会发生变化。想象你把手机抛到空中，在它落地摔碎之前[①]，只可能有空间位置和姿态的不同，而它自己的长度、各个面的角度等性质不会有任何变化。手机并不会像橡皮那样一会儿被挤扁，一会儿被拉长。此时，我们说手机坐标系到世界坐标之间，相差了一个**欧氏变换**（Euclidean Transform）。

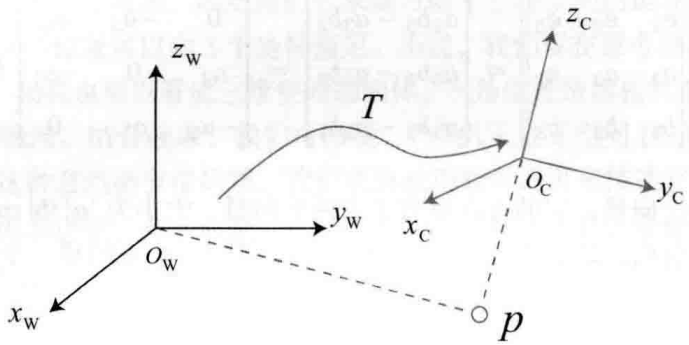


图 3-1 坐标变换。对于同一个向量 $\boldsymbol{p}$，它在世界坐标系下的坐标 $\boldsymbol{p}_W$ 和在相机坐标系下的坐标 $\boldsymbol{p}_c$ 是不同的。这个变换关系由变换矩阵 $\boldsymbol{T}$ 来描述

欧氏变换由旋转和平移组成。我们首先考虑旋转。设某个单位正交基 $(\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3)$ 经过一次旋转变成了 $(\boldsymbol{e}'_1, \boldsymbol{e}'_2, \boldsymbol{e}'_3)$。那么，对于同一个向量 $\boldsymbol{a}$（该向量并没有随着坐标系的旋转而发生运动），它在两个坐标系下的坐标为 $[a_1, a_2, a_3]^T$ 和 $[a'_1, a'_2, a'_3]^T$。因为向量本身没变，所以根据坐标的定义，有

$$[\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = [\boldsymbol{e}'_1, \boldsymbol{e}'_2, \boldsymbol{e}'_3] \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix}. \tag{3.5}$$

为了描述两个坐标之间的关系，我们对上述等式的左右两边同时左乘 $\begin{bmatrix} \boldsymbol{e}_1^T \\ \boldsymbol{e}_2^T \\ \boldsymbol{e}_3^T \end{bmatrix}$，那么左边的系数就变成了单位矩阵，所以：

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \boldsymbol{e}_1^T\boldsymbol{e}'_1 & \boldsymbol{e}_1^T\boldsymbol{e}'_2 & \boldsymbol{e}_1^T\boldsymbol{e}'_3 \\ \boldsymbol{e}_2^T\boldsymbol{e}'_1 & \boldsymbol{e}_2^T\boldsymbol{e}'_2 & \boldsymbol{e}_2^T\boldsymbol{e}'_3 \\ \boldsymbol{e}_3^T\boldsymbol{e}'_1 & \boldsymbol{e}_3^T\boldsymbol{e}'_2 & \boldsymbol{e}_3^T\boldsymbol{e}'_3 \end{bmatrix} \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} \overset{\text{def}}{=} \boldsymbol{R}\boldsymbol{a}'. \tag{3.6}$$

我们把中间的矩阵拿出来，定义成一个矩阵 $\boldsymbol{R}$。这个矩阵由两组基之间的内积组成，刻画了旋转

①请不要付诸实践。

前后同一个向量的坐标变换关系。只要旋转是一样的，这个矩阵就是一样的。可以说，矩阵 $\boldsymbol{R}$ 描述了旋转本身。因此，称为**旋转矩阵**（Rotation Matrix）。同时，该矩阵各分量是两个坐标系基的内积，由于基向量的长度为 1，所以实际上是各基向量夹角的余弦值。所以这个矩阵也叫**方向余弦矩阵**（Direction Cosine Matrix）。我们后文统一称它为旋转矩阵。

旋转矩阵有一些特别的性质。事实上，它是一个行列式为 1 的正交矩阵[①,②]。反之，行列式为 1 的正交矩阵也是一个旋转矩阵。所以，可以将 n 维旋转矩阵的集合定义如下：

$$\mathrm{SO}(n) = \{\boldsymbol{R} \in \mathbb{R}^{n\times n} | \boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}, \det(\boldsymbol{R}) = 1\}. \tag{3.7}$$

$\mathrm{SO}(n)$ 是**特殊正交群**（Special Orthogonal Group）的意思。我们把“群”的内容留到下一讲。这个集合由 n 维空间的旋转矩阵组成，特别地，SO(3) 就是指三维空间的旋转。通过旋转矩阵，我们可以直接谈论两个坐标系之间的旋转变换，而不用再从基开始谈起。

由于旋转矩阵为正交矩阵，它的逆（即转置）描述了一个相反的旋转。按照上面的定义方式，有

$$\boldsymbol{a}' = \boldsymbol{R}^{-1}\boldsymbol{a} = \boldsymbol{R}^{\mathrm{T}}\boldsymbol{a}. \tag{3.8}$$

显然，$\boldsymbol{R}^{\mathrm{T}}$ 刻画了一个相反的旋转。

在欧氏变换中，除了旋转还有平移。考虑世界坐标系中的向量 $\boldsymbol{a}$，经过一次旋转（用 $\boldsymbol{R}$ 描述）和一次平移 $\boldsymbol{t}$ 后，得到了 $\boldsymbol{a}'$，那么把旋转和平移合到一起，有

$$\boldsymbol{a}' = \boldsymbol{R}\boldsymbol{a} + \boldsymbol{t}. \tag{3.9}$$

其中，$\boldsymbol{t}$ 称为平移向量。相比于旋转，平移部分只需把平移向量加到旋转之后的坐标上，非常简单。通过上式，我们用一个旋转矩阵 $\boldsymbol{R}$ 和一个平移向量 $\boldsymbol{t}$ 完整地描述了一个欧氏空间的坐标变换关系。实际当中，我们会定义坐标系 1、坐标系 2，那么向量 $\boldsymbol{a}$ 在两个坐标系下的坐标为 $\boldsymbol{a}_1, \boldsymbol{a}_2$，它们之间的关系应该是：

$$\boldsymbol{a}_1 = \boldsymbol{R}_{12}\boldsymbol{a}_2 + \boldsymbol{t}_{12}. \tag{3.10}$$

这里的 $\boldsymbol{R}_{12}$ 是指“把坐标系 2 的向量变换到坐标系 1”中。由于向量乘在这个矩阵的右边，它的下标是**从右读到左**的。这也是本书的习惯写法。坐标变换很容易搞混，特别是存在多个坐标系的情况下。同理，如果我们要表达“从 1 到 2 的旋转矩阵”时，就写成 $\boldsymbol{R}_{21}$。请读者务必清楚本书的记法，因为不同书籍里写法不同，有的会记成左上/下标，而本书写在右侧下标。

关于平移 $\boldsymbol{t}_{12}$，它实际对应的是坐标系 1 原点指向坐标系 2 原点的向量，**在坐标系 1 下取的**

①正交矩阵即逆为自身转置的矩阵。旋转矩阵的正交性可以直接由定义得出。

②行列式为 1 是人为定义的，实际上只要求它的行列式为 ± 1，但行列式为 -1 的称为瑕旋转，即一次旋转加一次反射。

坐标，所以笔者建议读者把它记作“从 1 到 2 的向量”。但是反过来的 $\boldsymbol{t}_{21}$，即从 2 指向 1 的向量**在坐标系 2 下的坐标**，却并不等于 $-\boldsymbol{t}_{12}$，而是和两个系的旋转还有关系[①]。所以，当初学者问“我的坐标在哪里”这样的问题时，我们需要清楚地说明这句话的含义。这里“我的坐标”实际上指从世界坐标系指向自己坐标系原点的向量，在世界坐标系下取到的坐标。对应到数学符号上，应该是 $\boldsymbol{t}_{\mathrm{WC}}$ 的取值。同理，它也不是 $-\boldsymbol{t}_{\mathrm{CW}}$。

3.1.3 变换矩阵与齐次坐标

式 (3.9) 完整地表达了欧氏空间的旋转与平移，不过还存在一个小问题：这里的变换关系不是一个线性关系。假设我们进行了两次变换：$\boldsymbol{R}_1, \boldsymbol{t}_1$ 和 $\boldsymbol{R}_2, \boldsymbol{t}_2$：

$$\boldsymbol{b} = \boldsymbol{R}_1 \boldsymbol{a} + \boldsymbol{t}_1, \quad \boldsymbol{c} = \boldsymbol{R}_2 \boldsymbol{b} + \boldsymbol{t}_2.$$

那么，从 $\boldsymbol{a}$ 到 $\boldsymbol{c}$ 的变换为

$$\boldsymbol{c} = \boldsymbol{R}_2 \left(\boldsymbol{R}_1 \boldsymbol{a} + \boldsymbol{t}_1 \right) + \boldsymbol{t}_2.$$

这样的形式在变换多次之后会显得很啰嗦。因此，我们引入齐次坐标和变换矩阵，重写式 (3.9)：

$$\begin{bmatrix} \boldsymbol{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix} \overset{\text{def}}{=} \boldsymbol{T} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix}. \tag{3.11}$$

这是一个数学技巧：我们在一个三维向量的末尾添加 1，将其变成了四维向量，称为**齐次坐标**。对于这个四维向量，我们可以把旋转和平移写在一个矩阵里，使得整个关系变成线性关系。该式中，矩阵 $\boldsymbol{T}$ 称为**变换矩阵（Transform Matrix）**。

我们暂时用 $\tilde{\boldsymbol{a}}$ 表示 $\boldsymbol{a}$ 的齐次坐标。那么依靠齐次坐标和变换矩阵，两次变换的叠加就可以有很好的形式：

$$\tilde{\boldsymbol{b}} = \boldsymbol{T}_1 \tilde{\boldsymbol{a}}, \ \tilde{\boldsymbol{c}} = \boldsymbol{T}_2 \tilde{\boldsymbol{b}} \quad \Rightarrow \tilde{\boldsymbol{c}} = \boldsymbol{T}_2 \boldsymbol{T}_1 \tilde{\boldsymbol{a}}. \tag{3.12}$$

但是区分齐次和非齐次坐标的符号令我们感到厌烦，因为此处只需要在向量末尾添加 1 或者去掉 1[②]。所以，在不引起歧义的情况下，以后我们就直接把它写成 $\boldsymbol{b} = \boldsymbol{T}\boldsymbol{a}$ 的样子，默认其中进行了齐次坐标的转换[③]。

关于变换矩阵 $\boldsymbol{T}$，它具有比较特别的结构：左上角为旋转矩阵，右侧为平移向量，左下角为

①尽管从向量层面来看，它们确实是反向的关系，但这两个向量的坐标值并不是相反数。你能想清楚这是为什么吗？

②但齐次坐标的用途不止于此，我们在第 7 讲还会再介绍。

③注意，不进行齐次坐标转换时，这边的乘法在矩阵维度上是不成立的。

0 向量，右下角为 1。这种矩阵又称为特殊欧氏群（Special Euclidean Group）：

$$\mathrm{SE}(3)=\left\{\boldsymbol{T}=\begin{bmatrix}\boldsymbol{R} & \boldsymbol{t}\\ \boldsymbol{0}^{\mathrm{T}} & 1\end{bmatrix}\in\mathbb{R}^{4\times4}|\boldsymbol{R}\in\mathrm{SO}(3),\boldsymbol{t}\in\mathbb{R}^{3}\right\}. \tag{3.13}$$

与 SO(3) 一样，求解该矩阵的逆表示一个反向的变换：

$$\boldsymbol{T}^{-1}=\begin{bmatrix}\boldsymbol{R}^{\mathrm{T}} & -\boldsymbol{R}^{\mathrm{T}}\boldsymbol{t}\\ \boldsymbol{0}^{\mathrm{T}} & 1\end{bmatrix}. \tag{3.14}$$

同样，我们用 $\boldsymbol{T}_{12}$ 这样的写法来表示从 2 到 1 的变换。并且，为了保持符号的简洁，在不引起歧义的情况下，以后不刻意区别齐次坐标与普通坐标的符号，**默认使用的是符合运算法则的那一种**。例如，当我们写 $\boldsymbol{Ta}$ 时，使用的是齐次坐标（不然没法计算）。而写 $\boldsymbol{Ra}$ 时，使用的是非齐次坐标。如果写在一个等式中，就假设齐次坐标到普通坐标的转换是已经做好了的——因为齐次坐标和非齐次坐标之间的转换事实上非常容易，而在 C++ 程序中你可以使用**运算符重载**来完成这个功能，保证在程序中看到的运算是统一的。

回顾：首先，我们介绍了向量及其坐标表示，并介绍了向量间的运算；然后，坐标系之间的运动由欧氏变换描述，它由平移和旋转组成。旋转可以由旋转矩阵 SO(3) 描述，而平移直接由一个 $\mathbb{R}^3$ 向量描述。最后，如果将平移和旋转放在一个矩阵中，就形成了变换矩阵 SE(3)。

3.2　实践：Eigen

本讲的实践部分有两节。本节将讲解如何使用 Eigen 表示矩阵和向量，随后，引申至旋转矩阵与变换矩阵的计算。本节的代码在 slambook2/ch3/useEigen 中。

Eigen[①]是一个 C++ 开源线性代数库。它提供了快速的有关矩阵的线性代数运算，还包括解方程等功能。许多上层的软件库也使用 Eigen 进行矩阵运算，包括 g2o、Sophus 等。呼应本讲的理论部分，我们来学习 Eigen 的编程。

你的电脑上可能还没有安装 Eigen。请输入以下命令进行安装：

终端输入：

```
sudo apt-get install libeigen3-dev
```

大部分常用的库都已在 Ubuntu 软件源中提供。若想安装某个库，不妨先搜索 Ubuntu 的软件源中是否已提供。通过 apt 命令，我们能够方便地安装 Eigen。回顾第 2 讲的知识，我们知道一

①官方主页：http://eigen.tuxfamily.org/index.php?title=Main_Page。

个库由头文件和库文件组成。Eigen 头文件的默认位置在“/usr/include/eigen3/”中。如果不确定，则可以输入以下命令查找：

终端输入：

```
sudo updatedb
locate eigen3
```

与其他库相比，Eigen 的特殊之处在于，它是一个纯用头文件搭建起来的库（这非常神奇！）。这意味着你只能找到它的头文件，而没有类似.so 或.a 的二进制文件。在使用时，只需引入 Eigen 的头文件即可，不需要链接库文件（因为它没有库文件）。下面写一段代码来实际练习 Eigen 的使用：

slambook2/ch3/useEigen/eigenMatrix.cpp

```cpp
#include <iostream>
using namespace std;

#include <ctime>
// Eigen核心部分
#include <Eigen/Core>
// 稠密矩阵的代数运算（逆、特征值等）
#include <Eigen/Dense>
using namespace Eigen;

#define MATRIX_SIZE 50

/****************************
* 本程序演示了Eigen基本类型的使用
****************************/

int main(int argc, char **argv) {
    // Eigen中所有向量和矩阵都是Eigen::Matrix，它是一个模板类。它的前三个参数为数据类型、行、列
    // 声明一个2*3的float矩阵
    Matrix<float, 2, 3> matrix_23;

    // 同时，Eigen通过typedef提供了许多内置类型，不过底层仍是Eigen::Matrix
    // 例如，Vector3d实质上是Eigen::Matrix<double, 3, 1>，即三维向量
    Vector3d v_3d;
    // 这是一样的
    Matrix<float, 3, 1> vd_3d;

    // Matrix3d实质上是Eigen::Matrix<double, 3, 3>
```

```
Matrix3d matrix_33 = Matrix3d::Zero(); //初始化为零
// 如果不确定矩阵大小，可以使用动态大小的矩阵
Matrix<double, Dynamic, Dynamic> matrix_dynamic;
// 更简单的
MatrixXd matrix_x;
// 这种类型还有很多，我们不一一列举

// 下面是对Eigen阵的操作
// 输入数据（初始化）
matrix_23 << 1, 2, 3, 4, 5, 6;
// 输出
cout << "matrix 2x3 from 1 to 6: \n" << matrix_23 << endl;

// 用()访问矩阵中的元素
cout << "print matrix 2x3: " << endl;
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) cout << matrix_23(i, j) << "\t";
    cout << endl;
}

// 矩阵和向量相乘（实际上仍是矩阵和矩阵）
v_3d << 3, 2, 1;
vd_3d << 4, 5, 6;

// 但是在Eigen里你不能混合两种不同类型的矩阵，像这样是错的
// Matrix<double, 2, 1> result_wrong_type = matrix_23 * v_3d;
// 应该显式转换
Matrix<double, 2, 1> result = matrix_23.cast<double>() * v_3d;
cout << "[1,2,3;4,5,6]*[3,2,1]=" << result.transpose() << endl;

Matrix<float, 2, 1> result2 = matrix_23 * vd_3d;
cout << "[1,2,3;4,5,6]*[4,5,6]: " << result2.transpose() << endl;

// 同样，你不能搞错矩阵的维度
// 试着取消下面的注释，看看Eigen会报什么错
// Eigen::Matrix<double, 2, 3> result_wrong_dimension = matrix_23.cast<double>() * v_3d;

// 一些矩阵运算
// 四则运算就不演示了，直接用+-*/即可
matrix_33 = Matrix3d::Random();      // 随机数矩阵
cout << "random matrix: \n" << matrix_33 << endl;
cout << "transpose: \n" << matrix_33.transpose() << endl; // 转置
cout << "sum: " << matrix_33.sum() << endl;              // 各元素和
```

```
72    cout << "trace: " << matrix_33.trace() << endl;            // 迹
73    cout << "times 10: \n" << 10 * matrix_33 << endl;          // 数乘
74    cout << "inverse: \n" << matrix_33.inverse() << endl;      // 逆
75    cout << "det: " << matrix_33.determinant() << endl;        // 行列式
76
77    // 特征值
78    // 实对称矩阵可以保证对角化成功
79    SelfAdjointEigenSolver<Matrix3d> eigen_solver(matrix_33.transpose() * matrix_33);
80    cout << "Eigen values = \n" << eigen_solver.eigenvalues() << endl;
81    cout << "Eigen vectors = \n" << eigen_solver.eigenvectors() << endl;
82
83    // 解方程
84    // 我们求解 matrix_NN * x = v_Nd 方程
85    // N的大小在前边的宏里定义，它由随机数生成
86    // 直接求逆自然是最直接的，但是运算量大
87
88    Matrix<double, MATRIX_SIZE, MATRIX_SIZE> matrix_NN
89        = MatrixXd::Random(MATRIX_SIZE, MATRIX_SIZE);
90    matrix_NN = matrix_NN * matrix_NN.transpose();  // 保证半正定
91    Matrix<double, MATRIX_SIZE, 1> v_Nd = MatrixXd::Random(MATRIX_SIZE, 1);
92
93    clock_t time_stt = clock(); // 计时
94    // 直接求逆
95    Matrix<double, MATRIX_SIZE, 1> x = matrix_NN.inverse() * v_Nd;
96    cout << "time of normal inverse is "
97         << 1000 * (clock() - time_stt) / (double) CLOCKS_PER_SEC << "ms" << endl;
98    cout << "x = " << x.transpose() << endl;
99
100   // 通常用矩阵分解来求解，例如QR分解，速度会快很多
101   time_stt = clock();
102   x = matrix_NN.colPivHouseholderQr().solve(v_Nd);
103   cout << "time of Qr decomposition is "
104        << 1000 * (clock() - time_stt) / (double) CLOCKS_PER_SEC << "ms" << endl;
105   cout << "x = " << x.transpose() << endl;
106
107   // 对于正定矩阵，还可以用cholesky分解来解方程
108   time_stt = clock();
109   x = matrix_NN.ldlt().solve(v_Nd);
110   cout << "time of ldlt decomposition is "
111        << 1000 * (clock() - time_stt) / (double) CLOCKS_PER_SEC << "ms" << endl;
112   cout << "x = " << x.transpose() << endl;
113
114   return 0;
```

```
}
```

这个例程演示了 Eigen 矩阵的基本操作与运算。要编译它，需要在 CMakeLists.txt 里指定 Eigen 的头文件目录：

slambook2/ch3/useEigen/CMakeLists.txt

```
# 添加头文件
include_directories( "/usr/include/eigen3" )
```

重复一遍，因为 Eigen 库只有头文件，所以不需要再用 target_link_libraries 语句将程序链接到库上。不过，对于其他大部分库，大多数情况需要用到链接命令。这里的做法并不见得是最好的，因为其他人可能把 Eigen 安装在了不同位置，那么就必须手动修改头文件目录。在之后的工作中，我们会使用 find_package 命令搜索库，不过在本讲中暂时保持现在的样子。编译好程序后，运行它，可以看到各矩阵的输出结果。

终端输入：

```
% build/eigenMatrix
matrix 2x3 from 1 to 6:
1 2 3
4 5 6
print matrix 2x3:
1   2   3
4   5   6
[1,2,3;4,5,6]*[3,2,1]=10 28
[1,2,3;4,5,6]*[4,5,6]: 32 77
random matrix:
0.680375   0.59688 -0.329554
-0.211234  0.823295  0.536459
0.566198 -0.604897 -0.444451
transpose:
0.680375 -0.211234  0.566198
0.59688  0.823295 -0.604897
-0.329554  0.536459 -0.444451
sum: 1.61307
trace: 1.05922
times 10:
6.80375   5.9688 -3.29554
-2.11234  8.23295  5.36459
5.66198 -6.04897 -4.44451
inverse:
-0.198521   2.22739    2.8357
1.00605 -0.555135  -1.41603
```

```
-1.62213   3.59308   3.28973
det: 0.208598
......
```

由于在代码中给出了详细的注释，在此就不一一解释每行语句了。在本书中，我们将仅给出几处重要地方的说明（后面的实践部分也保持这个风格）。

1. 读者最好亲手输入一遍上面的代码（不包括注释）。至少要编译运行一遍程序。
2. Kdevelop 可能不会提示 C++ 成员运算。请照着上面的内容输入即可，不必理会它是否提示错误。Clion 则会完整地给出提示。
3. Eigen 提供的矩阵和 MATLAB 很相似，几乎所有的数据都当作矩阵来处理。但是，为了达到更高的效率，在 Eigen 中需要指定矩阵的大小和类型。对于在编译时期就知道大小的矩阵，处理起来会比动态变化大小的矩阵更快。因此，对于旋转矩阵、变换矩阵，完全可在编译时确定它们的大小和数据类型。
4. Eigen 内部的矩阵实现比较复杂，这里不做介绍，我们希望你像使用 float、double 等内置数据类型那样使用 Eigen 的矩阵。这应该是符合其设计初衷的。
5. Eigen 矩阵不支持自动类型提升，这和 C++ 的内建数据类型有较大差异。在 C++ 程序中，我们可以把一个 float 数据和 double 数据相加、相乘，**编译器会自动把数据类型转换为最合适的那种**。而在 Eigen 中，出于性能的考虑，必须**显式地**对矩阵类型进行转换。而如果忘了这样做，则 Eigen 会（不太友好地）提示你一个“YOU MIXED DIFFERENT NUMERIC TYPES ...”的编译错误。你可以尝试找到这条信息出现在错误提示的哪个部分。如果错误信息太长最好保存到一个文件里再找。
6. 同理，在计算过程中也需要保证矩阵维数的正确性，否则会出现“YOU MIXED MATRICES OF DIFFERENT SIZES”错误。请不要抱怨这种错误提示方式，对于 C++ 模板元编程，能够提示出可以阅读的信息已经是很幸运的了。以后，若发现 Eigen 出错，你可以直接寻找大写的部分，推测出了什么问题。
7. 我们的例程只介绍了基本的矩阵运算。你可以阅读 Eigen 官网教程：`http://eigen.tuxfamily.org/dox-devel/modules.html` 学习更多的 Eigen 知识。本节只演示了最简单的部分，能看懂演示程序不等于你已经能够熟练操作 Eigen。

最后一段代码中比较了求逆与求 QR 分解的运行效率，你可以在自己的计算机上比较它们的运行时间，两种方法是否有明显的差异？

3.3　旋转向量和欧拉角

3.3.1　旋转向量

我们重新回到理论部分。有了旋转矩阵来描述旋转，有了变换矩阵描述一个 6 自由度的三维刚体运动，是不是已经足够了呢？矩阵表示方式至少有以下两个缺点：

1. SO(3) 的旋转矩阵有 9 个量，但一次旋转只有 3 个自由度。因此这种表达方式是冗余的。同理，变换矩阵用 16 个量表达了 6 自由度的变换。那么，是否有更紧凑的表示呢？
2. 旋转矩阵自身带有约束：它必须是个正交矩阵，且行列式为 1。变换矩阵也是如此。当想估计或优化一个旋转矩阵或变换矩阵时，这些约束会使得求解变得更困难。

因此，我们希望有一种方式能够紧凑地描述旋转和平移。例如，用一个三维向量表达旋转，用一个六维向量表达变换，可行吗？事实上，任意旋转都可以用**一个旋转轴和一个旋转角**来刻画。于是，我们可以使用一个向量，其方向与旋转轴一致，而长度等于旋转角。这种向量称为**旋转向量**（或轴角/角轴，Axis-Angle），只需一个三维向量即可描述旋转。同样，对于变换矩阵，我们使用一个旋转向量和一个平移向量即可表达一次变换。这时的变量维数正好是六维。

考虑某个用 $\boldsymbol{R}$ 表示的旋转。如果用旋转向量来描述，假设旋转轴为一个单位长度的向量 $\boldsymbol{n}$，角度为 θ，那么向量 $\theta\boldsymbol{n}$ 也可以描述这个旋转。于是，我们要问，两种表达方式之间有什么联系吗？事实上推导它们的转换关系并不难。从旋转向量到旋转矩阵的转换过程由**罗德里格斯公式**（Rodrigues's Formula）表明，由于推导过程比较复杂，这里不做描述，只给出转换的结果[①]：

$$\boldsymbol{R} = \cos\theta\boldsymbol{I} + (1-\cos\theta)\,\boldsymbol{n}\boldsymbol{n}^{\mathrm{T}} + \sin\theta\boldsymbol{n}^{\wedge}. \tag{3.15}$$

符号 $^{\wedge}$ 是向量到反对称矩阵的转换符，见式 (3.3)。反之，我们也可以计算从一个旋转矩阵到旋转向量的转换。对于转角 θ，取两边的**迹**[②]，有

$$\begin{aligned}\operatorname{tr}(\boldsymbol{R}) &= \cos\theta\operatorname{tr}(\boldsymbol{I}) + (1-\cos\theta)\operatorname{tr}\left(\boldsymbol{n}\boldsymbol{n}^{\mathrm{T}}\right) + \sin\theta\operatorname{tr}(\boldsymbol{n}^{\wedge}) \\ &= 3\cos\theta + (1-\cos\theta) \\ &= 1 + 2\cos\theta\end{aligned} \tag{3.16}$$

因此：

$$\theta = \arccos\frac{\operatorname{tr}(\boldsymbol{R})-1}{2}. \tag{3.17}$$

---

①感兴趣的读者请参见https://en.wikipedia.org/wiki/Rodrigues%27_rotation_formula，事实上第 4 讲会从李代数层面给出一个证明。

②求迹（trace）即是求矩阵的对角线元素之和。

关于转轴 $\boldsymbol{n}$，旋转轴上的向量在旋转后不发生改变，说明：

$$\boldsymbol{R}\boldsymbol{n} = \boldsymbol{n}. \tag{3.18}$$

因此，转轴 $\boldsymbol{n}$ 是矩阵 $\boldsymbol{R}$ 特征值 1 对应的特征向量。求解此方程，再归一化，就得到了旋转轴。读者也可以从“旋转轴经过旋转之后不变”的几何角度看待这个方程。顺便提一下，这里的两个转换公式在第 4 讲仍将出现，你会发现它们正是 SO(3) 上李群与李代数的对应关系。

3.3.2 欧拉角

下面来介绍欧拉角。

无论是旋转矩阵还是旋转向量，它们虽然能描述旋转，但对人类来说是非常不直观的。当我们看到一个旋转矩阵或旋转向量时，很难想象出这个旋转究竟是什么样的。当它们变换时，我们也不知道物体是在向哪个方向转动。而欧拉角则提供了一种非常直观的方式来描述旋转——它使用了 **3 个分离的转角**，把一个旋转分解成 3 次绕不同轴的旋转。而人类很容易理解绕单个轴旋转的过程。但是，由于分解方式有许多种，所以欧拉角也存在着众多不同的、易于混淆的定义方法。例如，先绕 X 轴，再绕 Y 轴，最后绕 Z 轴旋转，就得到了一个 XYZ 轴的旋转。同理，可以定义 ZYZ、ZYX 等旋转方式。如果讨论得更细一些，则还需要区分每次是绕**固定轴**旋转的，还是绕**旋转之后的轴**旋转的，这也会给出不一样的定义方式。

这种定义方式上的不确定性带来了很多实际当中的困难，所幸在特定领域内，欧拉角通常有统一的定义方式。你或许在航空、航模中听说过“俯仰角”“偏航角”这些词。欧拉角当中比较常用的一种，便是用“偏航 – 俯仰 – 滚转”（yaw-pitch-roll）3 个角度来描述一个旋转。它等价于 ZYX 轴的旋转，因此就以 ZYX 为例。假设一个刚体的前方（朝向我们的方向）为 X 轴，右侧为 Y 轴，上方为 Z 轴，如图 3-2 所示。那么，ZYX 转角相当于把任意旋转分解成以下 3 个轴上的转角：

1. 绕物体的 Z 轴旋转，得到偏航角 yaw。
2. 绕**旋转之后**的 Y 轴旋转，得到俯仰角 pitch。
3. 绕**旋转之后**的 X 轴旋转，得到滚转角 roll。

此时，可以使用 $[r, p, y]^{\mathrm{T}}$ 这样一个三维的向量描述任意旋转。这个向量十分直观，我们可以从这个向量想象出旋转的过程。其他的欧拉角也是通过这种方式，把旋转分解到 3 个轴上，得到一个三维的向量，只不过选用的轴及顺序不一样。这里介绍的 rpy 角是比较常用的一种，只有很少的欧拉角种类会有 rpy 这样脍炙人口的名字。不同的欧拉角是按照旋转轴的顺序来称呼的。例如，rpy 角的旋转顺序是 ZYX。同样，也有 XYZ、ZYZ 这样的欧拉角——但是它们就没有专门的名字了。值得一提的是，大部分领域在使用欧拉角时都有各自的坐标方向和顺序上的习惯，不一定和我们这里说的相同。

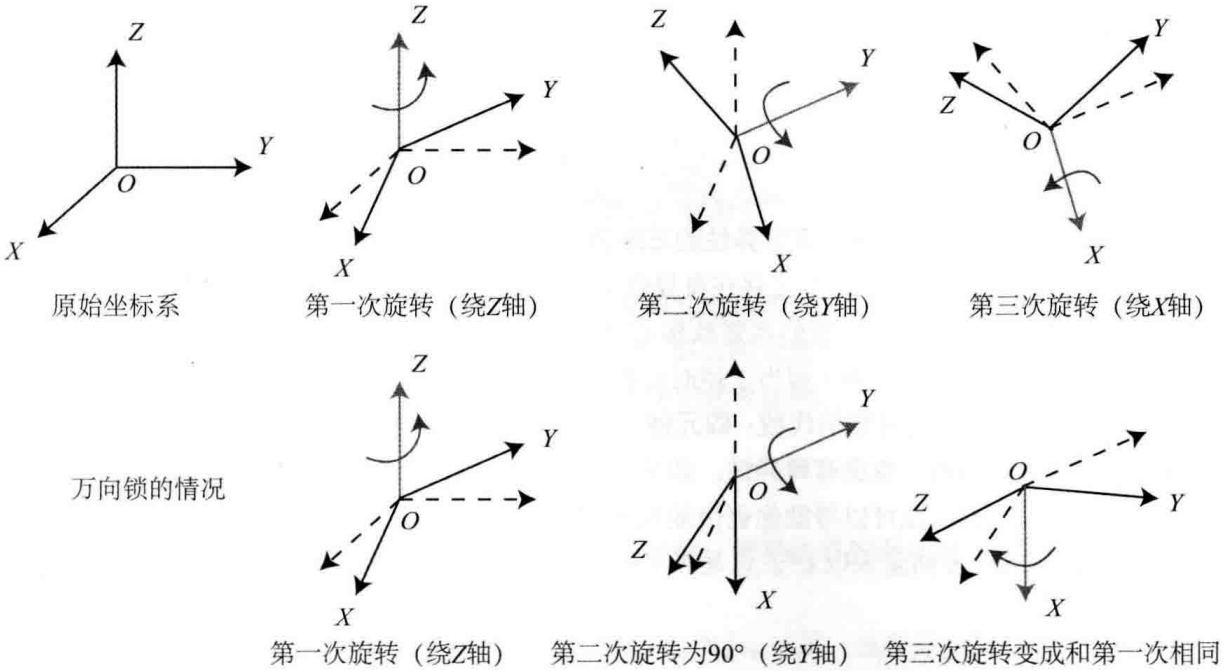


图 3-2　欧拉角的旋转示意图。上方为 ZYX 转角定义。下方为 pitch $= 90°$ 时，第三次旋转与第一次滚转角相同，使得系统丢失了一个自由度。如果你还没有理解万向锁，则可以看看相关视频，理解起来会更方便

欧拉角的一个重大缺点是会碰到著名的**万向锁问题**（Gimbal Lock[①]）：在俯仰角为 $\pm 90°$ 时，第一次旋转与第三次旋转将使用同一个轴，使得系统丢失了一个自由度（由 3 次旋转变成了 2 次旋转）。这被称为奇异性问题，在其他形式的欧拉角中也同样存在。理论上可以证明，只要想用 3 个实数来表达三维旋转，都会不可避免地碰到奇异性问题[②]。由于这种原理，欧拉角不适用于插值和迭代，往往只用于人机交互中。我们也很少在 SLAM 程序中直接使用欧拉角表达姿态，同样不会在滤波或优化中使用欧拉角表达旋转（因为它具有奇异性）。不过，若你想验证自己的算法是否有错，转换成欧拉角能够帮你快速分辨结果是否正确。在某些主体主要为 2D 运动的场合（例如扫地机、自动驾驶车辆），我们也可以把旋转分解成三个欧拉角，然后把其中一个（例如偏航角）拿出来作为定位信息输出。

①https://en.wikipedia.org/wiki/Gimbal_lock。

②旋转向量也有奇异性，发生在转角 θ 超过 2π 而产生周期性时。

3.4 四元数

3.4.1 四元数的定义

旋转矩阵用 9 个量描述 3 自由度的旋转，具有冗余性；欧拉角和旋转向量是紧凑的，但具有奇异性。事实上，我们**找不到不带奇异性的三维向量描述方式**[19]。这有点类似于用两个坐标表示地球表面（如经度和纬度），将必定存在奇异性（纬度为 $\pm 90°$ 时经度无意义）。

回忆以前学习过的复数。我们用复数集 $\mathbb{C}$ 表示复平面上的向量，而复数的乘法则表示复平面上的旋转：例如，乘上复数 i 相当于逆时针把一个复向量旋转 90°。类似地，在表达三维空间旋转时，也有一种类似于复数的代数：**四元数**（Quaternion）。四元数是 Hamilton 找到的一种扩展的复数。它**既是紧凑的，也没有奇异性**。如果说缺点，四元数不够直观，其运算稍复杂些。

把四元数与复数类比可以帮助你更快地理解四元数。例如，当我们想要将复平面的向量旋转 θ 角时，可以给这个复向量乘以 $\mathrm{e}^{\mathrm{i}\theta}$。这是极坐标表示的复数，它也可以写成普通的形式，只要使用欧拉公式即可：

$$\mathrm{e}^{\mathrm{i}\theta} = \cos\theta + \mathrm{i}\sin\theta. \tag{3.19}$$

这正是一个单位长度的复数。所以，在二维情况下，旋转可以由**单位复数**来描述。类似地，我们会看到，三维旋转可以由**单位四元数**来描述。

一个四元数 $\boldsymbol{q}$ 拥有一个实部和三个虚部。本书把实部写在前面（也有地方把实部写在后面），像下面这样：

$$\boldsymbol{q} = q_0 + q_1\mathrm{i} + q_2\mathrm{j} + q_3\mathrm{k}, \tag{3.20}$$

其中，i, j, k 为四元数的三个虚部。这三个虚部满足以下关系式：

$$\begin{cases} \mathrm{i}^2 = \mathrm{j}^2 = \mathrm{k}^2 = -1 \\ \mathrm{ij} = \mathrm{k}, \mathrm{ji} = -\mathrm{k} \\ \mathrm{jk} = \mathrm{i}, \mathrm{kj} = -\mathrm{i} \\ \mathrm{ki} = \mathrm{j}, \mathrm{ik} = -\mathrm{j} \end{cases}. \tag{3.21}$$

如果把 i, j, k 看成三个坐标轴，那么它们与自己的乘法和复数一样，相互之间的乘法和外积一样。有时，人们也用一个标量和一个向量来表达四元数：

$$\boldsymbol{q} = [s, \boldsymbol{v}]^{\mathrm{T}}, \quad s = q_0 \in \mathbb{R}, \quad \boldsymbol{v} = [q_1, q_2, q_3]^{\mathrm{T}} \in \mathbb{R}^3.$$

这里，s 称为四元数的实部，而 $\boldsymbol{v}$ 称为它的虚部。如果一个四元数的虚部为 $\mathbf{0}$，则称为**实四元数**；反之，若它的实部为 0，则称为**虚四元数**。

可以用**单位四元数**表示三维空间中任意一个旋转，不过这种表达方式和复数有着微妙的不同。在复数中，乘以 i 意味着旋转 90°。这是否意味着四元数中，乘 i 就是绕 i 轴旋转 90°？那么，$\mathrm{ij}=\mathrm{k}$ 是否意味着，先绕 i 轴转 90°，再绕 j 轴转 90°，就等于绕 k 轴转 90°？读者可以找一部手机比划一下——然后你会发现情况并不是这样的。正确的情形应该是，乘以 i 对应着旋转 180°，这样才能保证 $\mathrm{ij}=\mathrm{k}$ 的性质。而 $\mathrm{i}^2=-1$，意味着绕 i 轴旋转 360° 后得到一个相反的东西。这个东西要旋转两周才会和它原先的样子相等。

这似乎有些玄妙了，完整的解释需要引入太多额外的东西，我们还是冷静一下回到眼前。至少，我们知道单位四元数能够表达三维空间的旋转。那么四元数本身有些什么性质，它们互相之间又可以做哪些运算呢？下面我们先考察四元数之间的运算法则。

3.4.2　四元数的运算

四元数和通常的复数一样，可以进行一系列的运算。常见的有四则运算、共轭、求逆、数乘等。下面分别介绍。

现有两个四元数 $\boldsymbol{q}_a, \boldsymbol{q}_b$，它们的向量表示为 $[s_a, \boldsymbol{v}_a]^{\mathrm{T}}, [s_b, \boldsymbol{v}_b]^{\mathrm{T}}$，或者原始四元数表示为

$$\boldsymbol{q}_a = s_a + x_a\mathrm{i} + \mathrm{y_a j} + \mathrm{z_a k}, \quad \mathrm{q_b} = \mathrm{s_b} + \mathrm{x_b i} + \mathrm{y_b j} + \mathrm{z_b k}.$$

那么，其运算可表示如下。

1. *加法和减法*

   四元数 $\boldsymbol{q}_a, \boldsymbol{q}_b$ 的加减运算为

$$\boldsymbol{q}_a \pm \boldsymbol{q}_b = [s_a \pm s_b, \boldsymbol{v}_a \pm \boldsymbol{v}_b]^{\mathrm{T}}. \tag{3.22}$$

2. *乘法*

   乘法是把 $\boldsymbol{q}_a$ 的每一项与 $\boldsymbol{q}_b$ 的每项相乘，最后相加，虚部要按照式 (3.21) 进行。整理可得

$$\begin{aligned} \boldsymbol{q}_a\boldsymbol{q}_b = & s_a s_b - x_a x_b - y_a y_b - z_a z_b \\ & + (s_a x_b + x_a s_b + y_a z_b - z_a y_b)\,\mathrm{i} \\ & + (s_a y_b - x_a z_b + y_a s_b + z_a x_b)\,\mathrm{j} \\ & + (s_a z_b + x_a y_b - y_a x_b + z_a s_b)\,\mathrm{k}. \end{aligned} \tag{3.23}$$

   虽然稍为复杂，但形式上是整齐有序的。如果写成向量形式并利用内外积运算，该表达会更加简洁：

$$\boldsymbol{q}_a\boldsymbol{q}_b = \left[s_a s_b - \boldsymbol{v}_a^{\mathrm{T}}\boldsymbol{v}_b, s_a\boldsymbol{v}_b + s_b\boldsymbol{v}_a + \boldsymbol{v}_a \times \boldsymbol{v}_b\right]^{\mathrm{T}}. \tag{3.24}$$

在该乘法定义下，两个实四元数乘积仍是实的，这与复数是一致的。然而，我们注意到，由于最后一项外积的存在，四元数乘法通常是不可交换的，除非 $\boldsymbol{v}_a$ 和 $\boldsymbol{v}_b$ 在 $\mathbb{R}^3$ 中共线，此时外积项为零。

3. 模长

四元数的模长定义为

$$\|\boldsymbol{q}_a\| = \sqrt{s_a^2 + x_a^2 + y_a^2 + z_a^2}. \tag{3.25}$$

可以验证，两个四元数乘积的模即模的乘积。这使得单位四元数相乘后仍是单位四元数。

$$\|\boldsymbol{q}_a \boldsymbol{q}_b\| = \|\boldsymbol{q}_a\| \|\boldsymbol{q}_b\|. \tag{3.26}$$

4. 共轭

四元数的共轭是把虚部取成相反数：

$$\boldsymbol{q}_a^* = s_a - x_a \mathrm{i} - \mathrm{y_a j} - \mathrm{z_a k} = [\mathrm{s_a}, -\mathrm{v_a}]^{\mathrm{T}}. \tag{3.27}$$

四元数共轭与其本身相乘，会得到一个实四元数，其实部为模长的平方：

$$\boldsymbol{q}^* \boldsymbol{q} = \boldsymbol{q} \boldsymbol{q}^* = [s_a^2 + \boldsymbol{v}^{\mathrm{T}} \boldsymbol{v}, \mathbf{0}]^{\mathrm{T}}. \tag{3.28}$$

5. 逆

一个四元数的逆为

$$\boldsymbol{q}^{-1} = \boldsymbol{q}^* / \|\boldsymbol{q}\|^2. \tag{3.29}$$

按此定义，四元数和自己的逆的乘积为实四元数 $\mathbf{1}$：

$$\boldsymbol{q} \boldsymbol{q}^{-1} = \boldsymbol{q}^{-1} \boldsymbol{q} = \mathbf{1}. \tag{3.30}$$

如果 $\boldsymbol{q}$ 为单位四元数，其逆和共轭就是同一个量。同时，乘积的逆具有和矩阵相似的性质：

$$(\boldsymbol{q}_a \boldsymbol{q}_b)^{-1} = \boldsymbol{q}_b^{-1} \boldsymbol{q}_a^{-1}. \tag{3.31}$$

6. 数乘

和向量相似，四元数可以与数相乘：

$$\mathrm{k}\boldsymbol{q} = [\mathrm{k}s, \mathrm{k}\boldsymbol{v}]^{\mathrm{T}}. \tag{3.32}$$

3.4.3　用四元数表示旋转

我们可以用四元数表达对一个点的旋转。假设有一个空间三维点 $\boldsymbol{p} = [x, y, z] \in \mathbb{R}^3$，以及一个由单位四元数 $\boldsymbol{q}$ 指定的旋转。三维点 $\boldsymbol{p}$ 经过旋转之后变为 $\boldsymbol{p}'$。如果使用矩阵描述，那么有 $\boldsymbol{p}' = \boldsymbol{R}\boldsymbol{p}$。而如果用四元数描述旋转，它们的关系又如何表达呢？

首先，把三维空间点用一个虚四元数来描述：

$$\boldsymbol{p} = [0, x, y, z]^{\mathrm{T}} = [0, \boldsymbol{v}]^{\mathrm{T}}.$$

相当于把四元数的 3 个虚部与空间中的 3 个轴相对应。那么，旋转后的点 $\boldsymbol{p}'$ 可表示为这样的乘积：

$$\boldsymbol{p}' = \boldsymbol{q}\boldsymbol{p}\boldsymbol{q}^{-1}. \tag{3.33}$$

这里的乘法均为四元数乘法，结果也是四元数。最后把 $\boldsymbol{p}'$ 的虚部取出，即得旋转之后点的坐标。并且，可以验证（留作习题），计算结果的实部为 0，故为纯虚四元数。

3.4.4　四元数到其他旋转表示的转换

任意单位四元数描述了一个旋转，该旋转也可用旋转矩阵或旋转向量描述。现在来考察四元数与旋转向量、旋转矩阵之间的转换关系。在此之前，我们要说，四元数乘法也可以写成一种矩阵的乘法。设 $\boldsymbol{q} = [s, \boldsymbol{v}]^{\mathrm{T}}$，那么，定义如下的符号 $^+$ 和 $^\oplus$ 为[20]

$$\boldsymbol{q}^+ = \begin{bmatrix} s & -\boldsymbol{v}^{\mathrm{T}} \\ \boldsymbol{v} & s\boldsymbol{I} + \boldsymbol{v}^\wedge \end{bmatrix}, \quad \boldsymbol{q}^\oplus = \begin{bmatrix} s & -\boldsymbol{v}^{\mathrm{T}} \\ \boldsymbol{v} & s\boldsymbol{I} - \boldsymbol{v}^\wedge \end{bmatrix}. \tag{3.34}$$

这两个符号将四元数映射成为一个 4×4 的矩阵。于是四元数乘法可以写成矩阵的形式：

$$\boldsymbol{q}_1^+ \boldsymbol{q}_2 = \begin{bmatrix} s_1 & -\boldsymbol{v}_1^{\mathrm{T}} \\ \boldsymbol{v}_1 & s_1\boldsymbol{I} + \boldsymbol{v}_1^\wedge \end{bmatrix} \begin{bmatrix} s_2 \\ \boldsymbol{v}_2 \end{bmatrix} = \begin{bmatrix} -\boldsymbol{v}_1^{\mathrm{T}}\boldsymbol{v}_2 + s_1 s_2 \\ s_1\boldsymbol{v}_2 + s_2\boldsymbol{v}_1 + \boldsymbol{v}_1^\wedge \boldsymbol{v}_2 \end{bmatrix} = \boldsymbol{q}_1 \boldsymbol{q}_2. \tag{3.35}$$

同理亦可证：

$$\boldsymbol{q}_1 \boldsymbol{q}_2 = \boldsymbol{q}_1^+ \boldsymbol{q}_2 = \boldsymbol{q}_2^\oplus \boldsymbol{q}_1. \tag{3.36}$$

然后，考虑使用四元数对空间点进行旋转的问题。根据前面的说法，有

$$\begin{aligned} \boldsymbol{p}' &= \boldsymbol{q}\boldsymbol{p}\boldsymbol{q}^{-1} = \boldsymbol{q}^+ \boldsymbol{p}^+ \boldsymbol{q}^{-1} \\ &= \boldsymbol{q}^+ \boldsymbol{q}^{-1^\oplus} \boldsymbol{p}. \end{aligned} \tag{3.37}$$

代入两个符号对应的矩阵，得

$$\boldsymbol{q}^{+}\left(\boldsymbol{q}^{-1}\right)^{\oplus}=\begin{bmatrix}s & -\boldsymbol{v}^{\mathrm{T}}\\ \boldsymbol{v} & s\boldsymbol{I}+\boldsymbol{v}^{\wedge}\end{bmatrix}\begin{bmatrix}s & \boldsymbol{v}^{\mathrm{T}}\\ -\boldsymbol{v} & s\boldsymbol{I}+\boldsymbol{v}^{\wedge}\end{bmatrix}=\begin{bmatrix}1 & \boldsymbol{0}\\ \boldsymbol{0}^{\mathrm{T}} & \boldsymbol{v}\boldsymbol{v}^{\mathrm{T}}+s^{2}\boldsymbol{I}+2s\boldsymbol{v}^{\wedge}+\left(\boldsymbol{v}^{\wedge}\right)^{2}\end{bmatrix}. \tag{3.38}$$

因为 $\boldsymbol{p}'$ 和 $\boldsymbol{p}$ 都是虚四元数，所以事实上该矩阵的右下角即给出了**从四元数到旋转矩阵**的变换关系：

$$\boldsymbol{R}=\boldsymbol{v}\boldsymbol{v}^{\mathrm{T}}+s^{2}\boldsymbol{I}+2s\boldsymbol{v}^{\wedge}+\left(\boldsymbol{v}^{\wedge}\right)^{2}. \tag{3.39}$$

为了得到四元数到旋转向量的转换公式，对上式两侧求迹，得

$$\begin{aligned}\mathrm{tr}(\boldsymbol{R})&=\mathrm{tr}(\boldsymbol{v}\boldsymbol{v}^{\mathrm{T}}+3s^{2}+2s\cdot 0+\mathrm{tr}((\boldsymbol{v}^{\wedge})^{2})\\&=v_{1}^{2}+v_{2}^{2}+v_{3}^{2}+3s^{2}-2(v_{1}^{2}+v_{2}^{2}+v_{3}^{2})\\&=(1-s^{2})+3s^{2}-2(1-s^{2})\\&=4s^{2}-1.\end{aligned} \tag{3.40}$$

又由式 (3.17) 得

$$\begin{aligned}\theta&=\arccos\frac{\mathrm{tr}(\boldsymbol{R}-1)}{2}\\&=\arccos(2s^{2}-1).\end{aligned} \tag{3.41}$$

即

$$\cos\theta=2s^{2}-1=2\cos^{2}\frac{\theta}{2}-1, \tag{3.42}$$

所以：

$$\theta=2\arccos s. \tag{3.43}$$

至于旋转轴，如果在式 (3.38) 中用 $\boldsymbol{q}$ 的虚部代替 $\boldsymbol{p}$，易知 $\boldsymbol{q}$ 的虚部组成的向量在旋转时是不动的，即构成旋转轴。于是只要将它除掉它的模长，即得。总而言之，四元数到旋转向量的转换公式如下：

$$\begin{cases}\theta=2\arccos q_{0}\\ [n_{x},n_{y},n_{z}]^{\mathrm{T}}=[q_{1},q_{2},q_{3}]^{\mathrm{T}}/\sin\dfrac{\theta}{2}\end{cases}. \tag{3.44}$$

至于如何从其他方式转换到四元数，只须把上述步骤倒过来处理即可。在实际编程中，程序库通常会为我们准备好各种形式之间的转换。无论是四元数、旋转矩阵还是轴角，它们都可以用来描述同一个旋转。我们应该在实际中选择最方便的形式，而不必拘泥于某种特定的形式。在随后的实践和习题中，我们会演示各种表达方式之间的转换，以加深读者的印象。

3.5　* 相似、仿射、射影变换

除了欧氏变换，3D 空间还存在其他几种变换方式，只不过欧氏变换是最简单的。它们一部分和测量几何有关，因为在之后的讲解中可能会提到，所以先罗列出来。欧氏变换保持了向量的长度和夹角，相当于我们把一个刚体原封不动地进行了移动或旋转，不改变它自身的样子。其他几种变换则会改变它的外形。它们都拥有类似的矩阵表示。

1. 相似变换

   相似变换比欧氏变换多了一个自由度，它允许物体进行均匀缩放，其矩阵表示为

$$\boldsymbol{T}_S = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}. \tag{3.45}$$

   注意，旋转部分多了一个缩放因子 s，表示我们在对向量旋转之后，可以在 x, y, z 三个坐标上进行均匀缩放。由于含有缩放，相似变换不再保持图形的面积不变。你可以想象一个边长为 1 的立方体通过相似变换后，变成边长为 10 的样子（但仍然是立方体）。三维相似变换的集合也叫作**相似变换群**，记作 Sim(3)。

2. 仿射变换

   仿射变换的矩阵形式如下：

$$\boldsymbol{T}_A = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}. \tag{3.46}$$

   与欧氏变换不同的是，仿射变换只要求 $\boldsymbol{A}$ 是一个可逆矩阵，而不必是正交矩阵。仿射变换也叫正交投影。经过仿射变换之后，立方体就不再是方的了，但是各个面仍然是平行四边形。

3. 射影变换

   射影变换是最一般的变换，它的矩阵形式为

$$\boldsymbol{T}_P = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{a}^{\mathrm{T}} & v \end{bmatrix}. \tag{3.47}$$

   它的左上角为可逆矩阵 $\boldsymbol{A}$，右上角为平移 $\boldsymbol{t}$，左下角为缩放 $\boldsymbol{a}^{\mathrm{T}}$。由于采用了齐次坐标，当 $v \neq 0$ 时，我们可以对整个矩阵除以 v 得到一个右下角为 1 的矩阵；否则得到右下角为 0 的矩阵。因此，2D 的射影变换一共有 8 个自由度，3D 则共有 15 个自由度。射影变换是现在讲过的变换中，形式最为一般的。从真实世界到相机照片的变换可以看成一个射影变换。读者可以想象一个原本方形的地板砖，在照片中是什么样子：首先，它不再是方

形的。由于近大远小的关系，它甚至不是平行四边形，而是一个不规则的四边形。

表 3-1 总结了目前讲到的几种变换的性质。注意在“不变性质”中，从上到下是有包含关系的。例如，欧氏变换除了保体积，也具有保平行、相交等性质。

表 3-1 常见变换的性质比较

变换名称	矩阵形式	自由度	不变性质
欧氏变换	$\begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}$	6	长度、夹角、体积
相似变换	$\begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}$	7	体积比
仿射变换	$\begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}$	12	平行性、体积比
射影变换	$\begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{a}^{\mathrm{T}} & v \end{bmatrix}$	15	接触平面的相交和相切

我们之后会讲到，从真实世界到相机照片的变换是一个射影变换。如果相机的焦距为无穷远，那么这个变换为仿射变换。不过，在详细讲述相机模型之前，我们只要对它们有个大致的印象即可。

3.6 实践：Eigen 几何模块

3.6.1 Eigen 几何模块的数据演示

现在，我们来实际演练前面讲到的各种旋转表达方式。我们将在 Eigen 中使用四元数、欧拉角和旋转矩阵，演示它们之间的变换方式。我们还会给出一个可视化程序，帮助读者理解这几个变换的关系。

slambook2/ch3/useGeometry/useGeometry.cpp

```
#include <iostream>
#include <cmath>
using namespace std;

#include <Eigen/Core>
#include <Eigen/Geometry>

using namespace Eigen;
// 本程序演示了Eigen几何模块的使用方法
```

```
int main(int argc, char **argv) {
    // Eigen/Geometry模块提供了各种旋转和平移的表示
    // 3D旋转矩阵直接使用Matrix3d或Matrix3f
    Matrix3d rotation_matrix = Matrix3d::Identity();
    // 旋转向量使用AngleAxis, 它底层不直接是Matrix, 但运算可以当作矩阵(因为重载了运算符)
    AngleAxisd rotation_vector(M_PI / 4, Vector3d(0, 0, 1));     //沿Z轴旋转45°
    cout.precision(3);
    cout << "rotation matrix =\n" << rotation_vector.matrix() << endl;   //用matrix()转换成矩
    // 阵也可以直接赋值
    rotation_matrix = rotation_vector.toRotationMatrix();
    // 用AngleAxis可以进行坐标变换
    Vector3d v(1, 0, 0);
    Vector3d v_rotated = rotation_vector * v;
    cout << "(1,0,0) after rotation (by angle axis) = " << v_rotated.transpose() << endl;
    // 或者用旋转矩阵
    v_rotated = rotation_matrix * v;
    cout << "(1,0,0) after rotation (by matrix) = " << v_rotated.transpose() << endl;

    // 欧拉角: 可以将旋转矩阵直接转换成欧拉角
    Vector3d euler_angles = rotation_matrix.eulerAngles(2, 1, 0); // ZYX顺序, 即roll pitch
    yaw顺序
    cout << "yaw pitch roll = " << euler_angles.transpose() << endl;

    // 欧氏变换矩阵使用Eigen::Isometry
    Isometry3d T = Isometry3d::Identity();        // 虽然称为3D, 实质上是4*4的矩阵
    T.rotate(rotation_vector);                    // 按照rotation_vector进行旋转
    T.pretranslate(Vector3d(1, 3, 4));            // 把平移向量设成(1,3,4)
    cout << "Transform matrix = \n" << T.matrix() << endl;

    // 用变换矩阵进行坐标变换
    Vector3d v_transformed = T * v;                              // 相当于R*v+t
    cout << "v tranformed = " << v_transformed.transpose() << endl;

    // 对于仿射变换和射影变换, 使用Eigen::Affine3d和Eigen::Projective3d即可, 略

    // 四元数
    // 可以直接把AngleAxis赋值给四元数, 反之亦然
    Quaterniond q = Quaterniond(rotation_vector);
    cout << "quaternion from rotation vector = " << q.coeffs().transpose()
    << endl;   // 请注意coeffs的顺序是(x,y,z,w), w为实部, 前三者为虚部
    // 也可以把旋转矩阵赋给它
    q = Quaterniond(rotation_matrix);
```

```
    cout << "quaternion from rotation matrix = " << q.coeffs().transpose() << endl;
    // 使用四元数旋转一个向量，使用重载的乘法即可
    v_rotated = q * v; // 注意数学上是qvq^{-1}
    cout << "(1,0,0) after rotation = " << v_rotated.transpose() << endl;
    // 用常规向量乘法表示，则计算如下
    cout << "should be equal to " << (q * Quaterniond(0, 1, 0, 0) * q.inverse()).coeffs().
    transpose() << endl;

    return 0;
}
```

Eigen 中对各种形式的表达方式总结如下。请注意每种类型都有单精度和双精度两种数据类型，而且和之前一样，不能由编译器自动转换。下面以双精度为例，你可以把最后的 d 改成 f，即得到单精度的数据结构。

- 旋转矩阵（3×3）：Eigen::Matrix3d。
- 旋转向量（3×1）：Eigen::AngleAxisd。
- 欧拉角（3×1）：Eigen::Vector3d。
- 四元数（4×1）：Eigen::Quaterniond。
- 欧氏变换矩阵（4×4）：Eigen::Isometry3d。
- 仿射变换（4×4）：Eigen::Affine3d。
- 射影变换（4×4）：Eigen::Projective3d。

参考代码中对应的 CMakeLists 即可编译此程序。在这个程序中，演示了如何使用 Eigen 中的旋转矩阵、旋转向量、欧拉角和四元数。我们用这几种旋转方式旋转一个向量 $\boldsymbol{v}$，发现结果是一样的（不一样就真是见鬼了）。同时，也演示了如何在程序中转换这几种表达方式。想进一步了解 Eigen 的几何模块的读者可以参考（`http://eigen.tuxfamily.org/dox/group__TutorialGeometry.html`）。

请读者注意，**程序代码通常和数学表示有一些细微的差别**。例如，通过运算符重载，四元数和三维向量可以直接计算乘法，但在数学上则需要先把向量转成虚四元数，再利用四元数乘法进行计算，同样的情况也适用于变换矩阵乘三维向量的情况。总体而言，程序中的用法会比数学公式更灵活。

3.6.2 实际的坐标变换例子

下面我们举一个小例子来演示坐标变换。

例子 设有小萝卜一号和小萝卜二号位于世界坐标系中。记世界坐标系为 W，小萝卜们的坐标系为 R_1 和 R_2。小萝卜一号的位姿为 $\boldsymbol{q}_1 = [0.35, 0.2, 0.3, 0.1]^{\mathrm{T}}, \boldsymbol{t}_1 = [0.3, 0.1, 0.1]^{\mathrm{T}}$。小萝卜二号的位姿为 $\boldsymbol{q}_2 = [-0.5, 0.4, -0.1, 0.2]^{\mathrm{T}}, \boldsymbol{t}_2 = [-0.1, 0.5, 0.3]^{\mathrm{T}}$。这里的 $\boldsymbol{q}$ 和 $\boldsymbol{t}$ 表达的是 $\boldsymbol{T}_{R_k,W}, k = 1, 2,$

也就是世界坐标系到相机坐标系的变换关系。现在，小萝卜一号看到某个点在自身的坐标系下坐标为 $\boldsymbol{p}_{R_1} = [0.5, 0, 0.2]^{\mathrm{T}}$，求该向量在小萝卜二号坐标系下的坐标。

这是一个非常简单，但又具有代表性的例子。在实际场景中你经常需要在同一个机器人的不同部分，或者不同机器人之间转换坐标。下面我们写一段程序来演示这个计算。

slambook2/ch3/examples/coordinateTransform.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <Eigen/Core>
#include <Eigen/Geometry>

using namespace std;
using namespace Eigen;

int main(int argc, char** argv) {
    Quaterniond q1(0.35, 0.2, 0.3, 0.1), q2(-0.5, 0.4, -0.1, 0.2);
    q1.normalize();
    q2.normalize();
    Vector3d t1(0.3, 0.1, 0.1), t2(-0.1, 0.5, 0.3);
    Vector3d p1(0.5, 0, 0.2);

    Isometry3d T1w(q1), T2w(q2);
    T1w.pretranslate(t1);
    T2w.pretranslate(t2);

    Vector3d p2 = T2w * T1w.inverse() * p1;
    cout << endl << p2.transpose() << endl;
    return 0;
}
```

程序输出的答案是 $[-0.0309731, 0.73499, 0.296108]^{\mathrm{T}}$，计算过程也十分简单，只需计算

$$\boldsymbol{p}_{R_2} = \boldsymbol{T}_{R_2,W}\boldsymbol{T}_{W,R_1}\boldsymbol{p}_{R_1}$$

即可。注意四元数使用之前需要归一化。

3.7 可视化演示

3.7.1 显示运动轨迹

如果你是第一次接触旋转和平移这些概念，可能会觉得它们的形式看起来很复杂，因为毕竟每种表达方式都可以与其他方式互相转换，而转换公式有时还比较长。虽然旋转矩阵、变换矩阵的数值可能不够直观，但我们可以很容易地把它们画在窗口里。

本节我们演示两个可视化的例子。首先，假设我们通过某种方式记录了一个机器人的运动轨迹，现在想把它画到一个窗口中。假设轨迹文件存储于 trajectory.txt，每一行用下面的格式存储：

$$\text{time}, t_x, t_y, t_z, q_x, q_y, q_z, q_w,$$

其中，time 指该位姿的记录时间，$\boldsymbol{t}$ 为平移，$\boldsymbol{q}$ 为旋转四元数，均是以世界坐标系到机器人坐标系记录。下面我们从文件中读取这些轨迹，并显示到一个窗口中。原则上，如果只是谈论“机器人的位姿”，那么你可以使用 $\boldsymbol{T}_{\mathrm{WR}}$ 或者 $\boldsymbol{T}_{\mathrm{RW}}$，事实上它们也只差一个逆而已，这意味着知道其中一个就可以很轻松地得到另一个。如果你想要存储**机器人的轨迹**，那么可以存储所有时刻的 $\boldsymbol{T}_{\mathrm{WR}}$ 或者 $\boldsymbol{T}_{\mathrm{RW}}$，这并没有太大的差别。

在画轨迹的时候，我们可以把“轨迹”画成一系列点组成的序列，这和我们想象中的“轨迹”比较相似。严格说来，这其实是**机器人（相机）坐标系的原点在世界坐标系中的坐标**。考虑机器人坐标系的原点 $\boldsymbol{O}_{\mathrm{R}}$，此时的 $\boldsymbol{O}_{\mathrm{W}}$ 就是这个原点在世界坐标系下的坐标：

$$\boldsymbol{O}_{\mathrm{W}} = \boldsymbol{T}_{\mathrm{WR}} \boldsymbol{O}_{\mathrm{R}} = \boldsymbol{t}_{\mathrm{WR}}. \tag{3.48}$$

这正是 $\boldsymbol{T}_{\mathrm{WR}}$ 的平移部分。因此，可以从 $\boldsymbol{T}_{\mathrm{WR}}$ 中直接看到**相机在何处**，这也是我们说 $\boldsymbol{T}_{\mathrm{WR}}$ 更为直观的原因。因此，在可视化程序里，轨迹文件存储了 $\boldsymbol{T}_{\mathrm{WR}}$ 而不是 $\boldsymbol{T}_{\mathrm{RW}}$。

最后，我们需要一个支持 3D 绘图的程序库。有许多库都支持 3D 绘图，比如大家熟悉的 MATLAB，Python 的 Matplotlib、OpenGL 等。在 Linux 中，一个常见的库是基于 OpenGL 的 Pangolin 库[①]，它在支持 OpenGL 的绘图操作基础之上还提供了一些 GUI 的功能。本书中，我们使用 Git 的 submodule 功能管理本书依赖的第三方库。读者可以进入 3rdparty 文件夹直接安装所需的库，Git 保证了笔者和你使用的版本是一致的。

slambook2/ch3/examples/plotTrajectory.cpp

```
#include <pangolin/pangolin.h>
#include <Eigen/Core>
#include <unistd.h>
```

①https://github.com/stevenlovegrove/Pangolin

```
4
5  using namespace std;
6  using namespace Eigen;
7
8  // path to trajectory file
9  string trajectory_file = "./examples/trajectory.txt";
10
11 void DrawTrajectory(vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>>);
12
13 int main(int argc, char **argv) {
14     vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>> poses;
15     ifstream fin(trajectory_file);
16     if (!fin) {
17         cout << "cannot find trajectory file at " << trajectory_file << endl;
18         return 1;
19     }
20
21     while (!fin.eof()) {
22         double time, tx, ty, tz, qx, qy, qz, qw;
23         fin >> time >> tx >> ty >> tz >> qx >> qy >> qz >> qw;
24         Isometry3d Twr(Quaterniond(qw, qx, qy, qz));
25         Twr.pretranslate(Vector3d(tx, ty, tz));
26         poses.push_back(Twr);
27     }
28     cout << "read total " << poses.size() << " pose entries" << endl;
29
30     // draw trajectory in pangolin
31     DrawTrajectory(poses);
32     return 0;
33 }
34
35 void DrawTrajectory(vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>> poses) {
36     // create pangolin window and plot the trajectory
37     pangolin::CreateWindowAndBind("Trajectory Viewer", 1024, 768);
38     glEnable(GL_DEPTH_TEST);
39     glEnable(GL_BLEND);
40     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
41
42     pangolin::OpenGlRenderState s_cam(
43     pangolin::ProjectionMatrix(1024, 768, 500, 500, 512, 389, 0.1, 1000),
44     pangolin::ModelViewLookAt(0, -0.1, -1.8, 0, 0, 0, 0.0, -1.0, 0.0)
45     );
46
```

```
    pangolin::View &d_cam = pangolin::CreateDisplay()
        .SetBounds(0.0, 1.0, 0.0, 1.0, -1024.0f / 768.0f)
        .SetHandler(new pangolin::Handler3D(s_cam));

    while (pangolin::ShouldQuit() == false) {
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        d_cam.Activate(s_cam);
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
        glLineWidth(2);
        for (size_t i = 0; i < poses.size(); i++) {
            // 画每个位姿的三个坐标轴
            Vector3d Ow = poses[i].translation();
            Vector3d Xw = poses[i] * (0.1 * Vector3d(1, 0, 0));
            Vector3d Yw = poses[i] * (0.1 * Vector3d(0, 1, 0));
            Vector3d Zw = poses[i] * (0.1 * Vector3d(0, 0, 1));
            glBegin(GL_LINES);
            glColor3f(1.0, 0.0, 0.0);
            glVertex3d(Ow[0], Ow[1], Ow[2]);
            glVertex3d(Xw[0], Xw[1], Xw[2]);
            glColor3f(0.0, 1.0, 0.0);
            glVertex3d(Ow[0], Ow[1], Ow[2]);
            glVertex3d(Yw[0], Yw[1], Yw[2]);
            glColor3f(0.0, 0.0, 1.0);
            glVertex3d(Ow[0], Ow[1], Ow[2]);
            glVertex3d(Zw[0], Zw[1], Zw[2]);
            glEnd();
        }
        // 画出连线
        for (size_t i = 0; i < poses.size(); i++) {
            glColor3f(0.0, 0.0, 0.0);
            glBegin(GL_LINES);
            auto p1 = poses[i], p2 = poses[i + 1];
            glVertex3d(p1.translation()[0], p1.translation()[1], p1.translation()[2]);
            glVertex3d(p2.translation()[0], p2.translation()[1], p2.translation()[2]);
            glEnd();
        }
        pangolin::FinishFrame();
        usleep(5000);   // sleep 5 ms
    }
}
```

该程序演示了如何在 Panglin 中画出 3D 的位姿。我们用红、绿、蓝三种颜色画出每个位姿的三个坐标轴（实际上我们计算了各坐标轴的世界坐标），然后用黑色线将轨迹连起来。位姿可视

化的结果如图 3-3 所示。

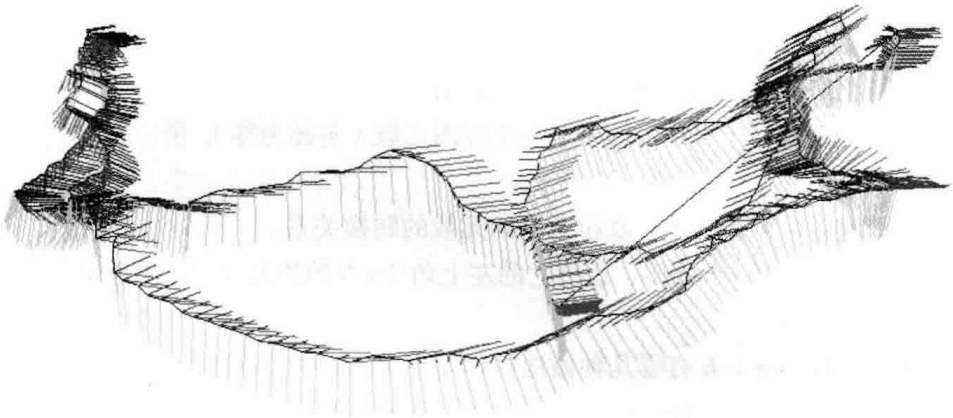


图 3-3　位姿可视化的结果（见彩插）

3.7.2　显示相机的位姿

除了显示轨迹，我们也可以显示 3D 窗口中相机的位姿。在 slambook2/ch3/visualizeGeometry 中，我们以可视化的形式演示相机位姿的各种表达方式（如图 3-4 所示）。当读者用鼠标操作相机时，左侧的方框里会实时显示相机位姿对应的旋转矩阵、平移、欧拉角和四元数，你可以看到数据是如何变化的。根据我们的经验，除了欧拉角，你应该看不出它们直观的含义。尽管旋转矩阵或变换矩阵并不直观，但是将它们可视化地显示出来并不困难。该程序使用 Pangolin 库作为 3D 显示库，请参考 Readme.txt 来编译该程序。

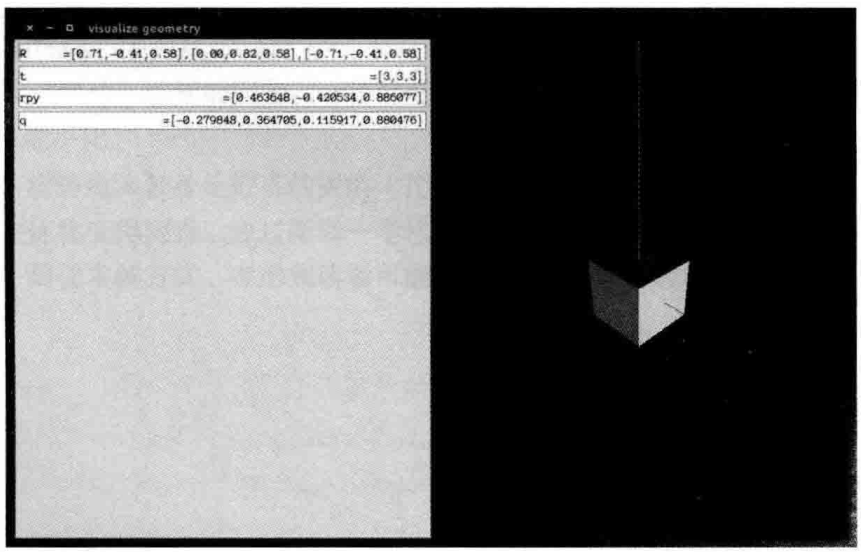


图 3-4　旋转矩阵、平移、欧拉角、四元数的可视化程序

习题

1. 验证旋转矩阵是正交矩阵。
2.* 寻找罗德里格斯公式的推导过程并加以理解。
3. 验证四元数旋转某个点后，结果是一个虚四元数（实部为零），所以仍然对应到一个三维空间点，见式 (3.33)。
4. 画表总结旋转矩阵、轴角、欧拉角、四元数的转换关系。
5. 假设有一个大的 Eigen 矩阵，想把它的左上角 3×3 的块取出来，然后赋值为 $\boldsymbol{I}_{3\times 3}$。请编程实现。
6.* 一般线性方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 有哪几种做法？你能在 Eigen 中实现吗？

第 4 讲

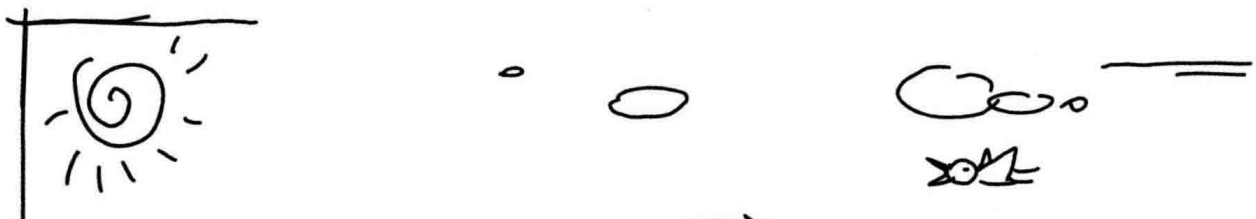
李群与李代数

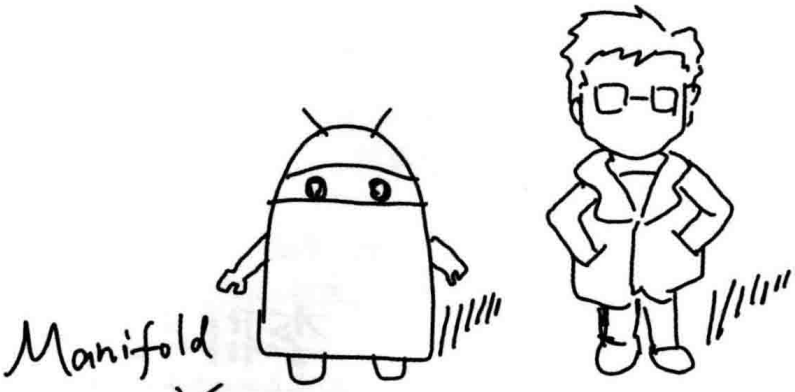
主要目标

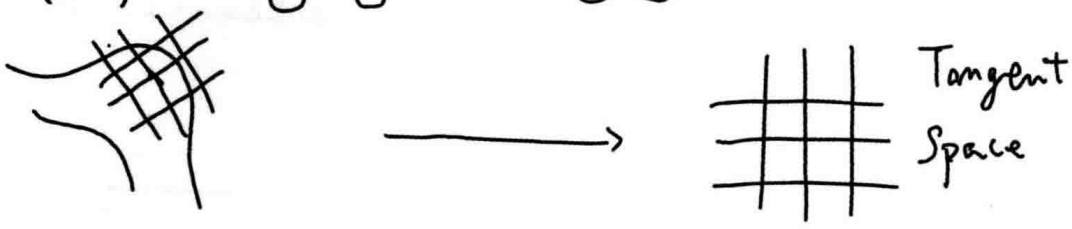
1. 理解李群与李代数的概念，掌握 SO(3)、SE(3) 与对应李代数的表示方式。
2. 理解 BCH 近似的意义。
3. 学会在李代数上的扰动模型。
4. 使用 Sophus 对李代数进行运算。

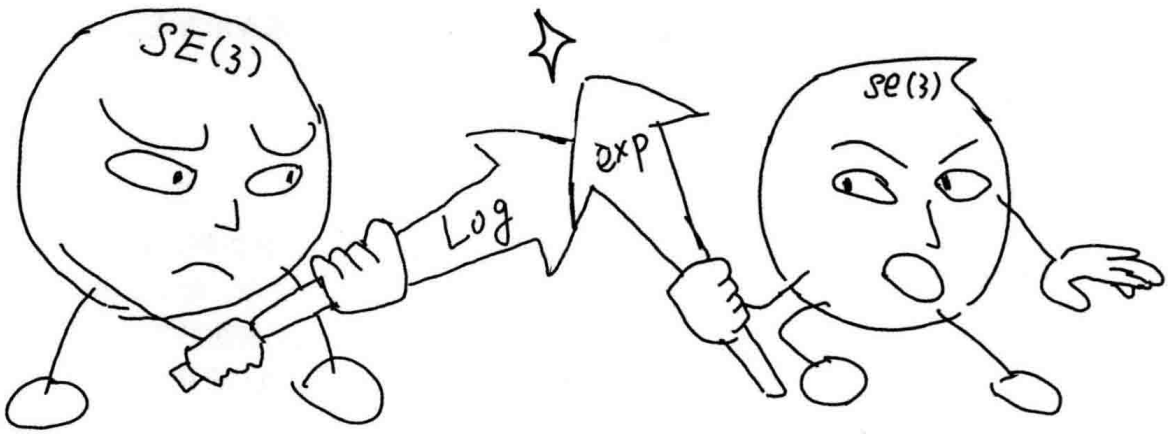
第 3 讲，我们介绍了三维世界中刚体运动的描述方式，包括旋转矩阵、旋转向量、欧拉角、四元数等若干种方式。我们重点介绍了旋转的表示，但是在 SLAM 中，除了表示，我们还要对它们进行估计和优化。因为在 SLAM 中位姿是未知的，而我们需要解决形如“**什么样的相机位姿最符合当前观测数据**”这样的问题。一种典型的方式是把它构建成一个优化问题，求解最优的 $\boldsymbol{R}, \boldsymbol{t}$，使得误差最小化。

如前所言，旋转矩阵自身是带有约束的（正交且行列式为 1）。它们作为优化变量时，会引入额外的约束，使优化变得困难。通过李群—李代数间的转换关系，我们希望把位姿估计变成无约束的优化问题，简化求解方式。考虑到读者可能还没有李群与李代数的基本知识，我们将从最基本的知识讲起。









$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \in SE(3) \qquad \xi = [\rho, \phi]^T \in se(3)$$

$$\log(T)^\vee = \xi$$

$$\exp(\xi^\wedge) = T$$

4×4 $\qquad$ 6×1

4.1　李群与李代数基础

第 3 讲中，我们介绍了旋转矩阵和变换矩阵的定义。当时，我们说三维旋转矩阵构成了**特殊正交群** SO(3)，而变换矩阵构成了**特殊欧氏群** SE(3)。它们写起来像这样：

$$\mathrm{SO}(3) = \{\boldsymbol{R} \in \mathbb{R}^{3\times 3} | \boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}, \det(\boldsymbol{R}) = 1\}, \tag{4.1}$$

$$\mathrm{SE}(3) = \left\{ \boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} | \boldsymbol{R} \in \mathrm{SO}(3), \boldsymbol{t} \in \mathbb{R}^3 \right\}. \tag{4.2}$$

不过，当时我们并未详细解释**群**的含义。细心的读者应该会注意到，旋转矩阵也好，变换矩阵也好，**它们对加法是不封闭的**。换句话说，对于任意两个旋转矩阵 $\boldsymbol{R}_1, \boldsymbol{R}_2$，按照矩阵加法的定义，和不再是一个旋转矩阵：

$$\boldsymbol{R}_1 + \boldsymbol{R}_2 \notin \mathrm{SO}(3), \quad \boldsymbol{T}_1 + \boldsymbol{T}_2 \notin \mathrm{SE}(3). \tag{4.3}$$

你也可以说两种矩阵并没有良好定义的加法，或者通常矩阵加法对这两个集合不封闭。相对地，它们只有一种较好的运算：乘法。SO(3) 和 SE(3) 关于乘法是封闭的：

$$\boldsymbol{R}_1\boldsymbol{R}_2 \in \mathrm{SO}(3), \quad \boldsymbol{T}_1\boldsymbol{T}_2 \in \mathrm{SE}(3). \tag{4.4}$$

同时，我们也可以对任何一个旋转或变换矩阵（在乘法的意义上）求逆。我们知道，乘法对应着旋转或变换的复合，两个旋转矩阵相乘表示做了两次旋转。对于这种只有一个（良好的）运算的集合，我们称之为**群**。

4.1.1　群

接下来，我们要稍微涉及一些抽象代数方面的知识。笔者觉得这是讨论李群与李代数的必要条件，但实际上除了数学、物理系的同学，大部分同学在本科学习中并不会接触这方面的知识。所以我们先来看一些基本的知识。

群（Group）是**一种集合**加上**一种运算**的代数结构。我们把集合记作 A，运算记作 $\cdot$，那么群可以记作 $G = (A, \cdot)$。群要求这个运算满足以下几个条件：

1. 封闭性：　$\forall a_1, a_2 \in A, \quad a_1 \cdot a_2 \in A.$
2. 结合律：　$\forall a_1, a_2, a_3 \in A, \quad (a_1 \cdot a_2) \cdot a_3 = a_1 \cdot (a_2 \cdot a_3).$
3. 幺元：　$\exists a_0 \in A, \quad \text{s.t.} \quad \forall a \in A, \quad a_0 \cdot a = a \cdot a_0 = a.$
4. 逆：　$\forall a \in A, \quad \exists a^{-1} \in A, \quad \text{s.t.} \quad a \cdot a^{-1} = a_0.$

读者可以将其记作“封结幺逆”[①]。容易验证，旋转矩阵集合和矩阵乘法构成群，同样，变换矩阵和矩阵乘法也构成群（因此才能称它们为旋转矩阵群和变换矩阵群）。其他常见的群包括整数的加法 $(\mathbb{Z}, +)$，去掉 0 后的有理数的乘法（幺元为 1）$(\mathbb{Q}\backslash 0, \cdot)$，等等。矩阵中常见的群有：

- 一般线性群 $\mathrm{GL}(n)$　指 $n \times n$ 的可逆矩阵，它们对矩阵乘法成群。
- 特殊正交群 $\mathrm{SO}(n)$　也就是所谓的旋转矩阵群，其中 $\mathrm{SO}(2)$ 和 $\mathrm{SO}(3)$ 最为常见。
- 特殊欧氏群 $\mathrm{SE}(n)$　也就是前面提到的 n 维欧氏变换，如 $\mathrm{SE}(2)$ 和 $\mathrm{SE}(3)$。

群结构保证了在群上的运算具有良好的性质，群论则是研究群的各种结构和性质的理论。对群论感兴趣的读者可以参考任意一本近世代数教材。**李群**是指具有连续（光滑）性质的群。像整数群 $\mathbb{Z}$ 那样离散的群没有连续性质，所以不是李群。而 $\mathrm{SO}(n)$ 和 $\mathrm{SE}(n)$ 在实数空间上是连续的。我们能够直观地想象一个刚体能够连续地在空间中运动，所以它们都是李群。由于 $\mathrm{SO}(3)$ 和 $\mathrm{SE}(3)$ 对于相机姿态估计尤其重要，所以我们主要讨论这两个李群。然而，严格地讨论“连续”“光滑”这些概念需要具备分析和拓扑学的知识，但我们不是数学书，所以只介绍一些重要的、与 SLAM 直接相关的结论。如果读者对李群的理论性质感兴趣，请参考文献 [21]。

通常有两种思路来介绍李群与李代数，一种是直接引入李群和李代数，然后告诉读者每个李群对应着一个李代数之类的事实，但这样的话，读者会觉得李代数似乎是一个从天而降的符号，不知道它有什么物理意义。所以，笔者准备稍微花一点时间从旋转矩阵引出李代数，类似于参考文献 [22] 的做法。我们先从较简单的 $\mathrm{SO}(3)$ 开始讨论，引出 $\mathrm{SO}(3)$ 上面的李代数 $\mathfrak{so}(3)$。

4.1.2 李代数的引出

考虑任意旋转矩阵 $\boldsymbol{R}$，我们知道它满足：

$$\boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}. \tag{4.5}$$

现在，我们说，$\boldsymbol{R}$ 是某个相机的旋转，它会随时间连续地变化，即为时间的函数：$\boldsymbol{R}(t)$。由于它仍是旋转矩阵，有

$$\boldsymbol{R}(t)\boldsymbol{R}(t)^{\mathrm{T}} = \boldsymbol{I}.$$

在等式两边对时间求导，得到

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}} + \boldsymbol{R}(t)\dot{\boldsymbol{R}}(t)^{\mathrm{T}} = 0.$$

整理得

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}} = -\left(\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}}\right)^{\mathrm{T}}. \tag{4.6}$$

①谐音“丰俭由你”。

可以看出，$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}}$ 是一个**反对称**矩阵。回忆一下，我们在式 (3.3) 介绍叉积时，引入了 $\wedge$ 符号，将一个向量变成了反对称矩阵。同理，对于任意反对称矩阵，我们也能找到唯一与之对应的向量。把这个运算用符号 $\vee$ 表示：

$$\boldsymbol{a}^{\wedge} = \boldsymbol{A} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \quad \boldsymbol{A}^{\vee} = \boldsymbol{a}. \tag{4.7}$$

于是，由于 $\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}}$ 是一个反对称矩阵，我们可以找到一个三维向量 $\boldsymbol{\phi}(t) \in \mathbb{R}^3$ 与之对应：

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}} = \boldsymbol{\phi}(t)^{\wedge}.$$

等式两边右乘 $\boldsymbol{R}(t)$，由于 $\boldsymbol{R}$ 为正交阵，有

$$\dot{\boldsymbol{R}}(t) = \boldsymbol{\phi}(t)^{\wedge}\boldsymbol{R}(t) = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix} \boldsymbol{R}(t). \tag{4.8}$$

可以看到，每对旋转矩阵求一次导数，只需左乘一个 $\boldsymbol{\phi}^{\wedge}(t)$ 矩阵即可。考虑 $t_0 = 0$ 时，设此时旋转矩阵为 $\boldsymbol{R}(0) = \boldsymbol{I}$。按照导数定义，可以把 $\boldsymbol{R}(t)$ 在 $t = 0$ 附近进行一阶泰勒展开：

$$\begin{aligned} \boldsymbol{R}(t) &\approx \boldsymbol{R}(t_0) + \dot{\boldsymbol{R}}(t_0)(t - t_0) \\ &= \boldsymbol{I} + \boldsymbol{\phi}(t_0)^{\wedge}(t). \end{aligned} \tag{4.9}$$

我们看到 $\boldsymbol{\phi}$ 反映了 $\boldsymbol{R}$ 的导数性质，故称它在 SO(3) 原点附近的正切空间（Tangent Space）上。同时在 t_0 附近，设 $\boldsymbol{\phi}$ 保持为常数 $\boldsymbol{\phi}(t_0) = \boldsymbol{\phi}_0$。那么根据式 (4.8)，有

$$\dot{\boldsymbol{R}}(t) = \boldsymbol{\phi}(t_0)^{\wedge}\boldsymbol{R}(t) = \boldsymbol{\phi}_0^{\wedge}\boldsymbol{R}(t).$$

上式是一个关于 $\boldsymbol{R}$ 的微分方程，而且有初始值 $\boldsymbol{R}(0) = \boldsymbol{I}$，解得

$$\boldsymbol{R}(t) = \exp(\boldsymbol{\phi}_0^{\wedge} t). \tag{4.10}$$

读者可以验证上式对微分方程和初始值均成立。这说明在 $t = 0$ 附近，旋转矩阵可以由

$\exp(\boldsymbol{\phi}_0^\wedge t)$ 计算出来[①]。我们看到，旋转矩阵 $\boldsymbol{R}$ 与另一个反对称矩阵 $\boldsymbol{\phi}_0^\wedge t$ 通过指数关系发生了联系。但是矩阵的指数是什么呢？这里我们有两个问题需要澄清：

1. 给定某时刻的 $\boldsymbol{R}$，我们就能求得一个 $\boldsymbol{\phi}$，它描述了 $\boldsymbol{R}$ 在局部的导数关系。与 $\boldsymbol{R}$ 对应的 $\boldsymbol{\phi}$ 有什么含义呢？我们说，$\boldsymbol{\phi}$ 正是对应到 SO(3) 上的李代数 $\mathfrak{so}(3)$；
2. 其次，给定某个向量 $\boldsymbol{\phi}$ 时，矩阵指数 $\exp(\boldsymbol{\phi}^\wedge)$ 如何计算？反之，给定 $\boldsymbol{R}$ 时，能否有相反的运算来计算 $\boldsymbol{\phi}$？事实上，这正是李群与李代数间的指数/对数映射。

下面，我们来解决这两个问题。

4.1.3 李代数的定义

每个李群都有与之对应的李代数。李代数描述了李群的局部性质，准确地说，是单位元附近的正切空间。一般的李代数的定义如下：

李代数由一个集合 $\mathbb{V}$、一个数域 $\mathbb{F}$ 和一个二元运算 $[,]$ 组成。如果它们满足以下几条性质，则称 $(\mathbb{V}, \mathbb{F}, [,])$ 为一个李代数，记作 $\mathfrak{g}$。

1. 封闭性　$\forall \boldsymbol{X}, \boldsymbol{Y} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{Y}] \in \mathbb{V}$.
2. 双线性　$\forall \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{Z} \in \mathbb{V}, a, b \in \mathbb{F}$, 有

$$[a\boldsymbol{X} + b\boldsymbol{Y}, \boldsymbol{Z}] = a[\boldsymbol{X}, \boldsymbol{Z}] + b[\boldsymbol{Y}, \boldsymbol{Z}], \quad [\boldsymbol{Z}, a\boldsymbol{X} + b\boldsymbol{Y}] = a[\boldsymbol{Z}, \boldsymbol{X}] + b[\boldsymbol{Z}, \boldsymbol{Y}].$$

3. 自反性[②]　$\forall \boldsymbol{X} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{X}] = \boldsymbol{0}$.
4. 雅可比等价　$\forall \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{Z} \in \mathbb{V}, [\boldsymbol{X}, [\boldsymbol{Y}, \boldsymbol{Z}]] + [\boldsymbol{Z}, [\boldsymbol{X}, \boldsymbol{Y}]] + [\boldsymbol{Y}, [\boldsymbol{Z}, \boldsymbol{X}]] = \boldsymbol{0}$.

其中二元运算被称为**李括号**。从表面上看，李代数所需要的性质还是挺多的。相比于群中的较为简单的二元运算，李括号表达了两个元素的差异。它不要求结合律，而要求元素和自己做李括号之后为零的性质。作为例子，三维向量 $\mathbb{R}^3$ 上定义的叉积 $\times$ 是一种李括号，因此 $\mathfrak{g} = (\mathbb{R}^3, \mathbb{R}, \times)$ 构成了一个李代数。读者可以尝试将叉积的性质代入上面四条性质中。

4.1.4 李代数 $\mathfrak{so}(3)$

之前提到的 $\boldsymbol{\phi}$，事实上是一种李代数。SO(3) 对应的李代数是定义在 $\mathbb{R}^3$ 上的向量，我们记作 $\boldsymbol{\phi}$。根据前面的推导，每个 $\boldsymbol{\phi}$ 都可以生成一个反对称矩阵：

$$\boldsymbol{\Phi} = \boldsymbol{\phi}^\wedge = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix} \in \mathbb{R}^{3\times 3}. \tag{4.11}$$

①此时，我们还没有说明 exp 是如何起作用的。我们马上会看到它的定义和计算过程。
②自反性是指自己与自己的运算为零。

在此定义下，两个向量 $\boldsymbol{\phi}_1, \boldsymbol{\phi}_2$ 的李括号为

$$[\boldsymbol{\phi}_1, \boldsymbol{\phi}_2] = (\boldsymbol{\Phi}_1 \boldsymbol{\Phi}_2 - \boldsymbol{\Phi}_2 \boldsymbol{\Phi}_1)^\vee . \tag{4.12}$$

读者可以验证该定义下的李括号是否满足上面的四条性质。由于向量 $\boldsymbol{\phi}$ 与反对称矩阵是一一对应的，在不引起歧义的情况下，就说 $\mathfrak{so}(3)$ 的元素是三维向量或者三维反对称矩阵，不加区别：

$$\mathfrak{so}(3) = \left\{ \boldsymbol{\phi} \in \mathbb{R}^3, \boldsymbol{\Phi} = \boldsymbol{\phi}^\wedge \in \mathbb{R}^{3\times 3} \right\}. \tag{4.13}$$

有些书里也会用 $\hat{\boldsymbol{\phi}}$ 这样的符号表示反对称，但意义是一样的。至此，我们已清楚了 $\mathfrak{so}(3)$ 的内容。它们是一个由**三维向量**组成的集合，每个向量对应一个反对称矩阵，可以用于表达旋转矩阵的导数。它与 SO(3) 的关系由指数映射给定：

$$\boldsymbol{R} = \exp(\boldsymbol{\phi}^\wedge). \tag{4.14}$$

指数映射会在稍后介绍。由于已经介绍了 $\mathfrak{so}(3)$，我们顺带先来看 SE(3) 上对应的李代数。

4.1.5　李代数 $\mathfrak{se}(3)$

对于 SE(3)，它也有对应的李代数 $\mathfrak{se}(3)$。为节省篇幅，这里就不介绍如何引出 $\mathfrak{se}(3)$ 了。与 $\mathfrak{so}(3)$ 相似，$\mathfrak{se}(3)$ 位于 $\mathbb{R}^6$ 空间中：

$$\mathfrak{se}(3) = \left\{ \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\phi} \end{bmatrix} \in \mathbb{R}^6, \boldsymbol{\rho} \in \mathbb{R}^3, \boldsymbol{\phi} \in \mathfrak{so}(3), \boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^{\mathrm{T}} & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4} \right\}. \tag{4.15}$$

我们把每个 $\mathfrak{se}(3)$ 元素记作 $\boldsymbol{\xi}$，它是一个六维向量。前三维为平移（但含义与变换矩阵中的平移不同，分析见后），记作 $\boldsymbol{\rho}$；后三维为旋转，记作 $\boldsymbol{\phi}$，实质上是 $\mathfrak{so}(3)$ 元素[①]。同时，我们拓展了 $\wedge$ 符号的含义。在 $\mathfrak{se}(3)$ 中，同样使用 $\wedge$ 符号，将一个六维向量转换成四维矩阵，但这里不再表示反对称：

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^{\mathrm{T}} & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}. \tag{4.16}$$

我们仍使用 $\wedge$ 和 $\vee$ 符号指代“从向量到矩阵”和“从矩阵到向量”的关系，以保持和 $\mathfrak{so}(3)$ 上的一致性。它们依旧是一一对应的。读者可以简单地把 $\mathfrak{se}(3)$ 理解成“由一个平移加上一个 $\mathfrak{so}(3)$ 元素构成的向量”（尽管这里的 $\boldsymbol{\rho}$ 还不直接是平移）。同样，李代数 $\mathfrak{se}(3)$ 也有类似于 $\mathfrak{so}(3)$ 的李

①请注意有些地方把旋转放前面，平移放后面，也是可行的。在程序里则无所谓前后，它们都存储在一个结构体中。

括号：

$$[\boldsymbol{\xi}_1, \boldsymbol{\xi}_2] = (\boldsymbol{\xi}_1^\wedge \boldsymbol{\xi}_2^\wedge - \boldsymbol{\xi}_2^\wedge \boldsymbol{\xi}_1^\wedge)^\vee . \tag{4.17}$$

读者可以验证它是否满足李代数的定义（留作习题）。至此，我们已经见过两种重要的李代数 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$ 了。

4.2 指数与对数映射

4.2.1 SO(3) 上的指数映射

现在来考虑第二个问题：如何计算 $\exp(\boldsymbol{\phi}^\wedge)$？显然它是一个矩阵的指数，在李群和李代数中，称为指数映射（Exponential Map）。同样，我们会先讨论 $\mathfrak{so}(3)$ 的指数映射，再讨论 $\mathfrak{se}(3)$ 的情形。

任意矩阵的指数映射可以写成一个泰勒展开，但是只有在收敛的情况下才会有结果，其结果仍是一个矩阵：

$$\exp(\boldsymbol{A}) = \sum_{n=0}^{\infty} \frac{1}{n!} \boldsymbol{A}^n. \tag{4.18}$$

同样地，对 $\mathfrak{so}(3)$ 中的任意元素 $\boldsymbol{\phi}$，我们也可按此方式定义它的指数映射：

$$\exp(\boldsymbol{\phi}^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} (\boldsymbol{\phi}^\wedge)^n. \tag{4.19}$$

但这个定义没法直接计算，因为我们不想计算矩阵的无穷次幂。下面我们推导一种计算指数映射的简便方法。由于 $\boldsymbol{\phi}$ 是三维向量，我们可以定义它的模长和方向，分别记作 θ 和 $\boldsymbol{a}$，于是有 $\boldsymbol{\phi} = \theta \boldsymbol{a}$。这里 $\boldsymbol{a}$ 是一个长度为 1 的方向向量，即 $\|\boldsymbol{a}\| = 1$。首先，对于 $\boldsymbol{a}^\wedge$，有以下两条性质：

$$\boldsymbol{a}^\wedge \boldsymbol{a}^\wedge = \begin{bmatrix} -a_2^2 - a_3^2 & a_1 a_2 & a_1 a_3 \\ a_1 a_2 & -a_1^2 - a_3^2 & a_2 a_3 \\ a_1 a_3 & a_2 a_3 & -a_1^2 - a_2^2 \end{bmatrix} = \boldsymbol{a}\boldsymbol{a}^\mathrm{T} - \boldsymbol{I}, \tag{4.20}$$

以及

$$\boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge = \boldsymbol{a}^\wedge (\boldsymbol{a}\boldsymbol{a}^\mathrm{T} - \boldsymbol{I}) = -\boldsymbol{a}^\wedge. \tag{4.21}$$

这两个式子提供了处理 $\boldsymbol{a}^\wedge$ 高阶项的方法。我们可以把指数映射写成

$$\exp(\boldsymbol{\phi}^\wedge) = \exp(\theta \boldsymbol{a}^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} (\theta \boldsymbol{a}^\wedge)^n$$

$$
\begin{aligned}
&= \boldsymbol{I} + \theta\boldsymbol{a}^\wedge + \frac{1}{2!}\theta^2\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge + \frac{1}{3!}\theta^3\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge + \frac{1}{4!}\theta^4(\boldsymbol{a}^\wedge)^4 + \cdots \\
&= \boldsymbol{a}\boldsymbol{a}^\mathrm{T} - \boldsymbol{a}^\wedge\boldsymbol{a}^\wedge + \theta\boldsymbol{a}^\wedge + \frac{1}{2!}\theta^2\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge - \frac{1}{3!}\theta^3\boldsymbol{a}^\wedge - \frac{1}{4!}\theta^4(\boldsymbol{a}^\wedge)^2 + \cdots \\
&= \boldsymbol{a}\boldsymbol{a}^\mathrm{T} + \underbrace{\left(\theta - \frac{1}{3!}\theta^3 + \frac{1}{5!}\theta^5 - \cdots\right)}_{\sin\theta}\boldsymbol{a}^\wedge - \underbrace{\left(1 - \frac{1}{2!}\theta^2 + \frac{1}{4!}\theta^4 - \cdots\right)}_{\cos\theta}\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge \\
&= \boldsymbol{a}^\wedge\boldsymbol{a}^\wedge + \boldsymbol{I} + \sin\theta\boldsymbol{a}^\wedge - \cos\theta\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge \\
&= (1 - \cos\theta)\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge + \boldsymbol{I} + \sin\theta\boldsymbol{a}^\wedge \\
&= \cos\theta\boldsymbol{I} + (1 - \cos\theta)\boldsymbol{a}\boldsymbol{a}^\mathrm{T} + \sin\theta\boldsymbol{a}^\wedge.
\end{aligned}
$$

最后，得到一个似曾相识的式子：

$$
\exp(\theta\boldsymbol{a}^\wedge) = \cos\theta\boldsymbol{I} + (1 - \cos\theta)\boldsymbol{a}\boldsymbol{a}^\mathrm{T} + \sin\theta\boldsymbol{a}^\wedge. \tag{4.22}
$$

回想第 3 讲的内容，它和罗德里格斯公式，即式 (3.15) 如出一辙。这表明，$\mathfrak{so}(3)$ 实际上就是由所谓的**旋转向量**组成的空间，而指数映射即罗德里格斯公式。通过它们，我们把 $\mathfrak{so}(3)$ 中任意一个向量对应到了一个位于 SO(3) 中的旋转矩阵。反之，如果定义对数映射，也能把 SO(3) 中的元素对应到 $\mathfrak{so}(3)$ 中：

$$
\boldsymbol{\phi} = \ln(\boldsymbol{R})^\vee = \left(\sum_{n=0}^{\infty}\frac{(-1)^n}{n+1}(\boldsymbol{R} - \boldsymbol{I})^{n+1}\right)^\vee. \tag{4.23}
$$

和指数映射一样，我们没必要直接用泰勒展开计算对数映射。在第 3 讲中，我们介绍过如何根据旋转矩阵计算对应的李代数，即使用式 (3.17)，利用迹的性质分别求解转角和转轴，采用这种方式更省事。

现在，我们介绍了指数映射的计算方法。读者可能会问，指数映射有何性质呢？是否对于任意的 $\boldsymbol{R}$ 都能找到一个唯一的 $\boldsymbol{\phi}$？很遗憾，指数映射只是一个满射，并不是单射。这意味着每个 SO(3) 中的元素，都可以找到一个 $\mathfrak{so}(3)$ 元素与之对应；但是可能存在多个 $\mathfrak{so}(3)$ 中的元素，对应到同一个 SO(3)。至少对于旋转角 θ，我们知道多转 360° 和没有转是一样的——它具有周期性。但是，如果我们把旋转角度固定在 $\pm\pi$ 之间，那么李群和李代数元素是一一对应的。

SO(3) 与 $\mathfrak{so}(3)$ 的结论似乎在我们的意料之中。它和我们前面讲的旋转向量与旋转矩阵很相似，而指数映射即罗德里格斯公式。旋转矩阵的导数可以由旋转向量指定，指导着如何在旋转矩阵中进行微积分运算。

4.2.2 SE(3) 上的指数映射

下面介绍 $\mathfrak{se}(3)$ 上的指数映射。为了节省篇幅，我们不再像 $\mathfrak{so}(3)$ 那样详细推导指数映射。$\mathfrak{se}(3)$ 上的指数映射形式如下：

$$\exp\left(\boldsymbol{\xi}^{\wedge}\right)=\begin{bmatrix}\sum_{n=0}^{\infty}\frac{1}{n!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n} & \sum_{n=0}^{\infty}\frac{1}{(n+1)!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n}\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix} \tag{4.24}$$

$$\triangleq\begin{bmatrix}\boldsymbol{R} & \boldsymbol{J}\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix}=\boldsymbol{T}. \tag{4.25}$$

只要有一点耐心，可以照着 $\mathfrak{so}(3)$ 上的做法推导，把 exp 进行泰勒展开推导此式。令 $\boldsymbol{\phi}=\theta\boldsymbol{a}$，其中 $\boldsymbol{a}$ 为单位向量，则

$$\begin{aligned}\sum_{n=0}^{\infty}\frac{1}{(n+1)!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n}&=\boldsymbol{I}+\frac{1}{2!}\theta\boldsymbol{a}^{\wedge}+\frac{1}{3!}\theta^{2}\left(\boldsymbol{a}^{\wedge}\right)^{2}+\frac{1}{4!}\theta^{3}\left(\boldsymbol{a}^{\wedge}\right)^{3}+\frac{1}{5!}\theta^{4}\left(\boldsymbol{a}^{\wedge}\right)^{4}\cdots\\&=\frac{1}{\theta}\left(\frac{1}{2!}\theta^{2}-\frac{1}{4!}\theta^{4}+\cdots\right)\left(\boldsymbol{a}^{\wedge}\right)+\frac{1}{\theta}\left(\frac{1}{3!}\theta^{3}-\frac{1}{5}\theta^{5}+\cdots\right)\left(\boldsymbol{a}^{\wedge}\right)^{2}+\boldsymbol{I}\\&=\frac{1}{\theta}\left(1-\cos\theta\right)\left(\boldsymbol{a}^{\wedge}\right)+\frac{\theta-\sin\theta}{\theta}\left(\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}-\boldsymbol{I}\right)+\boldsymbol{I}\\&=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}\overset{\text{def}}{=}\boldsymbol{J}.\end{aligned} \tag{4.26}$$

从结果上看，$\boldsymbol{\xi}$ 的指数映射左上角的 $\boldsymbol{R}$ 是我们熟知的 SO(3) 中的元素，与 $\mathfrak{se}(3)$ 中的旋转部分 $\boldsymbol{\phi}$ 对应。而右上角的 $\boldsymbol{J}$ 由上面的推导给出：

$$\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}. \tag{4.27}$$

该式与罗德里格斯公式有些相似，但不完全一样。我们看到，平移部分经过指数映射之后，发生了一次以 $\boldsymbol{J}$ 为系数矩阵的线性变换。请读者重视这里的 $\boldsymbol{J}$，因为后面还要用到。

同样地，虽然我们也可以类比推得对数映射，不过根据变换矩阵 $\boldsymbol{T}$ 求 $\mathfrak{so}(3)$ 上的对应向量也有更省事的方式：从左上角的 $\boldsymbol{R}$ 计算旋转向量，而右上角的 $\boldsymbol{t}$ 满足：

$$\boldsymbol{t}=\boldsymbol{J}\boldsymbol{\rho}. \tag{4.28}$$

由于 $\boldsymbol{J}$ 可以由 $\boldsymbol{\phi}$ 得到，所以这里的 $\boldsymbol{\rho}$ 也可由此线性方程解得。现在，我们已经弄清了李群、

李代数的定义与相互的转换关系，总结如图 4-1 所示。如果读者有哪里不明白，可以翻回去看看公式推导。

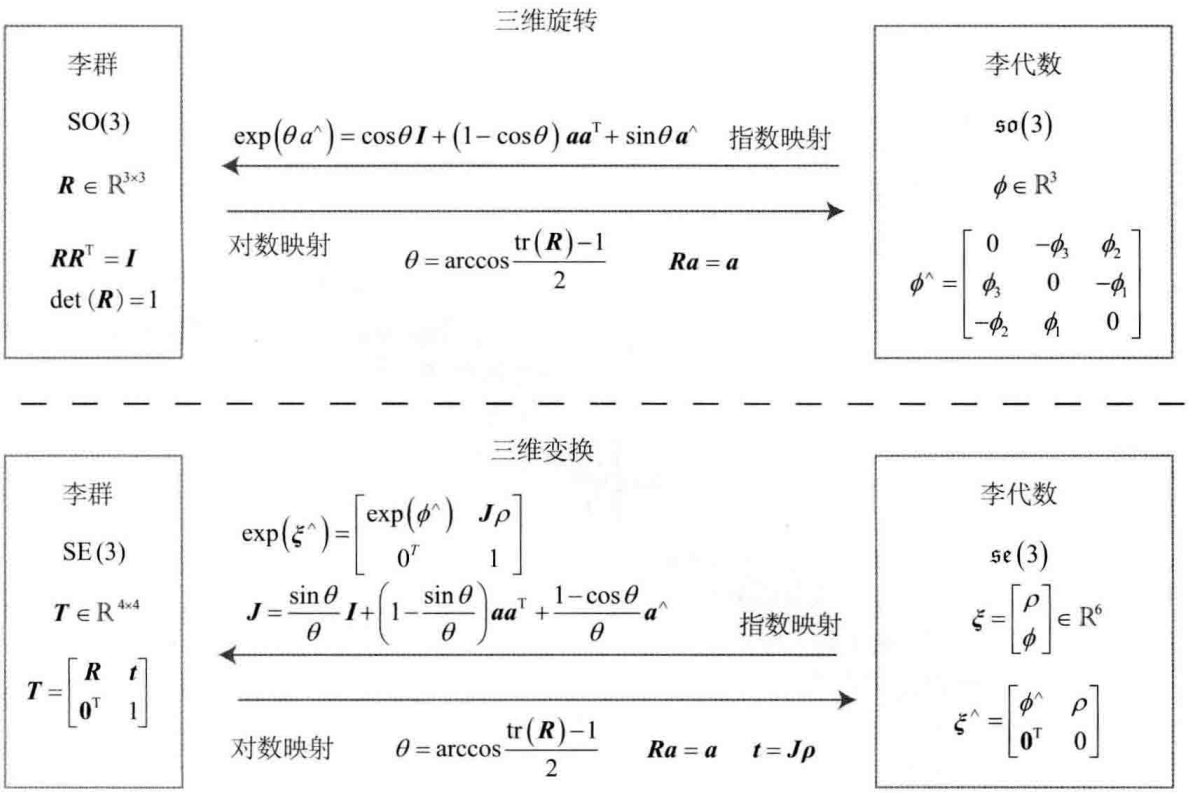


图 4-1　$\mathrm{SO}(3), \mathrm{SE}(3), \mathfrak{so}(3), \mathfrak{se}(3)$ 的对应关系

4.3　李代数求导与扰动模型

4.3.1　BCH 公式与近似形式

使用李代数的一大动机是进行优化，而在优化过程中导数是非常必要的信息（我们会在第 6 讲详细介绍）。下面来考虑一个问题。虽然我们已经清楚了 SO(3) 和 SE(3) 上的李群与李代数关系，但是，当在 SO(3) 中完成两个矩阵乘法时，李代数中 $\mathfrak{so}(3)$ 上发生了什么改变呢？反过来说，当 $\mathfrak{so}(3)$ 上做两个李代数的加法时，SO(3) 上是否对应着两个矩阵的乘积？如果成立，相当于：

$$\exp(\boldsymbol{\phi}_1^{\wedge})\exp(\boldsymbol{\phi}_2^{\wedge}) = \exp\left((\boldsymbol{\phi}_1 + \boldsymbol{\phi}_2)^{\wedge}\right)?$$

如果 $\boldsymbol{\phi}_1, \boldsymbol{\phi}_2$ 为标量，那么显然该式成立；但此处我们计算的是**矩阵**的指数函数，而非标量的

指数。换言之，我们在研究下式是否成立：

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}\ ?$$

很遗憾，该式在矩阵时并不成立。两个李代数指数映射乘积的完整形式，由 Baker-Campbell-Hausdorff 公式（BCH 公式）[①]给出。由于其完整形式较复杂，我们只给出其展开式的前几项：

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}+\frac{1}{2}\left[\boldsymbol{A},\boldsymbol{B}\right]+\frac{1}{12}\left[\boldsymbol{A},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]-\frac{1}{12}\left[\boldsymbol{B},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]+\cdots \tag{4.29}$$

其中 [ ] 为李括号。BCH 公式告诉我们，当处理两个矩阵指数之积时，它们会产生一些由李括号组成的余项。特别地，考虑 SO(3) 上的李代数 $\ln\left(\exp\left(\boldsymbol{\phi}_1^{\wedge}\right)\exp\left(\boldsymbol{\phi}_2^{\wedge}\right)\right)^{\vee}$，当 $\boldsymbol{\phi_1}$ 或 $\boldsymbol{\phi_2}$ 为小量时，小量二次以上的项都可以被忽略。此时，BCH 拥有线性近似表达[②]：

$$\ln\left(\exp\left(\boldsymbol{\phi}_1^{\wedge}\right)\exp\left(\boldsymbol{\phi}_2^{\wedge}\right)\right)^{\vee}\approx\begin{cases}\boldsymbol{J}_l\left(\boldsymbol{\phi}_2\right)^{-1}\boldsymbol{\phi}_1+\boldsymbol{\phi}_2 & \text{当}\boldsymbol{\phi}_1\text{为小量,}\\ \boldsymbol{J}_r\left(\boldsymbol{\phi}_1\right)^{-1}\boldsymbol{\phi}_2+\boldsymbol{\phi}_1 & \text{当}\boldsymbol{\phi}_2\text{为小量.}\end{cases} \tag{4.30}$$

以第一个近似为例。该式告诉我们，当对一个旋转矩阵 $\boldsymbol{R}_2$（李代数为 $\boldsymbol{\phi}_2$）左乘一个微小旋转矩阵 $\boldsymbol{R}_1$（李代数为 $\boldsymbol{\phi}_1$）时，可以近似地看作，在原有的李代数 $\boldsymbol{\phi}_2$ 上加上了一项 $\boldsymbol{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1$。同理，第二个近似描述了右乘一个微小位移的情况。于是，李代数在 BCH 近似下，分成了左乘近似和右乘近似两种，在使用时我们须注意使用的是左乘模型还是右乘模型。

本书以左乘为例。左乘 BCH 近似雅可比 $\boldsymbol{J}_l$ 事实上就是式 (4.27) 的内容：

$$\boldsymbol{J}_l=\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}. \tag{4.31}$$

它的逆为

$$\boldsymbol{J}_l^{-1}=\frac{\theta}{2}\cot\frac{\theta}{2}\boldsymbol{I}+\left(1-\frac{\theta}{2}\cot\frac{\theta}{2}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}-\frac{\theta}{2}\boldsymbol{a}^{\wedge}. \tag{4.32}$$

而右乘雅可比仅需要对自变量取负号即可：

$$\boldsymbol{J}_r(\boldsymbol{\phi})=\boldsymbol{J}_l(-\boldsymbol{\phi}). \tag{4.33}$$

这样，我们就可以谈论李群乘法与李代数加法的关系了。

为了方便读者理解，我们重新叙述 BCH 近似的意义。假定对某个旋转 $\boldsymbol{R}$，对应的李代数为

---

①参见https://en.wikipedia.org/wiki/Baker-Campbell-Hausdorff_formula。

②对 BCH 具体形式和近似表达的具体推导，本书不作讨论，请参考文献 [6]。